KH정보교육원

SQL 응용

목 차

Part 1. 절차형 SQL 작성하기

Chapter 1. 데이터베이스 모델링 개념 ····· 4
Chapter 2. 논리 데이터 모델 생성하기 ····· 6
Chapter 3. 데이터 아키텍처 이해하기 ····· 42
Chapter 4. 물리 데이터 모델 생성하기 ····· 43

Part 2. 응용 SQL 작성하기

Chapter 5. JDBC의 이해 ····· 74
Chapter 6. JDBC 구현 순서 ····· 76
Chapter 7. JDBC CRUD ····· 79
Chapter 8. JDBC Statement와 PreparedStatement ··· 87
Chapter 9. JDBC Connection Pool ····· 89
Chapter 10. JDBC 공통모듈 구현하기 ····· 91
Chapter 11. 공공데이터 PROCEDURE 적용하기 ····· 94

Appendix. 이클립스 디버깅 사용법 ····· 104

Part 1. 절차형 SQL 작성하기

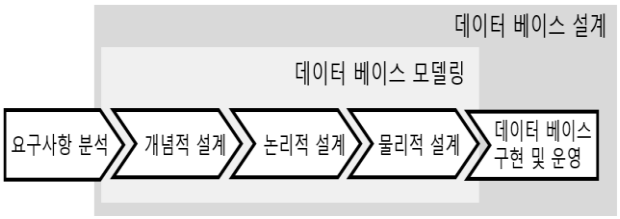
Chapter 1. 데이터 모델링 개념

1. 데이터 모델링의 이해

1.1 데이터 모델링이란

데이터 모델링이란 어떠한 프로젝트에 있어 필요한 데이터들을 분석하여 정의하고 해당 프로젝트에 맞게 절차를 구성하는 것을 말한다. 이 과정은 크게 다음과 같은 절차로 이루어 진다. 먼저 고객의 요구사항을 분석하여 데이터들을 개체(Entity)와 속성, 관계로 명확화하는 과정이 필요하며, 그에 따른 요소를 사용하여 각각의 과정을 설계해 나가야 한다.

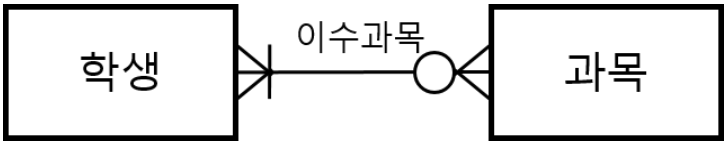
- **개체(Entity) :** 사람, 장소, 물건, 사건, 개념 등의 명사에 해당하며, 객체의 의미를 지닌다.
- **속성 :** 개체(Entity)가 가지는 항목으로 사람의 이름, 나이 등이 이에 해당한다.
- **관계 :** 부서와 직원의 1:N 관계 등 각 개체(Entity) 간의 상관관계를 의미한다.



1.2 개념적 설계 방법

개념적 데이터 설계는 요구분석 단계에서 정의된 핵심 개체(Entity)와 그들 간의 관계를 바탕으로 아래와 같은 개체(Entity)-관계 다이어그램(ERD)을 생성하는 단계이다.

다음 이미지는 학생이라는 개체(Entity)와 과목이라는 개체(Entity)로 관계를 두어 학생은 0~N 개의 과목, 과목은 1~N 명의 학생에 대하여 이수과목 관계를 ERD 로 표현한 것이다.



이렇게 추상화가 적용된 ER 다이어그램은 대상이 되는 조직이나 다양한 고객에게 다음과 같은 두 가지 기능을 제공한다.

첫째, 고객과 개발자간의 데이터 토론이 쉽게 이루어진다. 개념 데이터 모델은 추상적이기 때문에, 시스템의 구조화를 알기 쉬우며, 이에 따라 고객과 개발자가 시스템 기반에 대한 논의를 할 수 있는 기반을 형성한다.

둘째, 개념 데이터 모델은 데이터 구체화의 틀을 제공한다. 데이터 구조가 복잡한 시스템 일지라도 추상적 모델링을 통해 보다 쉽게 표현하고 설명할 수 있기 때문이다. 이 과정을 통해 개념적 스키마가 도출된다.

1.3 논리적 설계 방법

논리 데이터 모델링은 개념 설계에서 추상화된 데이터를 구체화하여 개체(Entity), 속성을 테이블화하고 상세화하는 과정이라 할 수 있다. 보통 데이터 모델링이라 하면 이 과정을 의미하며, 개념적 설계를 통해 도출된 구조를 컴퓨터가 이해할 수 있도록 컴퓨터 환경에 맞춰 변환하는 과정을 말한다. 논리적 데이터 모델링은 데이터 간의 관계를 어떻게 표현하느냐에 따라 관계 모델, 계층 모델, 네트워크 모델로 세분화 할 수 있다.

⊙ 관계형 데이터 모델

2차원적인 표를 이용하여 데이터 간의 관계를 정의하는 구조로, 간단하고 균등한 데이터 구조를 가진다. 하지만 각 관계를 모두 엮어 주어야 하므로, 성능이 다소 떨어질 수 있다.

⊙ 계층형 데이터 모델

데이터의 논리적 구조를 트리 형태로 만들어 개체(Entity)를 노드로 하여 각 관계를 링크로 연결하는 형태이다. 트리 구조 특성상 검색, 수정에 용이한 형태이나 데이터 상호 간 유연성이 부족하여 다 대 다 관계를 처리하기 어렵다.

⊙ 네트워크형(망형) 데이터 모델

그래프를 이용하여 데이터 논리 구조를 표현한 것으로 상위와 하위 레코드로 구성되어 있다. 복잡한 형태의 모델링에 용이하나 설계가 어렵고, 사용자가 이해하기 어렵다는 단점이 있다.

1.4 물리적 설계 방법

논리적 설계 단계에서 표현된 데이터를 실제 컴퓨터의 저장장치에 어떻게 표현할 것인가를 다룬다. 이를 물리적 스키마라고도 하는데, 이 단계에서 테이블, 칼럼 들을 구성하며 이를 구현하기 위해서 많은 시간이 소요된다. 따라서 보통 논리적 설계 이후에 작업하지 않고 논리적 설계와 물리적 설계를 한꺼번에 모델링으로 수행한다.

Chapter 2. 논리 데이터 모델 생성하기

1 논리 데이터 모델링 정의

논리적 데이터 모델링이란 데이터베이스 설계 프로세스의 Input 으로써 비즈니스 정보의 구조와 규칙을 명확하게 표현하는 기법이다. 논리적 모델은 데이터 모델링이 최종적으로 완료된 상태를 말한다. 즉, 물리적인 스키마 설계를 하기 전단계의 데이터 모델 상태를 일컫는 말이다.

논리적 데이터 모델링의 핵심은 어떻게 데이터에 액세스하며, 누가 데이터에 액세스하며, 그러한 액세스의 전산화와는 독립적으로 비즈니스 데이터에 존재하는 사실을 인식·기록하는 기법일 뿐만 아니라 철학이다.

특히 데이터 모델링 과정에서 가장 핵심이 되는 부분이 논리 데이터 모델링이라고 할 수 있다. 데이터 모델링이란 모델링 과정이 아닌 별도의 과정을 통해서 조사하고 결정한 사실을 단지 개체-관계 다이어그램(ERD, Entity Relationship Diagram)이라는 그림으로 그려내는 과정을 말하는 것이 아니다. 시스템 구축을 위해서 가장 먼저 시작할 기초적인 업무조사를 하는 초기 단계부터 인간이 결정해야 할 대부분의 사항을 모두 정의하는 시스템 설계의 전 과정을 지원하는 과정의 도구라고 해야 할 것이다.

1.1 논리 데이터 모델링 목적 및 효과

해당 비즈니스에 대한 데이터 관점에서의 명확한 이해

데이터 모델링을 한마디로 하면 업무의 데이터 관점에서의 표현 또는 설계라고 할 수 있다.

전사적인 통합 데이터 체계 확립

논리 데이터 모델링을 통하여 전사의 데이터에 대한 구조를 체계화하고 이를 통한 통합 데이터 체계를 확립한다.

데이터의 일관성 및 정확성 유지를 위한 규칙 도출

기업이 관리하는 데이터의 일관성, 정확성을 유지하기 위해서는 데이터에 대한 정확한 업무 규칙과 데이터 처리 규칙들을 생성 관리해야 한다. 이것을 표현하고 관리하는 것은 논리 데이터 모델을 통해서 하게 된다.

안정적인 데이터베이스 설계의 토대 마련

논리 데이터 모델을 통하여 물리 데이터 모델이 생성된다. 특히 데이터 모델의 구체적인 실체(객체)인 데이터베이스를 안정적이고 체계적으로 생성하는 기초가 된다.

사용자와의 명확한 의사소통을 위한 수단으로 활용

시스템 설계자와 업무 담당자 간의 의사소통의 수단으로 논리 데이터 모델이 사용된다. 반면 설계자와 개발자 간의 의사 소통의 수단은 물리 데이터 모델이라고 볼 수 있다. 하지만 이 과정에서 개발자라고 하더라도 논리 데이터 모델에 표현되어 있는 비즈니스 규칙에 대한 이해가 선행되어야 한다.

1.2 논리 데이터 모델링 필수 성공요소

업무에 능통한 현업 사용자와 함께 데이터 모델링을 진행하라

데이터 모델링 작업은 결국 업무를 데이터 관점에서 체계화하는 작업이다. 즉, 비즈니스에서 사용되는 데이터를 집합(엔터티)으로 생성하고 이들 간의 관계(Relationship)를 지정하는 작업이다. 이러한 작업을 하는 과정에서 업무를 알고 있는 현업 사용자의 참여는 필수적이다.

절차(Procedure)보다는 데이터에 초점을 두고 모델링을 진행하라

데이터 모델에는 순서와 시간, 흐름의 개념이 들어가지 않는다. 이러한 개념을 넣어서 마치 데이터 흐름도(DFD, Data Flow Diagram)처럼 업무의 흐름에 따라 엔터티가 발생하는 경우를 실전에서는 많이 보게 된다. 이런 식의 데이터 모델에서는 많은 데이터의 중복과 데이터 정합성을 훼손할 개연성을 내포한 모델이 만들어지게 마련이다. 그렇기 때문에 가능한 절차를 배제한 채로 데이터 모델을 생성해야 한다.

데이터의 구조(Structure)와 무결성(Integrity)을 함께 고려하라

개념화(Conceptualization)와 정규화(Normalization) 기법을 적용하라

가능하면 다이어그램(Diagram)을 이용하여 업무를 표현하라

데이터 모델링을 지원하는 데이터 사전을 구축하라

속성 개념

속성 정의

속성은 엔터티에서 관리되는 구체적인 정보 항목으로 더 이상 분리될 수 없는 최소의 데이터 보관 단위이다. 예를 들어, 엔터티 사원에 속하는 모든 엔터티는 이름을 갖고 있다. 또한 모든 사원에는 입사일자, 사원번호, 생년월일 등의 특성을 가지고 있다. 엔터티 사원에 속하는 모든 인스턴스들이 공통으로 가지는 이러한 특성을 속성(Attribute)이라고 한다. 각 엔터티들은 일련의 속성들에 의해 상세화될 수 있다.

속성 특징

1) 속성의 어원적 의미

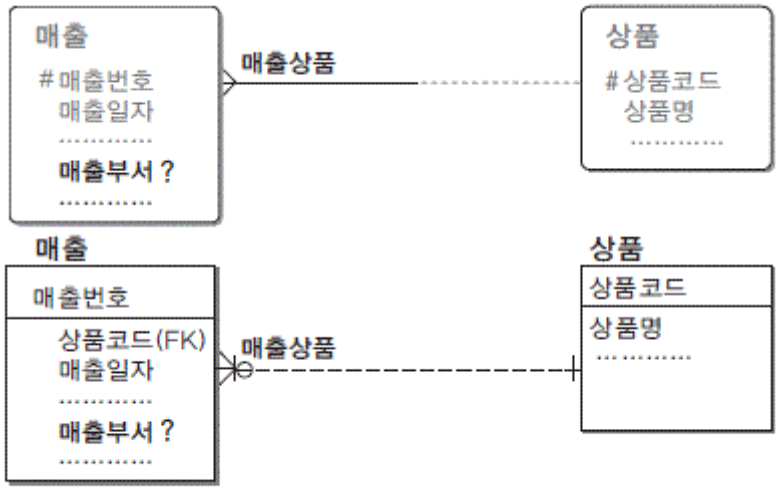
속성이라는 의미에는 가공되지 않은 것이라는 의미도 포함되어 있다. 이 말은 곧 원천적인 것을 의미한다. 또한 고유한 성질이란 의미도 가지고 있는데 이 말은 남의 도움을 받지 않더라도 자기만이 가지고 있는 독자적인 성질이 반드시 있어야 함을 뜻한다. 혼자서도 독자적인 성질을 가지고 있느냐에 대한 문제는 절대적이 아니라 상대적이라는 것 때문에 판단이 쉽지 않다. 즉 어떤 시스템의, 어떤 엔터티에서, 어떤 목적으로 사용하려고 하느냐에 따라서 달라지기 때문에 사람의 종합적인 판단이 필요하다는 것이다. 이러한 속성의 어원적인 특징들은 속성 검증의 원칙으로도 사용된다.

2) 속성도 일종의 집합이다.

물리 데이터 모델링 단계에서 엔터티는 테이블이 되고, 속성은 칼럼이 된다. 결국 속성에는 데이터 값이 들어가게 되며, 그 값들은 여러 종류를 가지게 된다. 가령, 사원 정보 엔터티의 직책 속성에는 사원, 대리, 과장, 부장 … 등 여러 종류의 값들이 존재한다. 만약 이 값들의 종류 하나 하나를 개체 라고 한다면, 직책이란 속성은 결국 이들을 구성요소로 하는 집합이란 뜻이 된다.

3) 릴레이션십도 속성이다.

물리 데이터 모델링 단계에 가서 보면 릴레이션십 또한 결국은 일종의 속성이 될 수 밖에 없다.



[그림 4-3-1] 바커 표기법(위)과 IE 표기법(아래)에 따른 매출 상품 모델 예

가) 매출 부서

[그림 4-3-1]에서 매출 부서는 엄격히 말해서 매출 엔터티의 속성이 아니라 부서 엔터티와의 릴레이션십이라고 하는 것이 정확한 표현이다. 사실 이론적으로 따진다면 속성은 전체 엔터티를 통틀어 반드시 유일하게 존재해야만 한다. 나 자신(속성)은 우리집(엔터티)에 속하고, 거기서 유일하지만 다른 모든 것(엔터티)들에 대해서도 반드시 유일한 존재이다.

매출 엔터티에 있는 매출 부서 속성에는 나중에 분명히 부서 코드가 들어온다. 부서 코드 속성이 오직 한 곳에만 위치해야 한다면 반드시 부서 엔터티에 있어야만 한다. 이 말은 다른 엔터티에 있는 모든 부서 코드는 사실 모두가 릴레이션십이라는 것을 의미한다. 이 릴레이션십은 나중에 데이터베이스 설계 단계에서 관계형 데이터베이스로 설계하고자 한다면 매출 테이블에 매출 부서라는 외래키 칼럼으로 존재하게 된다.

나) 매출 상품 릴레이션십

[그림 4-3-1]의 좌측에 있는 매출 상품 릴레이션십은 속성이 아닌 릴레이션십으로 제대로 표현 되어 있다. 설계 단계에서 결과적으로 동일해지는데 굳이 이것을 릴레이션십으로 표현하여 그림을 복잡하게 할 필요가 있겠는가?

물론, 원론적으로 본다면 이를 무조건 릴레이션십으로 표현하는 것이 옳다는 것에는 이견이 있을 수 없다. 왜냐하면 우리가 작성하는 논리적 데이터 모델이란 업무를 체계적으로 형상화하는 것이지 데이터베이스의 구조를 정의하는 것이 아니기 때문이다. 엄격히 말한다면 논리적 데이터 모델에는 데이터베이스나 컴퓨터와 같은 물리적 요소들은 전혀 감안하지 않은 상태에서 정의되어야 한다.

4) 속성들 간은 서로 독립적이다

속성들은 반드시 식별자에 직접 종속되어야 한다. 이 말은 정규화의 제 2 정규형에 해당하는 말이 다. 만약 속성들 간에 종속성이 존재한다면 이들은 별도의 엔터티로 분리되어야 한다. 이것이 바로 제 3 정규형이다.

속성 후보 도출

속성을 결정할 때에도 다양한 후보를 준비하고 이들 중에서 속성의 검증 규칙에 부합하는 것을 속성으로 최종 결정하게 된다. 특히 이러한 후보 도출 작업은 기존의 현장조사에 국한하지 않고 좀더 적극적이고 개선적인 사고를 가지고 사용자에게 많은 질문을 하고 확인해 속성 후보를 도출한다.

속성 후보 수집처

속성은 결국 속성 후보 중에서 선택되기 때문에 우리는 일단 다양한 경로를 통해서 좋은 후보를 가능하다면 최대한 많이 확보해야 한다.

1) 구 시스템의 문서자료

가동 중인 기존 시스템의 데이터 구조 및 프로세스 명세들이 나타나 있는 설계 자료에서부터 사용자를 위한 지침서에 이르기까지 다양한 도큐먼트가 있을 것이다. 이 자료는 엔터티 후보 및 속성 후 보를 도출하는데 가장 유용하게 사용할 수 있다.

2) 현업 장표/보고서

현업에서 사용하고 있는 각종 장표나 보고 자료들을 수집해서 조사해 보는 것이다. 현업 업무 중에는 업무를 효과적으로 처리하기 위하여 많은 장표와 각종 보고 자료를 만들고 있다. 물론 이들을 그대로 속성 후보로 선정할 수 있는 것은 결코 아니다. 그 중의 상당 부분은 다른 속성에 의해 만들어질 수 있는 가공된 결과(추출 속성, Derived Value)인 경우가 많기 때문이다. 더구나 이것들은 대부분 정규화가 되어 있지 않기 때문에 정확히 파악해야만 진정한 의미의 속성 후보를 찾아낼 수 있다.

3) 사용자와 협의

데이터 모델링 과정에서부터 시종일관 현업 담당자들과 같이 진행하는 것이 데이터를 모델링하는 최상의 방법이다.

4) DFD 의 DD(Data Dictionary)

업무 파악 및 시스템 분석을 위한 기능 설계로 자료 흐름도가 존재한다면 여기에 있는 데이터 저장소(Data Store)와 데이터 사전(Data Dictionary)에 있는 정보를 이용하여 속성의 후보를 도출할 수 있다. 자료 흐름도를 작성할 때 생성되는 자료 저장소는 비록 테이블처럼 표현되지만 사실은 이러한 내용의 데이터가 저장되어야 한다는 데이터의 추상화된 집합이라 할 수 있으며, 그 속에 들어가야할 구체적인 속성이 데이터 사전에 기술된다.

5) 전문 서적 및 자료

동일한 업무에 대한 전문 서적 또는 자료를 통해서도 속성의 후보를 도출할 수 있다.

6) 다른 시스템 자료

우리가 개발할 시스템의 주변을 살펴보면 사내외에 이와 관련된 시스템이나 유사한 시스템을 찾을 수 있을 것이다. 만약 여러분이 그들이 가지고 있던 속성 또한 후보로서 참조하는 것도 속성 후보를 찾는데 좋은 방법이다.

속성 후보 선정 원칙

속성의 후보를 선택할 때는 최대한 충분히 수집해야 한다. 그것은 검토한 결과 자신의 속성이 아닌것으로 판명될지라도 검토할 기회를 제공해 주는 단서라는 중요한 역할을 하기 때문이다.

1) 원시(Source) 속성으로 보이는 후보는 버리지 않는다

다른 속성에 의해 다시 재현할 수 있는 가공(추출, Derived) 속성이 아닌, 다시 말해서 만약 이 속성이 없다면 다시는 재현할 수가 없을 때 이 속성을 원시 속성이라고 부른다. 재현이 불가능하다면 이것을 버리는 순간, 이미 정보는 소실되어 버리므로 절대 버려서는 안된다.

2) 소그룹별로 후보군(Pool)을 만들고 가장 근접한 엔터티에 할당한다

모든 엔터티가 정의되어 있는 상태가 아니라 단지 핵심 엔터티들을 대상으로 모델링을 실시해왔을 뿐이므로 아직은 모든 엔터티가 드러나 있지는 않다. 그렇기 때문에 각 속성 후보들을 적절한 데이터 그룹으로 생성하여 두는 것이 필요하다.

속성의 기본 구성요소

1) 속성명

속성의 내용이나 목적이 무엇인지 알려주는 명사 또는 명사구이다. 기업에서 널리 사용하는 용어 를 쓴다.

- 속성의 의미를 명확히 표현하는 함축성 있는 명사 혹은 명사구를 사용한다.
- 해당 업무에서 일반적으로 사용하는 용어를 사용한다.
- 실체명은 속성명으로 사용하지 말아야 한다.
- 필요시 표준 약어를 제정하여 속성명을 생성하고 그 속성명을 단 하나의 실체에만 속하도록 하는 것이 바람직하다.

2) 도메인

속성이 지닐 수 있는 값에 대한 업무적인 제약 조건으로 파악된 일련의 특성이다. 모든 영역에서 같은 도메인을 사용하는 것이 좋다. 속성이 기존 도메인 집합에 속해 있지 않는 경우에는 새로운 도메인을 추가한다. 도메인은 다음과 같은 속성들을 가진다.

- 데이터 타입
- 길이
- 허용 값(Permitted Value): 속성에 지정할 수 있는 모든 값들의 집합
- 디폴트 값 및 디폴트 알고리즘

3) 선택성

모든 건의 해당 속성이 반드시 값을 가져야 하는지 여부를 나타낸다.

- 선택성 조건 => 선택성이 다른 속성 값에 의해 영향을 받는 경우
- 필요조건 / 금지조건 / 무관계조건

속성 검증 및 확정

다양한 경로를 통해 수집된 속성 후보를 엔터티에 배정시킨 후 이제 우리가 해야 할 일은 이들을 검증하여 제자리를 찾게 해 주는 일이다. 속성을 검증하는 작업은 총 4 단계로 나누어 실시한다.

최소 단위(Atomic Value)까지 분할하라

1) 검증 방법

- 집합 개념의 속성은 단순 개념으로 분할한다.
- 가능한 최소 단위까지 분할한 후 관리 필요에 따라 통합한다.
- 일자, 시간, 성명, 주민등록번호, 우편번호 등은 일반적으로 분할하지 않는 것이 좋다.
- 분할 및 통합의 기준은 업무의 요구 사항에 따른다.

2) 분할 속성의 대표적 유형

가) 일자(日字) 형태의 속성 : 매출일자

분리된 것들이 속성의 자격을 가지고 있는지를 검토하기 위해서는 이들이 자기 혼자서도 독립적 인 의미를 가지고 있는지 확인해 보아야 한다. 매출월(mm)이 매출년도(yyyy)나 매출일(dd)을 무 시하고 혼자서도 어떠한 의미를 가지고 있겠는가? 아마 대부분의 경우는 이들을 하나로 결합했을 때만 매출이 발생한 일자라는 의미를 가질 수 있을 것이므로 이런 경우라면 통합된 것이 속성이라고 보아야 한다.

	통합된 속성	분리된 속성
①	매출일자	매출년도 매출월 매출일
②	처리일시	처리일자 처리시간
③	우편번호	시군구 읍면동
④	전화번호	지역번호 국번 개별번호
⑤	전표번호	처리일자 일련번호
⑥	주소	지역주소 상세주소
⑦	발생일자	입고일자 출고일자 선적일자 하역일자 입문일자 출문일자

[그림 4-3-2] 분할 속성 예

나) 외부에서 공인된 속성 : 우편번호 유형

국가나 공공기관에 의해서 이미 공인되어 있는 각종 번호들(예; 우편번호, 주민등록번호, 사업자 등록번호, 법인번호, 여권번호 등)에 대한 속성 관리 형태는 매우 유사하다. 이들은 단지 길이에서 차이가 있지만 대부분 OOO-OOO 형식으로 나타난다. 우편번호의 앞의 세 자리는 마치 성명의 성과같이 나름대로 확실한 의미가 있다. 이 세자리 숫자는 반드시 단 하나의 시군구만 지칭하므로 분명한 의미가 있다는 것이다. 그러나 뒤의 세 자리는 앞의 시군구가 없으면 독자적으로 의미를 갖지는 못한다.

다) 전화번호 유형 : 전화, 팩스번호

전화번호는 지역번호(DDD)+국번+개별번호로 나누어진다. 일반적인 의미로 본다면 이들 각각 에는 분명히 독립적인 의미가 있다. 그러나 지금 우리가 검토하고자 하는 엔터티 내에서 과연 독립 적인 의미를 가지느냐는 또 다른 문제일 수 있다. 예를 들어, 고객 엔터티에 있는 고객의 연락전화 번호를 검토한다고 가정해보자. 국번만 독립적인 의미로 부여할 경우가 있는가? 뒷자리 4자리의 개별번호만 액세스하여 어떤 처리나 참조를 할 가치가 있는가? 지역번호만 독립적인 의미로 인정 하여 처리할 경우가 있겠는가? 물론 이러한 질문에 대한 대답은 그 엔터티의 내용이나 앞으로의 관리 수준에 많은 영향을 받는다.

라) 주소 유형 : 고객 주소

주소 형태로 관리되어야 하는 일단의 속성들은 어떤 시스템에서도 자주 등장하는 매우 중요한 속성의 한 유형이다. 주소를 세부적으로 나누는 방법은 여러 가지를 생각해 볼 수가 있다. 가령 도 (광역시) + 시군구 + 읍면동 + 단지(아파트,건물) + 동호수(번지)로 아주 세분화를 시킬 수도 있다. 크게 나누어 우편번호와 연결되어 사전에 이미 생성되어 있는 주소 부분(여기서는 지역주소라고 표현함)과 나머지 부분(상세주소로 표현)으로 분리하는 방법이다.

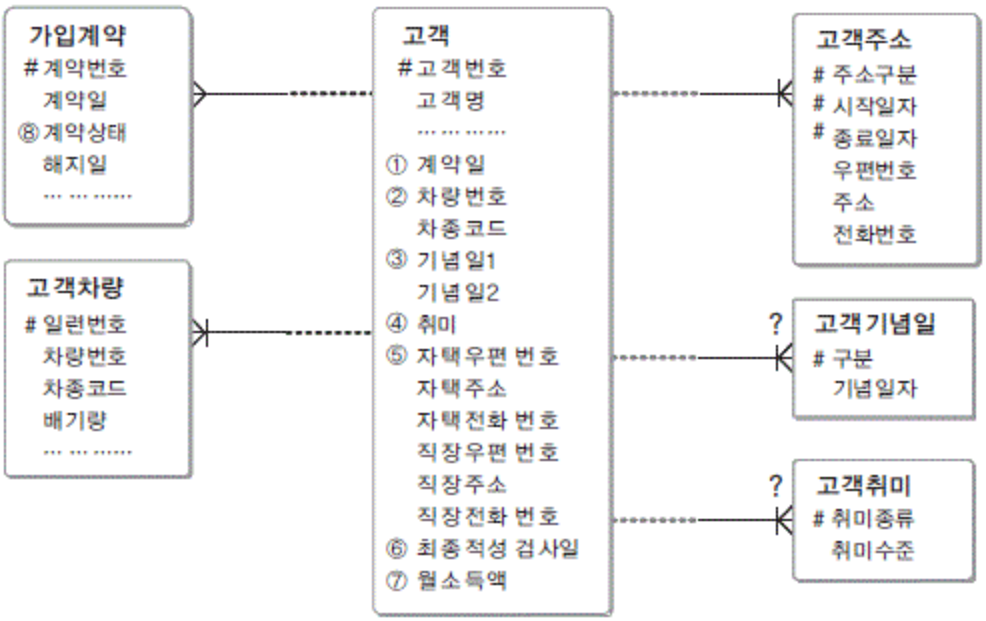
하나의 값(Single Value)만을 가지는지 검증한다

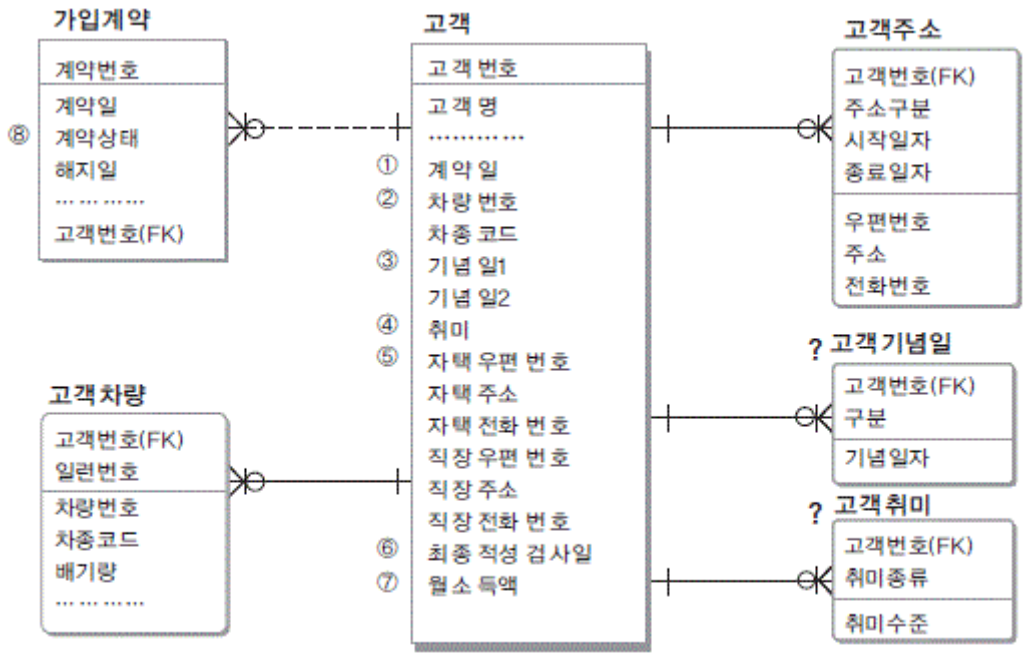
속성에서 관리되어야 할 값이 반드시 단 하나만 존재해야 한다는 것이다. 그러나 이 말의 진정한 의미는 그 속성에 들어올 수 있는 값의 종류가 반드시 하나만 존재하고 있다는 의미는 절대 아니다. 쉽게 설명한다면 그 엔터티에 들어가는 개체마다 반드시 하나의 값만 보유하고 있어야 한다는 것이다.

1) 검증 방법

- 여러 값을 가지거나 반복되는 속성은 잘못된 속성이다.
- 반복되는 속성은 새로운 엔터티로 분할해야 할 1 차 정규화의 대상이 된다.

2) 대표적 유형





[그림 4-3-3] 바커 표기법(위)과 IE 표기법(아래)에 따른 Single Value 예

가) 계약일

계약일은 고객 엔터티의 속성이 아니라 가입계약의 속성이다. 즉, 고객은 여러 개의 계약일을 가질 수 있기 때문에 고객의 속성이 될 수 없다. 상황에 따라서 아직 가입계약 엔터티가 도출되지 않 았을 수도 있고, 이미 도출되어 있을 수도 있다. 만약 아직 도출이 되지 않은 상태라면 이 속성에 대한 유일 값 검증에 의해서 가입계약 엔터티는 자연스럽게 도출될 것이며, 이미 존재한다면 이 속 성을 거기에 적절히 반영하면 된다.

나) 차량번호

어떤 고객은 하나 이상의 차량을 가질 수도 있다. 물론 지금까지 보유한 차량의 이력을 관리하고자 한다면 당연히 유일하지 않을 것이고, 현재 입장에서만 보더라도 하나 이상의 차량을 보유한 사 람들은 얼마든지 존재한다. 그러나 이처럼 실제로는 하나 이상이 존재하지만 업무적으로 판단했을 때 굳이 하나 이상의 차량을 관리할 필요성이 없다거나 과거의 이력을 관리할 의사가 없다면 모델 링 입장에서는 유일값이 되므로 이들은 이 엔터티의 속성이다.

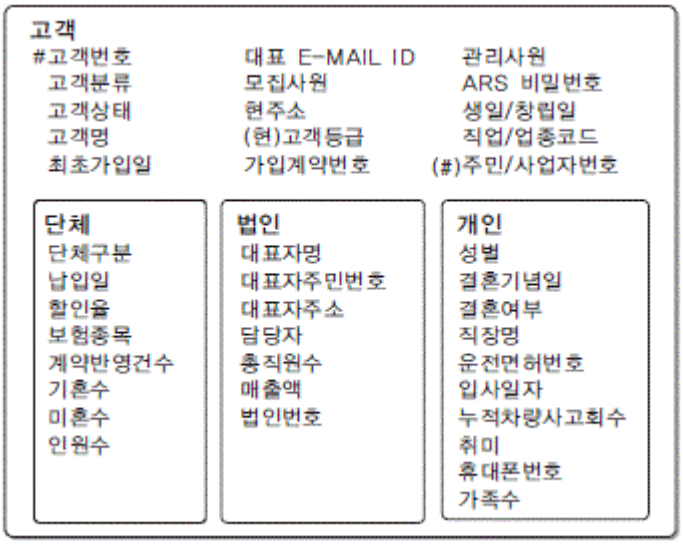
다) 취미

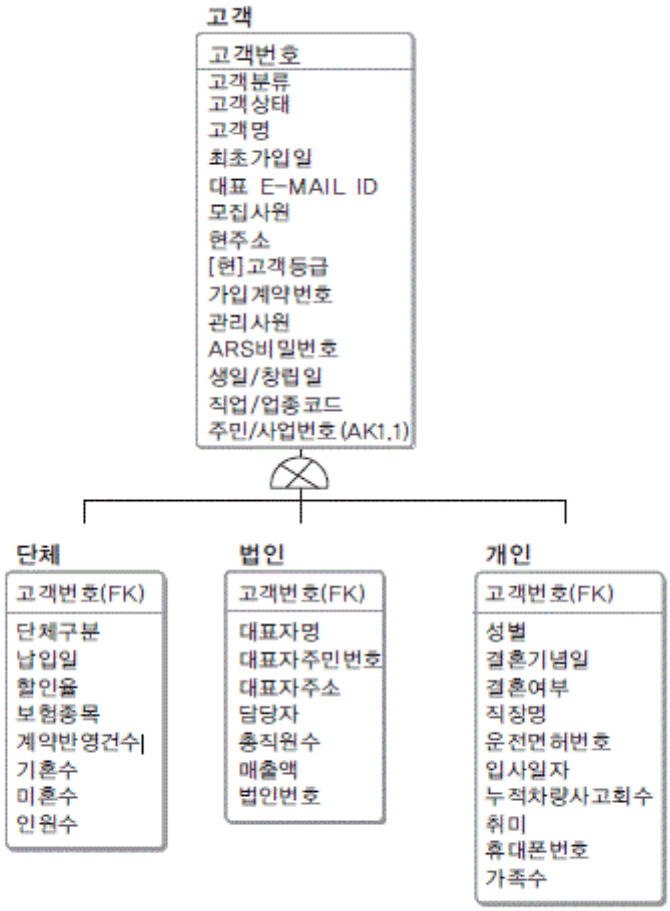
취미는 사람에 따라 당연히 여러 가지를 가질 수 있다. 취미와 같은 고객의 성향 정보는 마케팅의 중요한 자료가 된다. 엄청나게 많은 고객 모두를 대상으로 마케팅을 한다는 것은 너무 많은 비 용이 들어가기 때문에 가능성이 높은 고객들을 대상으로 표적 마케팅을 하지 않을 수 없다. 이때 고객의 각종 성향 정보는 매우 중요한 가치를 가진 정보이므로 이를 최대한 확보하려는 노력은 반드시 필요하다.

추출 속성(Derived Attribute)인지 검증한다

속성 검증의 제 3 단계는 그 속성이 원천적인 값인지, 다른 속성에 의해 가공되어서 만들어진 값인지를 검증하자는 것이다. 원천적인 값이란 말 그대로 다른 것에 의해 만들어진 것이 아닌 태초부터 창조된 것을 말하며, 추출 값이란 이들을 가지고 언제라도 쉽게 재현할 수 있는 속성을 말한다. 어 쩌면 이것은 매우 분명하고 단순한 개념이지만 실전적인 시각에서 바라보면 그리 간단하지만은 않다.

1) 대표적 유형





[그림 4-3-4] 바커 표기법(위)과 IE 표기법(아래)에 따른 추출 속성 사례

가) 현재 정보만 관리하는 형태 : 현주소, 고객 등급 등

자식 엔터티에 있는 맨 마지막 정보를 현재의 정보라는 의미로 중복화하는 경우라고할 수 있다. 엄밀하게 말하면 이력(선분) 개념이 들어가 있는 자식 정보 중에서 현재 시점을 통과하고 있는 선 분에 해당하는 데이터를 미리 가져다 두는 형태이다.

나) 최초 정보를 보관하는 형태 : 최초 가입일, 모집 사원 등

자식 엔터티의 여러 정보 중에서 하나만 선택하는 또 하나의 방법은 바로 최초로 발생한 정보만 가져다 두는 것이다. 실전에서 발생하는 대부분의 경우는 현재 정보나 집계 형태로 나타나지만, 가끔은 최초의 정보를 보관하고자 하는 경우도 나타난다.

다) 집계 정보를 관리하는 형태 : 인원 수, 가족 수, 총직원 수 등

추출 값을 관리하는 속성 중에서 아마 가장 많이 나타나는 형태는 수행 속도를 위해 하위 엔터티의 정보를 집계해 가져다 둔 경우일 것이다. 물론 집계의 형태에는 금액 관련 속성들에서 많이 발 생하는 합계(sum)와 발생회수(count)를 관리하기 위해 사용하는 건수, 회수, 차수 등의 단어가 들 어가는 속성들이 대부분이다.

라) 추출 릴레이션만 관리하는 형태 : 가입 계약번호, 관리 사원 등

추출 속성 중에는 자식 엔터티의 식별자를 전부 또는 일부만 옮겨둔 것을 자주 발견할 수 있다. 논리적으로는 M 쪽의 식별자가 1 쪽으로 올 수는 없지만 대부분의 경우 마지막 건의 식별자를 가져 다 두는 경우가 많다.

마) 대표 정보만 관리하는 형태 : 대표 전자메일 ID, 취미, 법인의 대표자 정보

자식 엔터티에 있는 여러 개의 정보를 하나로 만들어 올리는 방법 중에는 가장 대표적인 것 하나 만을 선정하는 경우도 있다. 예를 들어, 한 사람이 여러 개의 전자메일 ID을 가질 수 있다. 심지어 사람에 따라서는 십여 개를 가지고 있기도 한다.

바) 다른 속성의 일부 정보만 분리한 형태 : 성별, 결혼 여부 등

추출 속성 중에는 특별한 목적을 위해서 다른 속성의 일부를 떼어서 새로운 속성을 만드는 형태도 자주 등장하고 있다. 사실 우리가 확실한 속성이라고 할 수 있는 성별도 이러한 형태에 속한다. 만약 개인 서브타입에 속하는 모든 고객이 주민등록번호를 반드시 가져야 하고, 그 값이 확실하다고 가정한다면 일곱째 자리에 따라 남·여를 구별할 수 있다.

사) 일부분만 추출 값인 형태 : 인원 수, 법인의 대표자 정보 등

일부분이란 이 속성에 값이 들어가는 수많은 개체들 중에서 일부는 다른 엔터티에서 가공하여 만들 수 있지만 다른 일부는 원시 값인 경우를 말하는 것이다.

보다 상세하게 관리할 것이냐?

‘보다 근원적인 정보를 관리해야 하지 않겠느냐?’고 질문하는 것이다. 다시 말해 현재까지는 이대로도 충분했지만 수 많은 변화가 예상되는 미래를 대비하기 위해 좀더 근원적인 정보를 관리할 필요 성에 대해 검토해 보자는 것이다. 이 단계가 바로 데이터 모델링이 현재뿐만 아니라 미래의 시스템 구조를 찾아 나가는 과정이라는 것을 의미한다

추출 속성 규칙

다른 엔터티나 속성으로부터 유도되거나 계산된 속성들은 유도 또는 계산에 사용된 기준 속성(Base Attribute)에 대한 종속성과 함께 그 방법이 데이터 모델에 표현되어야 데이터 가공에 따른연관관계를 명확하게 전달하고 기록으로서 의미를 가질 수 있다.

- 초기 데이터 모델링 이론을 보면 도출된 속성이나 가공된 속성은 중복으로 인식하고 이를 데이터 모델에 표현하지 말것을 권고했다. 그러나 현재는 이러한 속성들이야 말로 기업의 관리자들이 관 심을 가지고 보는 속성으로 데이터 모델 내 반드시 기술할 것을 권장하고 있다.
- 추출(Derived) 속성이란 하나 이상의 다른 데이터 속성에 축적된 여러 사례의 값을 토대로 어떤 추가 계산 작업을 수행함으로써 선택적으로 주제를 도출하는 속성에 대하여 새로운 값을 창출하는 속성이다.
- 계산(Calculated) 속성은 엔터티의 단일 사례에 대한 어떤 특성을 기술하며, 일반적으로 관련 속성의 또다른 단일 사례로부터 계산된다.
- 추출 속성은 경영층이 진실로 원하고 필요로 하는 정보를 대표한다.
- 추출 속성은 사용자들의 데이터 요구 사항을 나타낸다.
- 데이터 모델에 추출 속성을 포함시키는 것은 그들의 물리적 구현에 대한 어떤 것도 내포하지 않는다.
- 추출 속성은 향후 과정에서 결코 기본키(Primary Key)로서의 역할을 맡지 않아야 한다.

속성 정의시 유의사항

데이터 모델링을 진행하는 과정에서 속성의 정의 시 흔히 범하는 오류는 필요할지도 모른다고 생각 되는 속들을 필요할 때마다 이것저것 나열해 두는 방법으로 속성을 정의하는 것이다. 이런 방식의 속성 정의에는 진정한 엔터티 자신의 속성 이외의 다른 엔터티에서 빌려 온 것들이거나 여기저기에 있는 정보를 이용하여 가공을 해놓은 속성들이 많이 존재한다. 하지만 이러한 방법으로 정의된 속성들은 항상 데이터의 정합성을 해칠 수 있는 개연성을 가지고 있기 마련이다. 데이터 모델링에서 데이터가 들어가 살게 될 방이라 할 수 있는 속성을 명확하게 정의하는 일은 중요하다. 속성을 정의할 때에는 다음과 같은 유의 사항을 지켜야 한다.

의미가 명확한 속성 명칭을 부여한다.

속성의 명칭을 명확히 하는 것은 매우 중요하다. 이 말의 진정한 의미는 그 속성의 개념을 정확히 부여한다는 것을 뜻한다.

1) 직업이라는 속성 정의

직업의 사전적인 의미는 생계를 위하여 일상적으로 하는 일이라는 뜻이다. 하지만 이것이 '회사원, 전문직, 자영업, 공무원 …'처럼 수십 가지 정도로 분류한 것을 말하는 것인

지, 아니면 이 보다 훨씬 상세하게 '전산설계, 프로그래머 …' 등과 같이 수백 가지로 분류한 것을 말하는지, 아니면 수 천 가지 이상으로 상세하게 분류한 것을 말하는지를 분명하게 정의해야 한다.

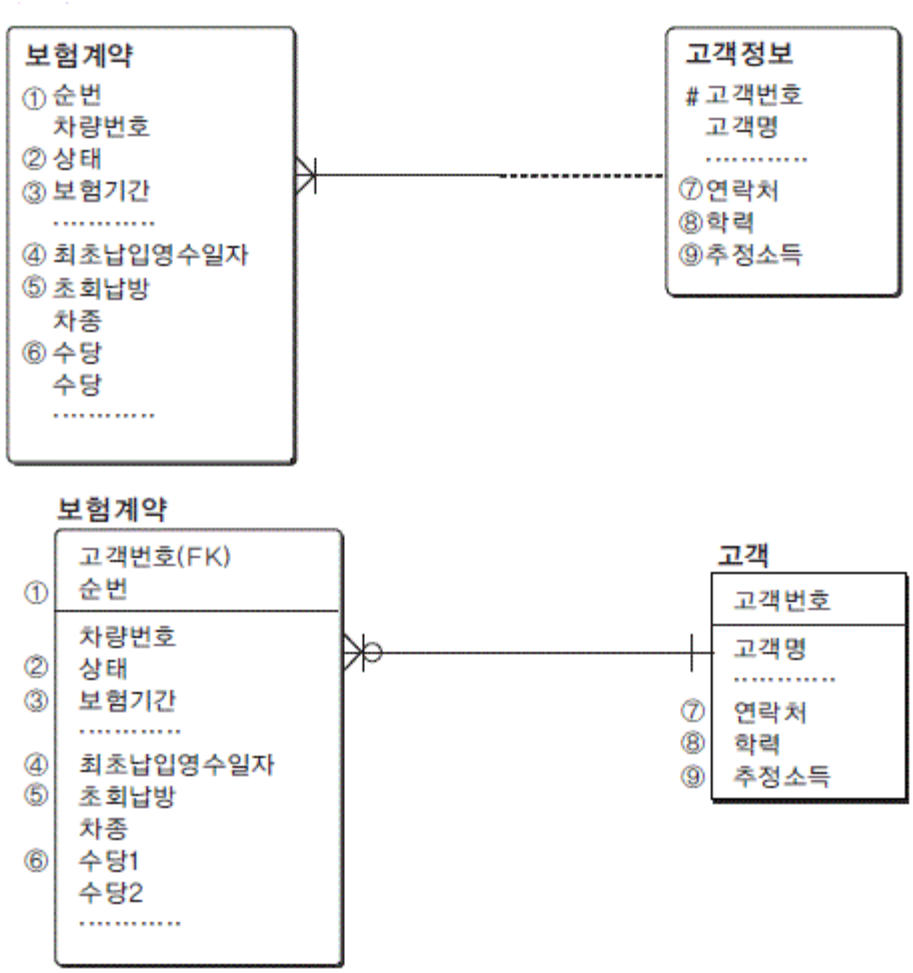
2) 학점이라는 속성

학생이 수강한 강좌에서 취득한 'A 학점, B 학점 …'을 말하는 것인지, 그 강좌를 이수했을 때 받는 '2 학점, 3 학점 …'을 말하는지 분명히 해야 한다는 것이다. 이런 상황에서는 설사 최초 설계 단계에서는 매우 명확한 정의가 되었더라도 의미의 와전이나 단절은 필연적으로 발생한다.

유일한 복합명사 사용

속성의 개념을 구체적이고 명확하게 정의했다면 그 명칭을 제대로 부여하는 것도 매우 중요한 일이다. 남들에게 구구절절 보충설명을 하지 않더라도 거의 유사한 생각을 가질 수 있도록 보편적이고 함축성 있는 단어를 찾아내어야 한다. 속성이란 자신만이 가지는 분명한 독립적인 의미를 가지고 있 기 때문에 명칭 또한 단순히 일반 용어만으로 부여해서는 결코 구체적인 의미를 나타낼 수 없다. 결론적으로 말하면, 보편적인 용어를 적절히 결합한 복합명사를 만듦으로써 구체적인 표현을 할수 있다는 것이다.

[그림 5-3-5]와 같은 논리 데이터 모델을 예로 보면 모호하게 이름지어진 속성이 많이 존재한다.



[그림 4-3-5] 바커 표기법(위)과 IE 표기법(아래)에 따른 속성 명칭 예

1) 순번

많은 데이터 모델에서 순번 혹은 일련번호라는 명칭의 속성을 자주 접하고 있다. 이 속성은 인조식별자를 생성하기 위해서 사용한 것이 분명하며, 누구나 그렇게 받아들이고 있기도 하다. 그러나 분명히 이 속성의 명칭에는 문제가 있다. 여기서 만약 정확하게 표현하고자 한다면 보험계약순번 혹은 계약순번으로 지정하는 것이 옳다. 만약 순번이라고만 표현한다면 나중에 테이블로 설계가 되었을 때 이 테이블을 참조하는 하위 테이블에서 보면 어디서 온 칼럼인지 분명치 않게 된다. 만약 또 다른 엔터티에서도 순번이라는 속성을 가지고 있고 그것도 참조한다면 실제로는 서로 다른 속성이었지만 참조하는 쪽에서 보면 동일한 이름으로 나타난다는 문제가 있다.

2) 상태

이 명칭만으로는 속성의 의미가 참으로 애매모호하기 짝이 없다. 물론 심증적으로는 보험계약상태를 말하는 것으로 보이지만 확실하지 않다. 실전에서는 이처럼 오해의 여지가 남아있는 애매한 명칭들이 많이 나타난다. 이 상태가 자기 엔터티에서 지정되는 값을 관리하고자 하는 것인지, 하위 엔터 티의 최종 상태를 가져다 둔것인지, 그것도 아니면 하위 엔터티의 상태를 종합해서 새로 의미를 부여한 상태를 의미하는지 구분할 수 없다. 어떤 상태를 의미하는지 누가 보아도 알 수 있도록 명칭을 분명하게 지정해야 한다.

3) 보험기간

보험계약이 적용되는 기간일 것으로 예상되기는 하지만 가령 12개월, 20개월 …로 표현되는 것인지 1년, 2년, …으로 표현되는 것인지를 알 수가 없다.

4) 최초납입영수일자

간결한 명사를 최초 + 납입 + 영수 + 일자로 잘 결합하여 복합명사를 만듦으로서 누가 보더라도 오해가 없을 만큼 상세하게 표현되어 있다.

5) 초회납방

최초로 납입한 회차의 납입 방법을 말하는 것인데 설사 그 업무에 관련 있는 대부분의 사람들이 이해한다고 하더라도 최초납입회차 납입방법으로 표현하는 것이 바람직하다.

이렇듯 우리가 데이터 모델링에서 사용하는 속성명을 분명하게 표현하기 위해서 가능한 한 복합명사를 사용해야 한다.

단수형으로 속성명을 사용한다

속성은 단일 값만을 저장하기 때문에 단수형으로 적는 것이 바람직하다. 예를 들어 고객 엔터티의 전자메일 속성 이름을 [전자메일들]과 같이 지었다면, 이 속성은 여러개의 값들을 저장하는 것인가? 그렇다면 속성이 될 수 없으므로 여러 속성으로 분리하든지 아니면 별도의 고객 전자메일 엔터티로 분리해야 한다.

표준 단어 제정

표준 용어 사전을 데이터 모델링을 하기 이전에 생성하여 두면 데이터 모델의 각 객체들(엔터티, 속성, 테이블, 칼럼 등)이 최대한 그 기준을 준수하도록 유도하기가 용이하며 뚜렷한 일관성이 생길것이다. 또한 표준화 준수 여부를 관리하거나, 표준으로 변경할 대상을 선정하는 일, 어떤 속성의 파생 근원을 분석하는 데도 굉장한 힘을 발휘한다. 뿐

만 아니라 속성을 구성하는 단어들을 관리하는 용어 사전에 데이터베이스 설계 단계에서 속성을 칼럼(Column)으로 전환시킬 때 사용할 영문 약어를 같이 제정해 두는 것도 바람직하다. 최근에는 모델링 도구뿐만 아니라 데이터 표준화를 전문적으로 수행하는 툴도 등장하고 있는 추세이다.

작의적인 전용 금지

전용(轉用)이란 말을 사전에서 찾아보면 예정되어 있는 곳에 쓰지 않고, 다른 데로 돌려서 쓰는것이라고 되어 있다. 이말과 통합이란 말에는 미묘한 관련이 있다. 가령 A 를 위해??부터 A 와 B 를 위해서 만들었다면 통합이다. 그러나 결과만 놓고 본다면 어쨌거나 A 와 B 를 위해 사용되었다는 것은 동일하다.

전용이 발생하는 경우는 크게 세 가지로 나눌 수 있다.

- 모델러가 속성의 의미를 매우 추상적이고, 모호하게 정의했기 때문에 그것을 적용하는 개발자들이 각기 다르게 이해함으로써 나타나는 경우이다.
- 개발이 막바지에 도달했거나 이미 운용 중인 시스템에서 새로운 사용자 요구 사항으로 인해 새로운 속성의 추가가 필요할 때 나타나는 경우이다.
- ERP 패키지에서 자주 나타나는 형태로써 미리 전용의 용도로 속성을 정의해 두는 경우를 말한다.

식별자(UID, Unique Identifier) 확정

엔터티 내의 모든 인스턴스는 유일하게 구분되어야 한다. 이러한 유일성을 보장하기 위해서 필요한 것이 식별자이다. 식별자에는 크게 본질 식별자(Primary UID 또는 Intrinsic UID), 주(실질) 식별자(Actual UID), 대체(보조) 식별자(Secondary UID)로 구분할 수 있다. 대체 식별자는 방법론에 따라서 AK 혹은 Alternate Key 로 부르기도 하지만 현재 단계에서는 Key 보다 식별자가 더 적절한 표현이다.

본질 식별자

엔터티에는 인조 식별자(Artificial Unique Identifier)가 있고, 진주어(眞主語)에 해당하는 관계나 속성이 어딘가에 있다. 개념 데이터 모델링 과정에서 키 엔터티와 일부 핵심 엔터티에 대해서 본질 식별자를 정의하였다. 여기에서는 엔터티 상세화 단계에서 정의되어질 핵심,액션(행위) 엔터티에 대한 본질 식별자를 정의한다.

본질 식별자를 정의하는 방법은 키 엔터티와 행위 엔터티가 서로 다르다. 행위 엔터티를 정의하는 방법에는 상황에 따라 하향식과 상향식으로 접근하는 방식을 적용할 수 있다. 키 엔터티는 부모가 없이 창조된 집합이므로 식별자 또한 창조시켜 주어야 한다. 그러나 행위 엔터티는 항상 부모가 누구인지를 확인하는 방식으로 진행된다.

1) 키 엔터티의 본질 식별자

키 엔터티는 우리가 잘 알고 있듯이 사원, 고객, 상품과 같이 부모 엔터티 없이도 혼자서 정의될 수 있는 엔터티이다. 예를 들면 사원 엔터티에는 이미 이전부터 정의해서 사용하고 있던 사원번호가 있다는 것은 굳이 언급할 필요도 없다. 그러나 좀더 깊이 생각해 본다면 사원 집합의 개체는 사람이므로 사실 본질 식별자는 주민등록번호라고 볼 수 있다. 물론 외국인 사원이 있다면 여권번호, 외국인 등록번호 등이나 임시로 부여한 임의의 값을 광의의 주민등록번호로 정의한다면 이것을 개념적으로는 원래의 본질 식별자로 볼 수 있다. 뒤에서 확정 식별자를 설명할 때 전략적인 부여 방법에 대해 자세하게 언급하겠지만 이 주민등록번호는 몇 가지 전략적인 이유 때문에 사원번호라는 인조 식별자에게 대표 권한을 물려준 것이다.

2) 절대 종속 / 상대 종속 의미

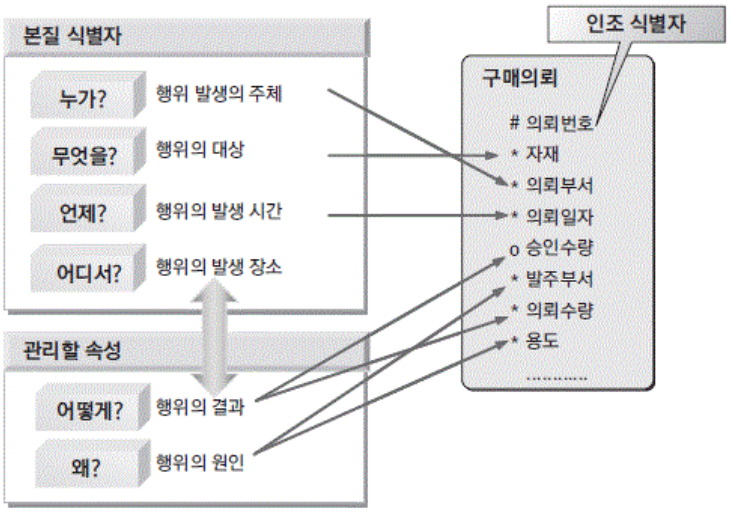
절대 종속과 상대 종속은 나를 태어나게 하는데 절대적인 영향을 주었는지, 그렇지 않는지를 따지 는 것이다. 다시 말해서 내가 태어나기 위해서 절대적으로 존재했어야 하는지, 아니면 그것이 없어도 내가 태어날 수가 있는지를 확인하는 것이다. 절대 종속을 확인하는 방법은 매우 간단하다. 영화 백 투더 퓨쳐(Back To the Future)처럼 과거로 돌아가서 누구를 없앴더니 내가 태어나지 않게 된다면 그는 나를 태어나게 하기 위해 절대적으로 필요한 존재이다. 반면에 누군가가 우리 집안과 분명히 어떤 관계를 맺고 살았지만 설혹 그가 없다고 하더라도 내 탄생에는 아무런 영향을 미치지 않는다면 그는 나의 탄생에 대해서는 상대 종속이 된다.

3) 직접 종속 / 간접 종속 의미

‘1 촌이냐? 1 촌 이상이냐?’를 구분한다. 즉, 부모 엔터티와의 관계가 1 촌이면 직접 종속이고 1 촌 이상이면 간접 종속이라고 볼 수 있다.

4) 행위 엔터티의 본질 식별자

행위 엔터티의 본질 식별자란 이와 같이 절대종속이면서도 직접종속인 것을 찾고자 하는 것이며, 결국은 자신을 태어나게 한 근본을 찾는 것이다.



[그림 4-3-6] 본질 식별자 예

본질 식별자를 찾는 가장 확실한 방법은 사건의 전모를 가장 체계적으로 표현할 수 있다는 기사 작성의 여섯 가지 원칙인 육하원칙(六何原則)을 이용하는 것이다. '누가, 무엇을, 언제, 어디서, 어떻 게, 왜'라는 이 여섯 가지의 질문을 통해 발생된 행위를 구체적으로 규명하면 상황에 대한 정확한 사실을 규명할 수 있다.

후보 식별자 도출

이전 단계에서 정의된 본질 식별자를 기본으로 식별자의 기본 목적인 자기를 식별할 수 있어야 한다는 유일성 유지의 목적과 다른 엔터티에서 정보로 참조해야 하는 목적을 적절히 판단하여 최종 식별자를 확정해야 한다. 하나의 엔터티 내에는 식별자로 사용할 수 있는 하나 이상의 식별자가 있다. 이 중에서 하나가 식별자로 선택되게 된다. 나머지 식별자들을 후보 식별자(Candidate UID, 방법 론에 따라서는 CK 혹은 Candidate Key 라고도 하지만 현재 단계에서 Key 라는 표현보다는 식별자라는 표현이 더 적절하다)라고 한다. 이러한 후보 식별자들은 다음과 같은 조건을 만족해야 한다.

1) 각 인스턴스를 유일하게 식별할 수 있어야 한다

가장 기본적인 전제 조건이며, 후보 키들은 유일한 값을 가지고 이를 통해 나머지 인스턴스와 자신을 식별하는 능력을 가져야 한다. 후보 식별자는 단일 속성뿐만 아니라 하나 이상의 속성이 모인 집합으로서도 후보 식별자가 될 수 있다. 그러므로 속성 혹은 여러 속성들이 조합된 속성 집합은 전체 인스터스에서 유일값을 가져야 한다. 그래야만 특정한 인스턴스를 선택하기 위해서 전체 인스턴스에서 유일한 값을 가지는 후보 식별자를 선택할 수 있다.

2) 나머지 속성들을 직접 식별할 수 있어야 한다

인스턴스 간에서 뿐만이 아니라 후보 식별자는 나머지 속성을 식별할 수 있는 능력을 가지고 있어야만 한다. 이는 후보 식별자가 유일 값을 가지고 있는 상황에서 특정 인스턴스를 추출하기 위해서 유일값을 가진 후보 식별자를 찾아내면, 후보 식별자에 관련된 나머지 속성을 다시 찾아낼 수 있다는 의미이다.

3) NULL 이 될 수 없다

후보 식별자들은 NULL 이 될 수 없다. NULL 이란 해당 속성에 값이 지정되지 않았다는 적극적인 표시라고 볼 수 있다. 따라서 NULL 이 허용되는 속성이 있다면 값을 넣지 않은 경우에는 NULL 이 할당된다. NULL 이 할당되었다는 것은 값이 없다는 것이므로 NULL 이 있는 속성은 식별할 수가 없다.

4) 후보 식별자로 속성 집합을 선택하는 경우에는 개념적으로 유일해야 한다

집합으로 후보 식별자를 선택하는 경우에는 개념적으로도 유일할 것이라는 판단을 하고서 후보 식별자로 선정해야만 한다. 만일, 회원 테이블에 대한 견본 데이터를 보고서 부서+성명 집합이 유일한 값을 가진다고 판단했다고 하더라도 개념적으로 부서에 동일한 이름을 가지는 사원이 없다고 확신할 수 있겠는가? 하지만 만일에 부서를 관리하는 엔터티에서 부서명+팀명을 후보 식별자로 선택하는 것은 개념적으로 유일할 수 있다. 한 부서에 동일한 이름을 가지는 팀은 없기 때문이다.

5) 후보 식별자의 데이터는 자주 변경되지 않는 것이어야 한다

데이터가 자주 변경된다고 해서 후보 식별자가 될 수 없는 것은 아니지만 일반적으로 후보 식별자의 값은 자주 변경되지 않는다. 데이터베이스 설계 단계에서 이러한 식별자들은 대부분 인덱스를 통해 구현된다. 인덱스는 물리적으로 트리 구조를 가지고 수많은 노드 및 포인터를 관리하게 된다. 어떤 노드의 데이터가 변경되면 트리 구조를 재수정하는 데에 너무나 많은 시간이 필요하게 되어 데이터베이스의 성능을 떨어뜨리게 되므로, 인덱스에 선택되는 칼럼의 데이터가 자주 변경되는 것은 좋지 않다.

대체(보조) 식별자

대체(보조) 식별자란 원래의 식별자를 대신할 수 있는 또 다른 속성들이나 관계(관계속성)를 말한다. 가령 사원 엔터티에 공식적으로 부여된 식별자는 사원번호이지만, 만약 주민등록번호 속성이 유일한 값을 가지면서 필수적(Mandatory)으로 정의되었다면 비록 공식적인 식별자는 아니지만 식별자로서의 역할을 할 자격은 충분히 갖추고 있다. 특히 대체(보조) 식별자는 여러 참조 엔터티 중에서 원래의 식별자 보다 대체(보조) 식별자로 연결을 맺는 것이 자신에게는 훨씬 유리한 경우에 의미가 있게 된다.

인조 식별자 지정

인조 식별자란 식별자 확정시 기존의 본질 식별자를 그대로 실질 식별자로 인정할 수 없는 여러가지 상황이 발생했을 때, 전부 혹은 일부를 임의의 값을 가진 속성들로 대체하여 새롭게 구성한 식별자를 말한다. 가령, 사원 엔터티에 이미 존재하고 있는 속성 중에서 원래의 본질 식별자를 찾으라고 한다면 주민등록번호가 될 것이다. 그러나 이 속성은 너무 길고 관리상 여러 가지 문제점이 발생하기때문에 새롭게 사원번호라는 임의의 값을 가진 인조 속성을 영입하여 공식적인 식별자 자리까지 부여받은 것이다. 인조 식별자는 다음과 같은 기준을 가지고 지정하는 것이 바람직하다.

1) 최대한 범용적인 값을 사용한다

인조 식별자의 속성은 남들이 알지 못하는 임의의 값일 수 있기 때문에 특별한 결격 사유가 없다면 가능한 한 기존에 범용적으로 사용하던 것을 그대로 사용하는 것이 좋다. 예를 들어, 이미 회사 내에 공인되어 사용하고 있는 사원번호, 상품코드, 국가나 공공기관에서 부여한 은행코드, 국제적으로 사용하고 있는 무역상품 분류체계인 HS 코드, 범용적으로 사용하고 있는 국가나 통화(通貨)코드 등은 가능하다면 그대로 사용하는 것이 여러 가지로 유리한 점이 많다.

2)유일한 값을 만들기 위한 인조 식별자를 사용한다

지금까지 정의해 왔던 본질 식별자를 그대로 사용하면 심각한 문제가 발생하는 경우가 몇 가지 있다. 여기에서는 그 중에서 한 가지인 유일값에 대한 확신이 없을 때 이를 해결하기 위해 인조 속성을 영입하는 경우에 대해 설명하고자 한다. 어떤 경우에는 본질 식별자는 논리적으로야 문제가 없지만 실제적으로는 유일성에 대한 현실적인 문제가 발생하므로 인조 속성의 도입을 검토해야만 한다. 물론 그 인조 속성을 아예 전체 본질 식별자를 대체하도록 하든, 그 중의 일부를 대체하게 하든 종합적 인 판단에 따라 달라지겠지만 인조 속성이 필요하다는 것은 분명하다. 사실 실전에서 인조 속성을 도 입하는 상당 부분은 바로 이런 경우라고 할 수 있다.

3)하나의 인조 식별자 속성으로 대체할 수 없는 형태를 주의한다

인조 속성을 만들 때 이 속성이 구체적으로 본질 식별자의 어느 부분을 대체하고 있는지를 분명하게 정의해야 한다. 그러나 하나의 인조 식별자 속성에 같이 대체될 수 있는 것과 절대로 그렇게 해서는 안 되는 경우가 있다는 것에 유의해야 한다. 만약 이런 원칙을 어긴다면 나중에 집합 내의 개체의 정의가 모호해져 엔터티 전체를 애매한 집합으로 변질시키게 되므로 주의해야 한다. 예를 들면, 이력을 대체하는 일련번호 인조식별자를 갖고 있는 엔터티가 자식을 갖게 되면 일련번호 인조식별자가 상속되게 되고, 이때 부모에 변경이력이 발생하게 되면 자식에 변경이 없더라도 새로운 이력일련번호를 상속

하여 자식을 다시 만들어야 하는 상황이 발생할 수 있으며, 이로 인해 자식의 정보를 활용하는데 있어서 혼란이나 왜곡이 생길 수 있다.

4)편의성 ·단순성 확보를 위한 인조 식별자를 사용할 수 있다

속성의 길이가 너무 길거나 기억하기가 어려워서 좀더 쉽고 간편한 이름으로 변경할 목적으로도 인조 속성을 추가시킬 수도 있다. 마치 사람의 이름을 간편하고 부르기 쉽도록 애칭이나 별명을 만드는 것과 유사한 목적이라 할 수 있다. 성을 포함해서 대부분 세 글자에 지나지 않는 우리나라 사람들에게는 그리 흔치 않는 일이지만 훨씬 긴 이름을 가진 외국 사람들에게 흔하게 사용된다. 이처럼 본질 식별자를 그대로 사용해도 불편이 없다면 굳이 인조 속성을 만들 필요가 없겠지만 그렇지 않다면 고려해 볼 만한 충분한 가치가 있다.

5)의미의 체계화를 위한 인조 식별자를 사용할 수 있다

의미를 체계화한다는 말을 다른 말로 쉽게 표현하면 코드화를 한다는 것과 유사하다. 과거에는 전산 은 곧 코드라는 말이 있을 정도로 모든 것을 코드화하려고 애를 썼다. 코드화한다는 말에는 곧 속성의 자릿수마다 나름의 의미를 부여하겠다는 의미를 포함한다. 논리적으로 임의의 값이란 말에는 이미 정해진 값의 형태가 포함되어 있다. 인조 속성이란 임의의 값을 정의하는 것이라고 했다. 그렇다고 해서 그야말로 아무 값이나 괜찮다는 것은 아니다. 특별한 의미를 부여할 필요가 없다면 순차적인 번호, 곧 일련번호를 정의하는 것으로도 충분하겠지만 특정한 의미를 부여함으로써 변별력이 향상된다거나 처리의 규칙이 생겨난다면 너무 지나치지 않는한 그것을 무조건 나쁘다고만 할 수는 없다.

6)내부적으로만 사용하는 인조 식별자

인조 식별자를 생성하는 또 한 가지의 경우는 현업 사용자들에게는 전혀 알려주지 않으면서 시스템 내부적으로만 사용하는 형태이다. 물론 데이터 모델링 입장에서 본다면 현업 사용자뿐만 아니라 시스템 개발이나 관리를 하는 사용자 또한 엄연한 사용자임에 틀림이 없다. 여기서 설명하고자 하는 것은 새롭게 정의한 인조 식별자가 비록 업무적으로는 아무런 의미도 없지만 시스템적인 필요성에 의해서 도입하는 경우를 소개하려는 것이다.

식별자 확정

식별자는 자기 엔터티를 위해 생성하는 것처럼 보인다. 하지만 보다 중요한 결정 요소는 나를 참조 할 다른 엔터티가 원하는 형태로 결정되어야 하는 것이기 때문에 주변 엔터티의 상황을 종합적으로 살피고, 주변 엔터티의 상황도 최대한 수렴할 수 있도록 하

는 것이 매우 중요하다. 그래서 식별자를 확정할 때는 자기 자신에 대한 존재 가치뿐만 아니라 남들에 대한 배려를 어떻게 조화시키느냐가 중요한 관건이다.

1) UID BAR 의 두 가지 의미

UID BAR 가 가지고 있는 본질을 자세히 살펴보면 크게 두 가지의 의미를 가지고 있다.

가) 식별자로서의 역할

엔터티 자신의 입장에서 보았을 때 자신의 개체들을 다른 것들과 구별될 수 있도록 유일한 값을 만드는데 일조를 한다는 의미이다.

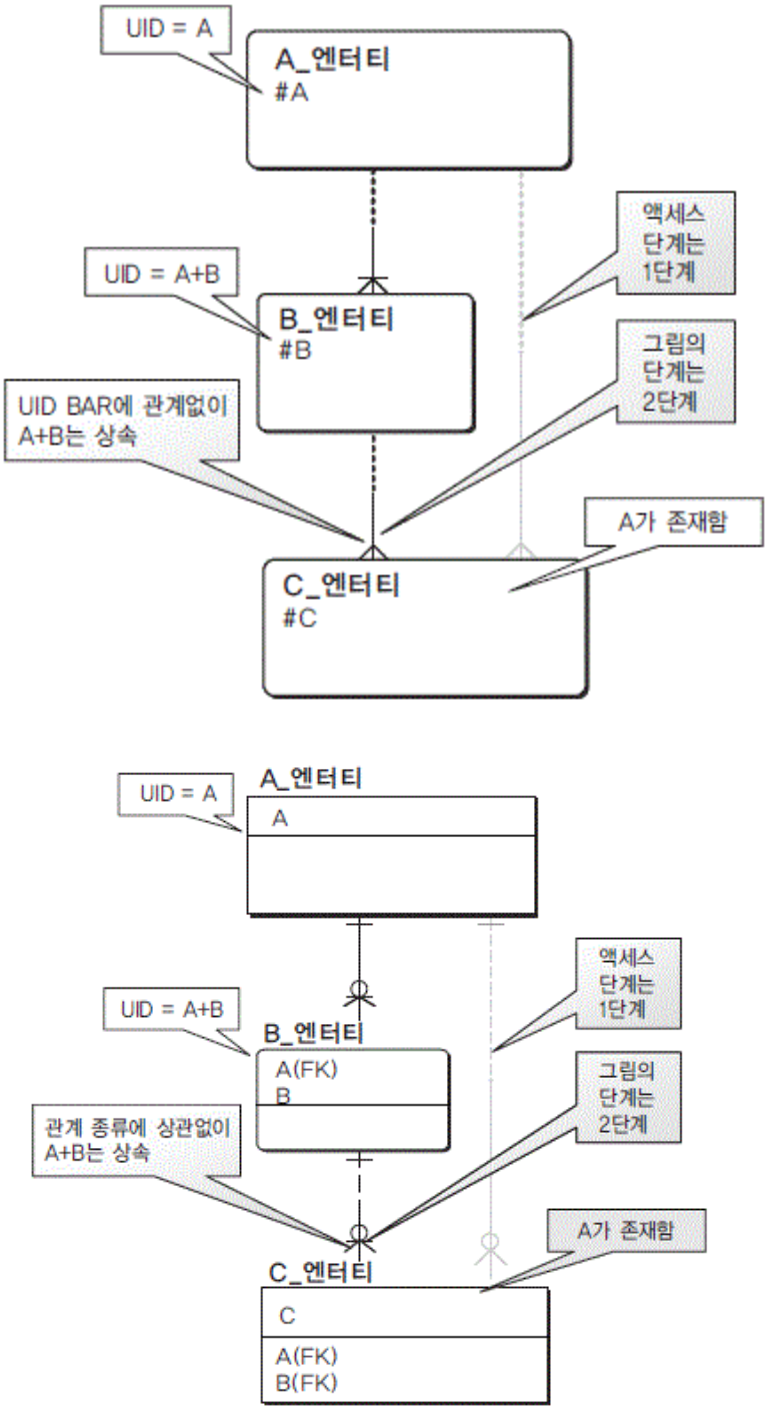
나) 정보로서의 역할

참조하는 엔터티의 입장에서 보았을 때 상대방의 식별자를 상속 받았기 때문에 자신이 보유한 정보가 증가했다는 의미도 가지고 있다.

조상의 식별자를 내가 상속 받고, 다시 그것을 내 자손들에게 물려준다면 자손들은 굳이 나를 경유하지 않고서도 조상이 누구인지 알 수 있는 것과 유사하다. 인조 식별자를 설명하면서 언급했듯이 단지 유일한 값을 만들기 위한 목적 뿐이었다면 굳이 부모에게 받은 식별자를 자??을 만드는 것이 가능하다. 이 말에는 부모의 식별자를 내 이름에 넣는 것에 상관없이 내가 태어난 이상 부모는 존재한다는 의미가 내포되어 있다. 비록 부모가 분명하더라도 부모의 식별자가 나의 식별자를 만들기 위해서 반드시 필요한 것은 아니라는 것이다. 그러나 내 식별자에 부모의 식별자를 포함시키지 않는 순간에 내 자손들에게는 더 이상 조상의 식별자를 상속시키지 못한다.

2) UID 상속과 단절의 원리

식별자 확정이란 단지 자기 식별자의 형태를 결정한다는 단순한 의미로만 생각해서는 안된다. 데이터 모델이 복잡한 업무로 인해 어쩔 수 없이 복잡해지더라도 식별자 상속이 전략적으로 이루어진다면 나중에 데이터를 처리할 때의 실제 액세스 단계에서는 훨씬 간편하게 만들 수 있다. 즉, UID 의 적절한 상속과 단절 전략은 실질적인 처리의 단순화를 가져다 줄 수 있으므로 그 전략적 가치를 가지고 있다.



[그림 4-3-7] 바커 표기법(위)과 IE 표기법(아래)에 따른 UID 상속 예

- [그림 4-3-7]에서 보면 A_엔터티와 B_엔터티 사이의 릴레이션에 UID BAR 가 있다는 것은 C_엔터티와 B_엔터티 사이의 릴레이션에 UID BAR 의 존재 여부와 상관 없이 무조건 조부모인 A_엔터티의 식별자 A 를 상속 받게 된다는 것을 의미한다. 이는 곧 C_엔터티에는 A 가 있으므로 이미 아버지인 B_엔터티를 경유하지 않고서도 직접 A_엔터티와 연결될 수 있음을 의미한다. 이 릴레이션십에 의해 부모나 그 이상의 조상에게 있던 식별자 속성이 설계 단계에서 나에게 상속된다는 의미일 뿐이지 참조하는 경로가 그렇게 되어야 함을 뜻하는 것은 결코 아니다. 이렇게 평범하고 당연해 보이는 사실이 갖는 진정한 가치는 상속 여부를 결정할 때 매우 중요한 판단의 기준이 된다.

3)식별자 확정 절차

하향식(Top-down) 방식, 즉 상위 엔터티부터 시작해서 하위 엔터티로 순차적으로 결정해 가는 것이 좋다. 식별자 상속이란 상위에서 하위로 이루어지기 때문이다.

가) 키 엔터티 식별자 확정

부모를 가지지 않는 최상위 엔터티이므로 서로 독립적으로 식별자를 확정할 수 있다. 사실 엄밀히 말하면 최상위 엔터티인 키 엔터티는 본질 식별자와 굳이 다르게 할 필요가 없으므로 본질 식별자를결정하는 단계에서 미리 식별자를 확정하는 것이 좀더 현실적이다. 물론 식별자 확정 단계에서 주변의 상황과 여건을 감안하여 일부를 조정하여 최종적으로 확정한다.

나) 메인 엔터티 식별자 확정

메인 엔터티는 해당 업무의 근본이 되는 엔터티라고 할 수 있으므로 자신이 하위에 거느리고 있는 수많은 엔터티의 상황을 종합적으로 감안한 전략적인 결정을 해야 할 것이다. 이러한 엔터티는 자신의 하위 엔터티에게는 최상위 조상이기도 하므로 될 수 있는 대로 식별자 속성의 개수를 적게 하는 것도 중요하다. 이런 이유 때문에 경우에 따라서는 전체를 대신하는 인조 식별자를 생성하기도 한다.

다) 하위 엔터티 식별자 확정

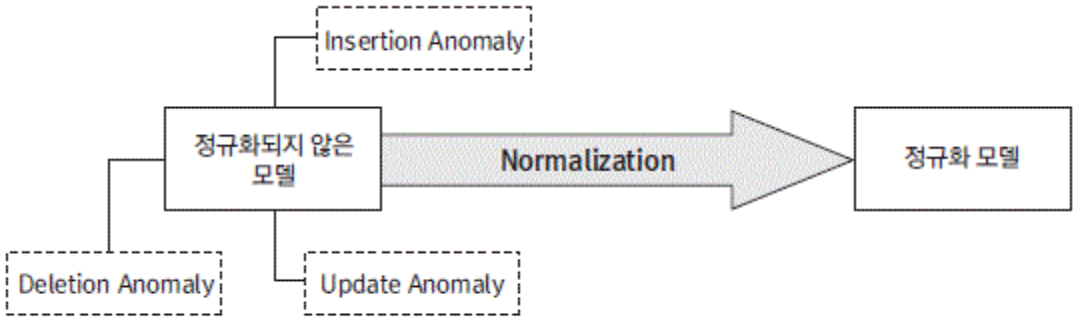
이런 부류의 엔터티는 가능하다면 인조 속성을 많이 사용하지 않는 것이 바람직하다. 그 이유는 인조 속성이란 임의의 값을 의미하므로 유일성에는 도움을 줄지 모르지만 정보로서의 가치는 현저하게 감소하기 때문이다.

정규화(Normalization)

정규화는 논리적 데이터 모델을 일관성이 있고 중복을 제거하여 보다 안정성을 갖는 바람직한 자료구조로 만들기 위해 여러 단계를 거친다. 그 단계는 제 1 차 정규형에서부터 제 5 차 정규형과 BCNF(Boyce-Codd Normal Form)까지로 구성되어 있다. 대체로 적절하고 일관성을 유지하면서 중복이 없는 논리적 데이터 모델을 구축하는 데에는 흔히 3 차 정규형이 사용된다.

정규화의 의미

- 잘 만들어진 데이터 모델이라고 해도 엔터티에 데이터를 삽입, 수정, 삭제할 때 오류가 발생할 개연성을 가지고 있다. 이러한 것들을 통칭해서 변경 이상(Modification Anomaly)이라고 한다. 여기에는 구체적으로 삽입 이상(Insertion Anomaly), 수정 이상(Update Anomaly), 삭제 이상(Deletion Anomaly) 등이 있다.
- 변경 이상이 발생하는 엔터티를 그대로 운용하게 되면 데이터가 신뢰할 수 없는 값들로 채워질 가 능성이 있다. 즉, 데이터의 일관성, 무결성을 해칠 가능성이 있는 것이다.
- 정규화 과정을 통해서 변경 이상의 엔터티를 정규화된 엔터티로 변환하게 된다.



[그림 4-3-8] 정규화 과정

1) 입력 이상

별도의 사실이 발생하기 전까지 원하는 데이터를 삽입할 수 없음. 어떤 데이터를 삽입하려고 할 때 불필요하게 원하지 않는 데이터도 함께 삽입된다.

2) 삭제 이상

일부 정보를 삭제함으로써 유지되어야 할 정보까지도 삭제되는 연대 삭제가 발생한다.

3)갱신 이상

일부 속성 값을 갱신 함으로써 원하지 않는 정보의 이상 현상(무결성 파괴, 정보의 모순성)이 발생 한다.

정규화의 장점

1) 중복값이 줄어든다

궁극적으로 칼럼 간, 레코드 간, 테이블 간에 중복되는 데이터들을 최소화할 수 있다. 이것이 정규 화의 최대 성과라고 할 수 있다. 사실은 모든 장점이 이것에서 시작된다.

2) NULL 값이 줄어든다

전체적으로 NULL 값의 사용이 줄어들게 된다.

3) 복잡한 코드로 데이터 모델을 보완할 필요가 없다

데이터에 중복된 값이 적고, 그 부모가 누구인지가 항상 명시되어 있는 상황에서 데이터의 무결성을 지키기 위한 복잡한 코드를 사용할 필요가 없어진다. 데이터베이스 코드를 작성하는데 복잡한 변환 과정과 조인 질의, NULL 값 처리들이 필요하다는 것은 그 데이터베이스 설계가 문제가 있다는 말이 기도 하다.

4) 새로운 요구 사항의 발견 과정을 돕는다

실제 정규화 과정에서 많은 엔터티 혹은 속성들이 태어나게 된다. 또한 많은 결정 과정에서 업무 담 당자와의 협의를 통해서 현재뿐만 아니라 미래까지도 고려한 요구 사항의 발견 과정이기도 하다.

5) 업무 규칙의 정밀한 포착을 보증한다

정체계화되고, 규칙의 Value 화에도 도움을 주게 된다.

6) 데이터 구조의 안정성을 최대화한다

중복된 값이 최소화되고 모든 정보들이 자기가 있어야 할 자리에 존재하게 되기 때문에 향후 발생 하게 될 모델 변화에도 유연하게 대처할 수 있다.

정규화 단계

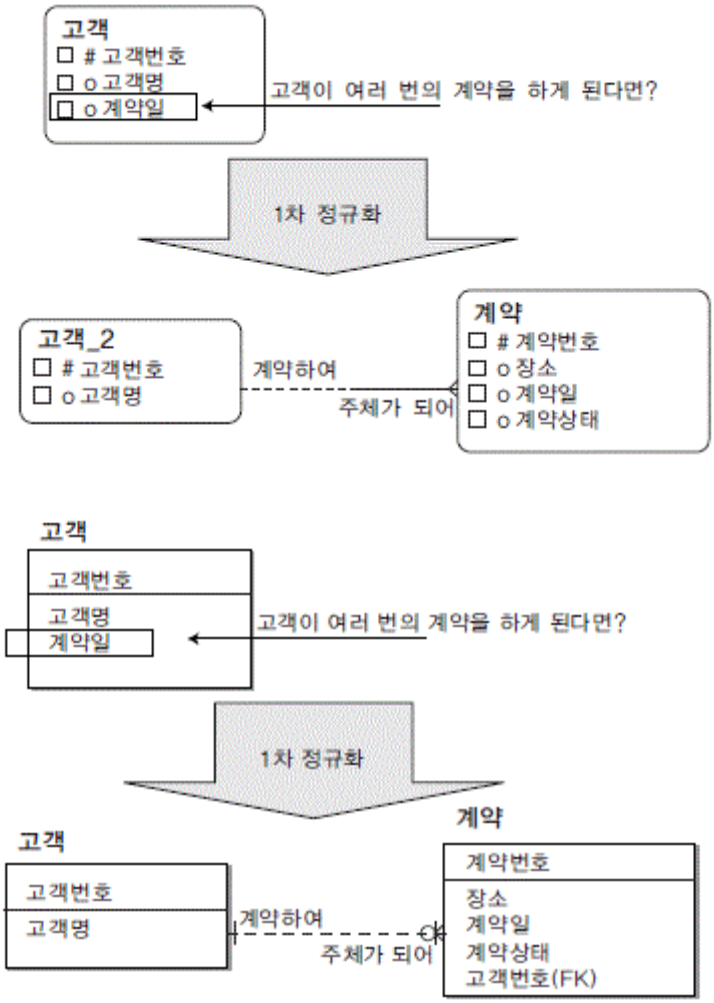
1) 1 차 정규형 (1NF, First Normal Form)

가) 정의

- 모든 속성은 반드시 하나의 값을 가져야 한다. 즉, 반복 형태가 있어서는 안된다.
- 각 속성의 모든 값은 동일한 형식이어야 한다.
- 각 속성들은 유일한 이름을 가져야 한다.
- 레코드들은 서로 간에 식별 가능해야 한다.

나) 정규화 작업

- [그림 4-3-9]에서와 같이 고객 엔터티의 계약일 속성의 값이 여러 건이 된다면 1 차 정규형을 위배하는 것이 된다.



[그림 4-3-9] 바커 표기법(위)과 IE 표기법(아래)에 따른 1차 정규형 예

- 어떤 속성이 다수의 값을 가지고 있다면 M:1 관계의 새로 엔터티를 추가한다.
- 관계형 모델에서는 관계(Relation) 정의상 한 속성이 하나의 값만을 가져야 한다. 그러므로 비정규형 관계는 엄밀히 말하면 관계로 간주할 수 없다.
- 비정규형 관계가 관계로서의 모습을 갖추기 위해선 여러 개의 복합적인 의미를 가지고 있는 속 성이 분해되어 하나의 의미만을 표현하는 속성들로 분해되어야 한다. 즉 속성수가 늘어나야 한 다.
- 비정규형 관계가 관계로서의 모습을 갖추기 위해선 하나의 속성이 하나의 값을 가질 수 있어야 하며, 이 조건을 만족시키기 위해선 로우(Row)가 늘어나야 한다. 또는 다른 관계로 분리되어야 한다.

- 분석 또는 모델링 진행 과정에서 발생하며, 최종적인 모델링 완성 단계에서는 나올 수 없다. 그러나 분석(모델링) 초기 단계에는 상세히 분해된 속성보다는 위와 같은 레벨의 속성 추출이 복잡도를 줄일 수 있으므로 실전에서는 효율적이고 유리하게 이용될 수도 있다.

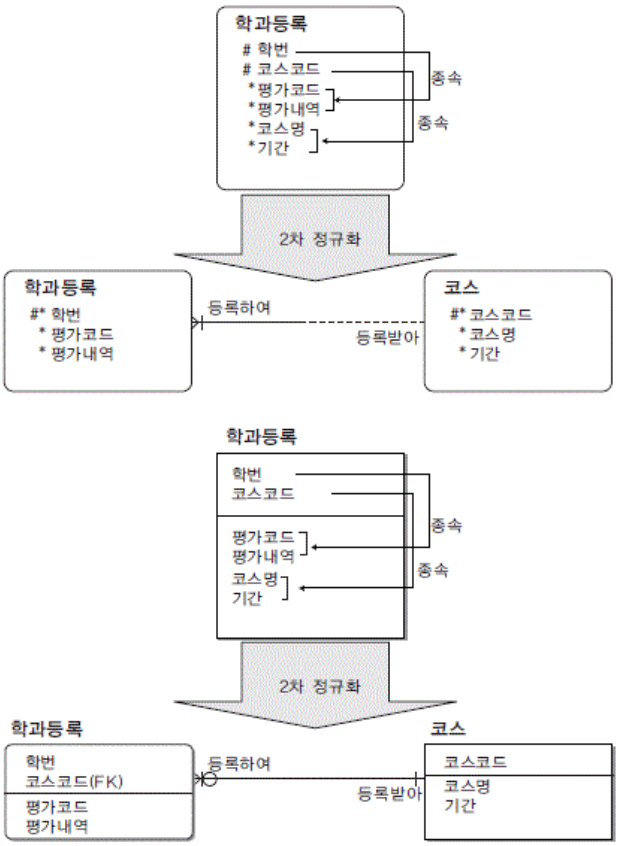
2) 2 차 정규형 (2NF, Second Normal Form)

가) 정의

- 식별자가 아닌 모든 속성들은 식별자 전체 속성에 완전 종속되어야 한다.
- 이것을 물리 데이터 모델의 테이블로 말하면 기본키가 아닌 모든 칼럼들이 기본키에 종속적이어 야 2 차 정규형을 만족할 수 있다는 것이다.

나) 정규화 작업

- [그림 4-3-10]에서와 같이 식별자가 학번 + 코스코드로 이루어진 학과등록 엔터티에서 학번 속성에 평가코드, 평가내역 속성들이 종속적이다. 그렇기 때문에 이것은 2 차 정규형을 위반하고 있는 것이다.
- [그림 4-3-10]에서와 같이 식별자가 학번 + 코스코드로 이루어진 학과등록 엔터티에서 코스코드 속성에 코스명, 기간 속성들이 종속적이다. 그렇기 때문에 이 또한 2 차 정규형을 위반하고 있는 것이다.
- 의미상의 주어 즉, 본질식별자를 알아야 식별자 부분 종속인지를 구분할 수 있다.



[그림 4-3-10] 바커 표기법(위)과 IE 표기법(아래)에 따른 2차 정규형 예

- 속성의 의미가 명확해야 종속성을 비교할 수 있다.
- 어떤 속성 식별자 전체에 종속되어 있지 않으면 잘못된 위치이며 새로운 엔터티 즉, 상위 부모 엔터티를 생성하고 UID BAR 를 상속받게 된다.

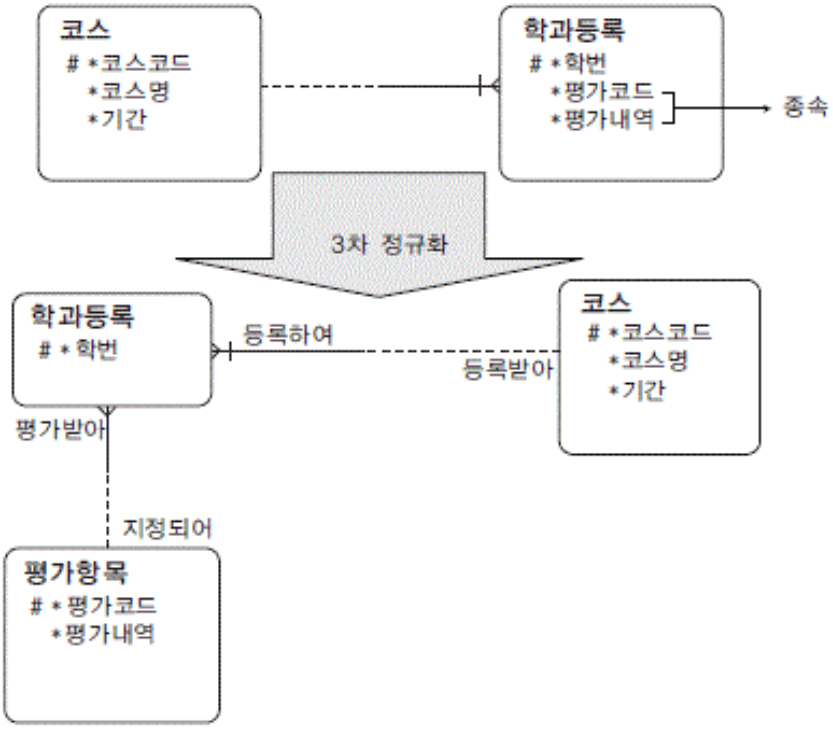
3) 3 차 정규형(3NF, Third Normal Form)

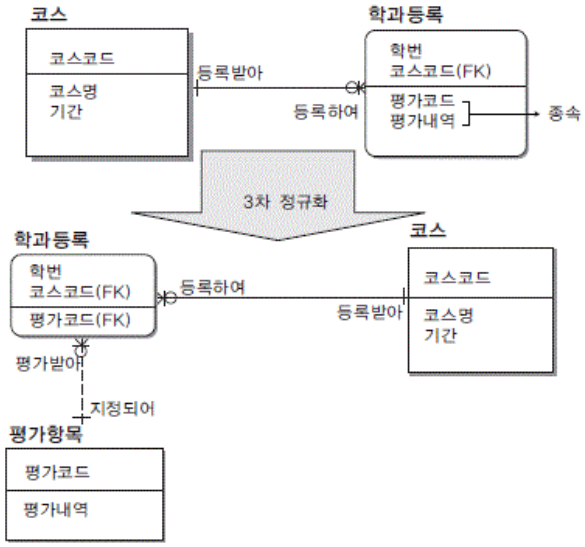
가) 정의

- 2 차 정규형을 만족하고 식별자를 제외한 나머지 속성들 간의 종속이 존재하면 안된다. 이것이 3 차 정규형을 만족하는 것이다.

나) 정규화 작업

- [그림 4-3-11]에서 학과등록 엔터티에서 평가코드, 평가내역 속성들이 서로 간에 종속적이다. 즉, 평가내역 속성은 평가코드 속성에 종속적이다. 그렇기 때문에 이것은 3 차 정규형을 위반하 고 있는 것이다.
- 3 차 정규형을 위반하고 있을 시에는 [그림 4-3-11]에서의 평가항목 엔터티처럼 부모 엔터티가 생성되고 그 부모 엔터티로부터 UID Bar 가 없는 관계를 상속받게 된다.





[그림 4-3-11] 바커 표기법(위)과 IE 표기법(아래)에 따른 3차 정규형 예

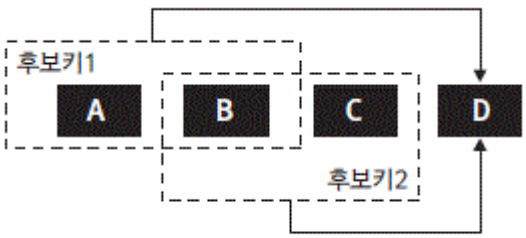
4) BCNF 정규형

가) 정의

- 모든 결정자가 키인 릴레이션이 BCNF 이다. 반대로 어떠한 결정자 하나라도 키가 아닌 릴레이 션이라면 BCNF 가 될 수 없다.
- 기존의 2 차 정규형과 3 차 정규형을 보완하려는 목적으로 만들어졌다. 즉 부분 종속이나 이행 종속이 없는 3 차 정규형도 변경 이상 현상이 나타날 수 있기 때문이고, 이것은 어떤 Non-Key 속성이 결정자로 동작하기 때문에 발생한다.

나) 3차 정규형의 문제점

- 한 릴레이션에 여러 개의 후보키(Candidate Key)가 있으며,
- 모든 후보키들이 적어도 둘 이상의 속성으로 이루어지는 복합(Composite) 키이며,
- 모든 후보키들이 적어도 하나 이상의 공통 속성이 포함되는 경우이다.



[그림 4-3-12] BCNF 정규형

다) 3차 정규형을 만족하고 BCNF가 아닌 경우

- 각 속성이 단 하나의 값으로 구성된 경우(1 차 정규형)

- Non-Key 속성인 D 는 후보키(Candidate Key)인 A+B 또는 B + C 에 완전 함수 종속이므로 2 차 정규형에 속한다.
- 또한 Non-Key 속성이 D 하나 뿐이므로 Non-Key 속성 간의 종속 관계가 존재하지 않으므로 제 3 차 정규형을 만족한다.

3. M:M 관계 해소

M:M 관계의 의미

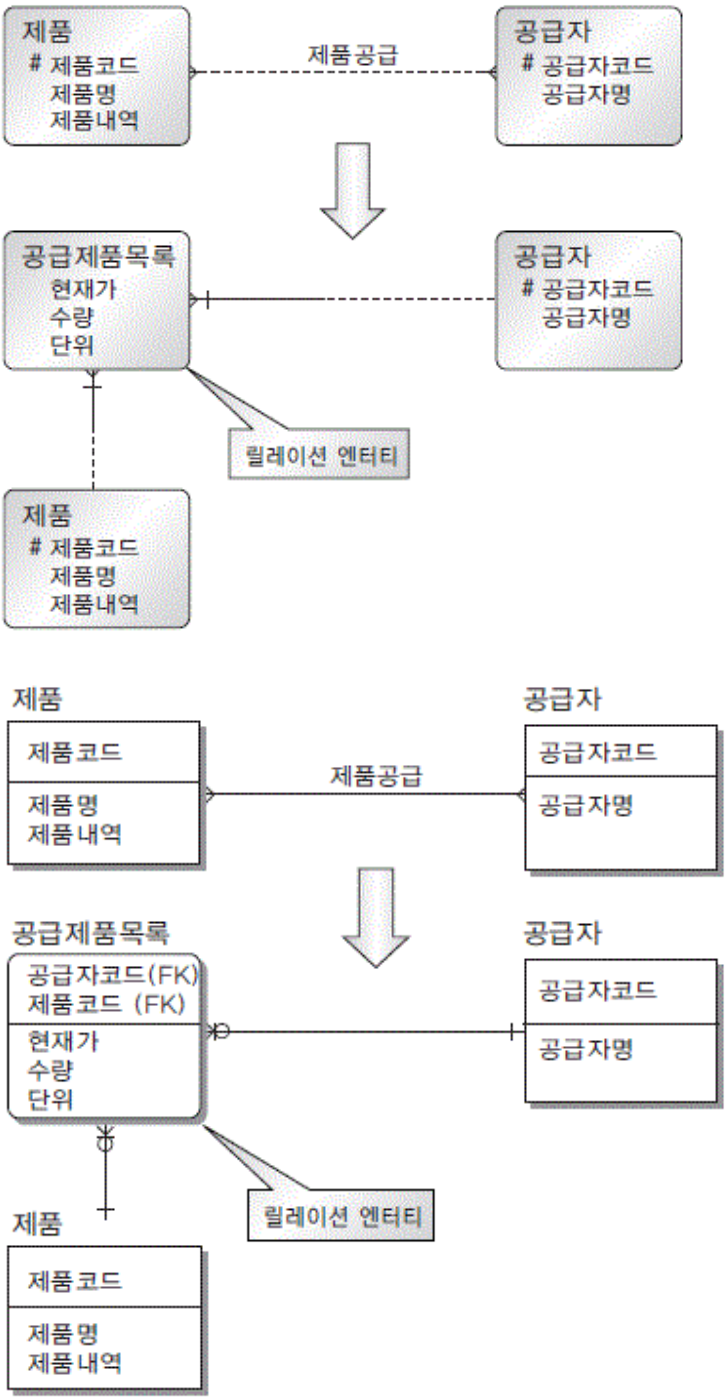
M:M 관계는 논리 데이터 모델링 과정에서 많이 나타난다. 이러한 관계는 데이터 모델이 아직 덜 완성된 모습이라고 할 수 있다. 그래서 M:M 관계는 최종적으로 완성된 데이터 모델에는 존재할 수 없는 형태라고 할 수 있다.

- 실제 업무 중 대부분은 M:M 관계이다. 즉, 기업이 관리하고 있는 많은 데이터 중에서 기업의 업무 내용에 해당하는 데이터가 이러한 M:M 관계로 표현되고 향후에 모델링이 더 진행됨에 따라 이것 이 해소된다.
- 키 엔터티와 키 엔터티 간에는 대부분 M:M 관계이다. 그래서 데이터 모델 상세화 단계에서 이러한 M:M 관계가 해소되면서 액션 엔터티가 새롭게 도출되기도 한다.
- 지속적으로 발생되는 대다수의 엔터티는 M:M 관계이다. 즉, 기업의 업무 내용을 관리하는 데이터는 대부분 이러한 관계에 의해서 생겨난 엔터티라고 볼 수 있다.

M:M 관계 해소의 의의

M:M 관계는 불특정 관계로도 알려져 있으며, 데이터 구조에 있어서 어떠한 실제적 방법으로도 구 현이 불가능하다. 이것이 이 관계를 해결하는데 충분한 이유이지만, 데이터 모델 구축 작업 초기에 그렇게 하는 데에는 또 다른 동기가 있다.

- M:M 릴레이션십은 새로운 릴레이션 엔터티를 추가하여 M:1 관계로 변경한다.
- 연관 실체(associative, relational) 엔터티는 M:M 관계 미결 시 간과해 버렸을 추가 업무 규칙 또는 업무 논리를 내포할 수 있다. 특히 업무 규칙이 정밀하게 정의됨에 따라 하위 유형(subtype)계층 구조가 연관 실체로부터 나타나기 쉽다는 점, 분석 초기에 M:M 관계가 해결되지 않은 모델에서 이들이 간과된다는 점이다.
- M:M 관계는 데이터 종속성에 대한 결정을 어렵게 하여, 모델의 논리적 완성과 부분집합 식별 능력을 제한한다.
- M:M 관계 해결시까지 모델은 불안정 상태에 머물 것이다. 모델은 정규화되지 못하고, 모델에 대한 문서화 작업도 완비되지 못할 것이다.
- [그림 4-3-13]에서와 같이 제품 엔터티와 공급자 엔터티 간에는 M:M 관계가 존재한다. 이 관계는 제품 공급이라는 업무 내용에 의해서 두 엔터티 간에 생겨난 M:M 관계이다.
- 이 관계가 해소된 결과로 공급제품목록이라는 새로운 엔터티가 만들어지고 또한 각각의 두 엔터티 즉 제품, 공급자와 1:M 의 관계를 부모로서 가지게 된다.



[그림 4-3-13] 바커 표기법(위)과 IE 표기법(아래)에 따른 M:M 관계 해소 예

실습 예제 ----------

[문] 다음 지문의 내용에 대한 엔터티를 정의하고 이들의 관계를 작성하시오.

고객은 각기 다른 배송지 정보를 설정하여 두고 주문 시에 선택할 수 있다. 단, 최초의 기본 배송지는 가입 시에 등록했 던 주소지로 설정하며, 기본 배송지는 변경할 수 있다. 고객은 여러 상품을 하나의 주문으로 묶어 처리할 수 있으며, 하나의 주문은 신용카드, 포인트 등과 같은 여러 개의 결제 수단을 사용하여 결제할 수 있다.

참조무결성 규칙 정의

- 관계 테이블의 모든 외부 식별자 값은 관련 있는 관계 테이블의 모든 주 식별자 값이 존재해야 한다.
- 실체의 주 식별자(PK)와 마찬가지로 외부 식별자(FK)도 데이터 무결성에 관한 업무 규칙을 내포하고 있다.
- 데이터베이스 설계 관점에서 선택하지 말고, 사용자의 업무 규칙에 따라 적절한 규칙을 선택한다.

입력 규칙

자식 실체의 인스턴스(Instance)를 입력할 때 참조무결성 규칙의 종류는 다음과 같다.

1) Dependent

대응되는 부모 실체에 인스턴스가 있는 경우에만 자식 실체에 입력을 허용한다.

2) Automatic

자식 실체 인스턴스의 입력을 항상 허용하고, 대응되는 부모 건이 없는 경우 이를 자동 생성한다.

3) Nullify

자식 실체 인스턴스의 입력을 항상 허용하고, 대응되는 부모 건이 없는 경우 자식 실체의 참조키 (FK)를 Null 값으로 처리한다.

4) Default

자식 실체 인스턴스의 입력을 항상 허용하고, 대응되는 부모 건이 없는 경우 참조키(FK)를 지정된 기본 값으로 처리한다.

5) Customized

특정한 검증 조건이 만족되는 경우에만 자식 실체 인스턴스의 입력을 허용한다.

6) No Effect

자식 실체 인스턴스의 입력을 조건 없이 허용한다.

삭제 규칙

부모 실체의 인스턴스를 삭제할 때(또는 그것의 주 식별자를 수정할 때) 사용되는 무결성 규칙은 다음과 같다.

1) Restrict

대응되는 자식 실체의 인스턴스가 없는 경우에만 부모 실체 인스턴스 삭제를 허용한다.

2) Cascade

부모 실체 인스턴스의 삭제를 항상 허용하고, 대응되는 자식 실체의 인스턴스를 자동 삭제한다.

3) Nullify

부모 실체 인스턴스의 삭제를 항상 허용하고, 대응되는 자식 실체의 인스턴스가 존재하면, 그것의 참 조키(FK)를 Null 값으로 수정한다.

4) Default

부모 실체 인스턴스의 삭제를 항상 허용하고, 대응되는 자식 실체의 인스턴스가 존재하면, 그것의 참 조키(FK)를 기본 값으로 수정한다.

5) Customized

특정한 검증 조건이 만족되는 경우에만 부모 실체 인스턴스의 삭제를 허용한다.

6) No Effect

부모 실체 인스턴스 삭제를 조건 없이 허용한다.

Chapter 3. 데이터 아키텍처 이해하기

uData Architecture는 Data Principle, DA Framework, Data Reference Model 통제하기 위한 Data Governance 로 구성되어 있다.

Data Principle : 전사 데이터를 관리하고, 유지하기 위한 기본 원칙을 제공한다.

Data Architecture Framework : 전사 Data 를 해석, 이해하는 기본 틀을 제공한다.(구조, 흐름)

Data Reference Model : 전사 데이터의 참조 모델의 역할을 수행하며, 전사 데이터 상호 운용성을 지원한다.(표준)

Data Governance : 전사 데이터를 관리하고, 유지, 통제 수단을 제공한다.(관리원칙, 관리조직, 관리절차, QA)

전통적인 시스템 개발을 위한 모델링 접근방법과 적의 차이가 있다.

데이터 모델링 : 기업의 정보구조를 체계적으로 나타내는 방법으로 기업의 데이터를 사용자 관점에서 인식, 분석하여 이를 표준화된 심볼을 이용하여 표현하는 기법

u 시스템 구현을 목적으로 주제영역을 분 류함으로써 중복되고 상이함 u 해당 주제영역의 데이터를 사용하는 업 무활동 중심으로 정의함 u 단위 시스템 별로 정의 된 데이터 구조 이므로 동일 데이터 일지라도 식별에 어려움이 존재함 u 논리적으로 일관된 기준이 부재하므로 통합 정보 제공에 한계가 존재함

데이터 아키텍처 : 최상위의 개괄적인 단계에서부터 데이터 베이 스 단계까지 데이터에 관한 모든 구조를 통합하 여 연계 시킨 아키텍처 또는 체계

u 전사적인 데이터 아키텍처 구축을 목적으로 분류하고 정의함 u 전사 중심 데이터인 키 주제영역을 정의하고 업무 활동을 고려하여 메인 주제영역정의 u 전사적인 참조모델을 개발하여 일관된 데이터 구조 구축 기반을 제공하기 위한 분류기준 u 개체집합이나 행위집합 단위 그룹으로 주제영역을 분류하고 데이터 모델링을 진행하므로 향후 통합에 유리

Data Architecture Framework 은 Data Structure, Data Flow, (Data Management) 세가지 요소로 구성된다.

Chapter 4. 물리 데이터 모델 생성하기

물리 데이터 모델 정의

물리 데이터 모델이란 논리적 모델을 특정 데이터베이스로 설계함으로써 생성된, 데이터를 저장할 수 있는 물리적인 스키마를 말한다. 데이터 모델의 엔터티와 서브타입은 논리적인 집합이며, 만약 관계형 데이터베이스로 설계한다면 이 단계에 와서 물리적인 테이블(Table)로 확정된다. 하나의 논리적 집합(엔터티 혹은 서브타입)은 하나 이상의 테이블이 될 수 있으며, 경우에 따라서는 속성의 일부만으로 생성될 수 있다.

물리 데이터 모델링은 논리 데이터 모델을 사용하고자 하는 각 DBMS 의 특성을 고려하여 데이터 베이스 저장 구조(물리 데이터 모델)로 변환하는 것이다. 여기에서 물리 데이터 모델링과 데이터베이스 디자인과의 개념을 정리하자면 물리 데이터 모델링은 데이터의 구조에 관련된 것들을 물리적인 모습까지 설계하는 것이고, 반면에 데이터베이스 디자인은 이러한 물리적인 모델(설계도면)을 DBMS 관점의 오브젝트로 생성하는 최적의 설계(디자인)를 하는 것이다. 데이터베이스 디자인의 예로는 오브젝트별 저장공간의 효율적 사용 계획, 오브젝트 파티셔닝 설계, 최적의 인덱스 설계 등이 여기에 속한다고 할 수 있다. 물론, 이러한 기준에 대해서 이론이 있을 수 있을 것이다. 하지만, 이 수험서에서는 이러한 기준을 가지고 물리 데이터 모델링과 데이터베이스 디자인을 구분하였다.

물리 데이터 모델 의의

물리적 데이터 모델링은 관계 데이터 모델링(RDM, Relation Data Modeling)이라고도 한다. 사전적으로 작성된 논리적 데이터 모델을 각각의 관계형 데이터베이스 관리 시스템의 특성, 기능, 성능 등을 고려하여 데이터베이스의 물리적인 구조(Schema)를 작성해 가는 과정이다. 많은 사람들이 물리 데이터 모델링을 단순히 설계된 논리 데이터 모델의 개체 명칭이나 속성 명칭, 데이터 형태, 길이, 영역값등을 변환하는 것 정도로 생각하고 있다. 그러나 물리적 데이터 모델링 단계는 논리 데이터 모델에서 도출된 내용 변환을 포함하여 데이터의 저장공간, 데이터의 분산, 데이터 저장 방법 등을 함께 고려하는 단계이다. 또한, 이 과정에서 결정되는 많은 부분이 데이터베이스 운용 성능(Performance)으로 나타나므로 소홀히 다루면 안된다.

논리 데이터 모델–물리 데이터 모델

하나의 논리적 데이터 모델을 가지고 서로 다른 형태의 물리적 데이터 모델을 설계하는 경우는 크게 네 가지로 나눌 수가 있다.

분산 데이터베이스 구축 시

분산 데이터베이스를 구축하고자 할 때 노드별로 자신이 원하는 형태의 물리적 모델을 생성하고자 할 때 적용하는 경우이다.

물리 데이터 모델 비교

각자 나름대로의 특징을 가지고 있는 여러 개의 물리적 모델을 생성하여 종합적인 비교 검토를 하기 위하여 적용하는 경우이다.

물리적 환경의 변화

논리적인 모델에는 변화가 발생하지 않지만 물리적인 환경에는 변경이 발생했을 때 기존의 물리적 모델을 새로운 목표 물리적 모델로 개선하고자 할 때 적용하는 경우이다.

물리적 모델의 형상관리

물리적 모델이 세월의 흐름에 따라 조금씩 변해갈 때 그 이력을 관리할 목적으로 여러 개의 버전을 보유하고자 할 때 사용하는 경우이다.

논리 데이터 모델–물리 데이터 모델 변환(Transformation) 용어

논리 영역과 물리 영역을 보는 시각은 여러 가지 관점에서 조금씩은 다르다. 특히 학자, 모델링 툴도 이러한 차이는 존재한다. 하지만, 하지만 이 책에서는 다음과 같은 기준으로 논리 데이터 모델의 영역과 물리 데이터 모델의 영역을 구분하여 접근하고 있다.



Entity → Table
Arribute → Column
Primary UID → Primary Key
Secondary(Alternate) UID → Unique Key
Relationship → Foreign Key
Business Constraints → Check Constraints

[그림 4-4-1] 논리 모델 – 물리 모델 변환 용어

엔터티-테이블 변환

테이블 설명

테이블은 데이터를 저장하기 위해서 생성된 데이터베이스에서의 가장 기본적인 오브젝트이다. 기 본적인 모습은 아래와 같은 모양으로 만들어지게 된다.

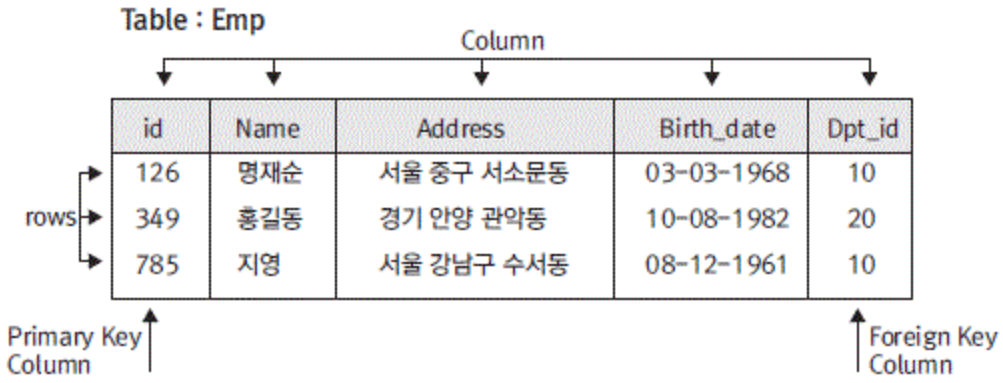
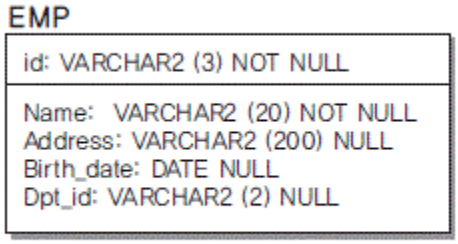


Table Diagram : EMP

EMP(Employee)
Id VARCHAR2(3)
* Name VARCHAR2(20)
o Address VARCHAR2(200)
o Birth_date DATE
o Dpt_id VARCHAR2(2)



[그림 4-4-2] 바커 표기법(위)과 IE 표기법(아래)에 따른 테이블 레이아웃

1)테이블(Table)

테이블은 기본적으로 칼럼(Column)과 로우(Row)를 가진다. 각각의 칼럼은 지정된 유형의 데이터 값을 저장하는 데 사용된다. [그림 4-4-2]에서 테이블 EMP 는 사원 정보를 저장하기 위해서 생성된 구조이다.

2)로우(Rows)

테이블의 한 로우에 대응. 튜플 , 인스턴스, 어커런스라고도 한다.

3)칼럼(Columns)

각 사원 개개인의 관리 항목에 대한 Value 를 저장한다.

4)기본키(Primary keys)

하나의 칼럼 혹은 몇 개의 칼럼 조합으로 어떤 경우라도 테이블 내에 동일한 값을 갖는 튜플이 존재하지 않도록 한다.

5)외래키(Foreign keys)

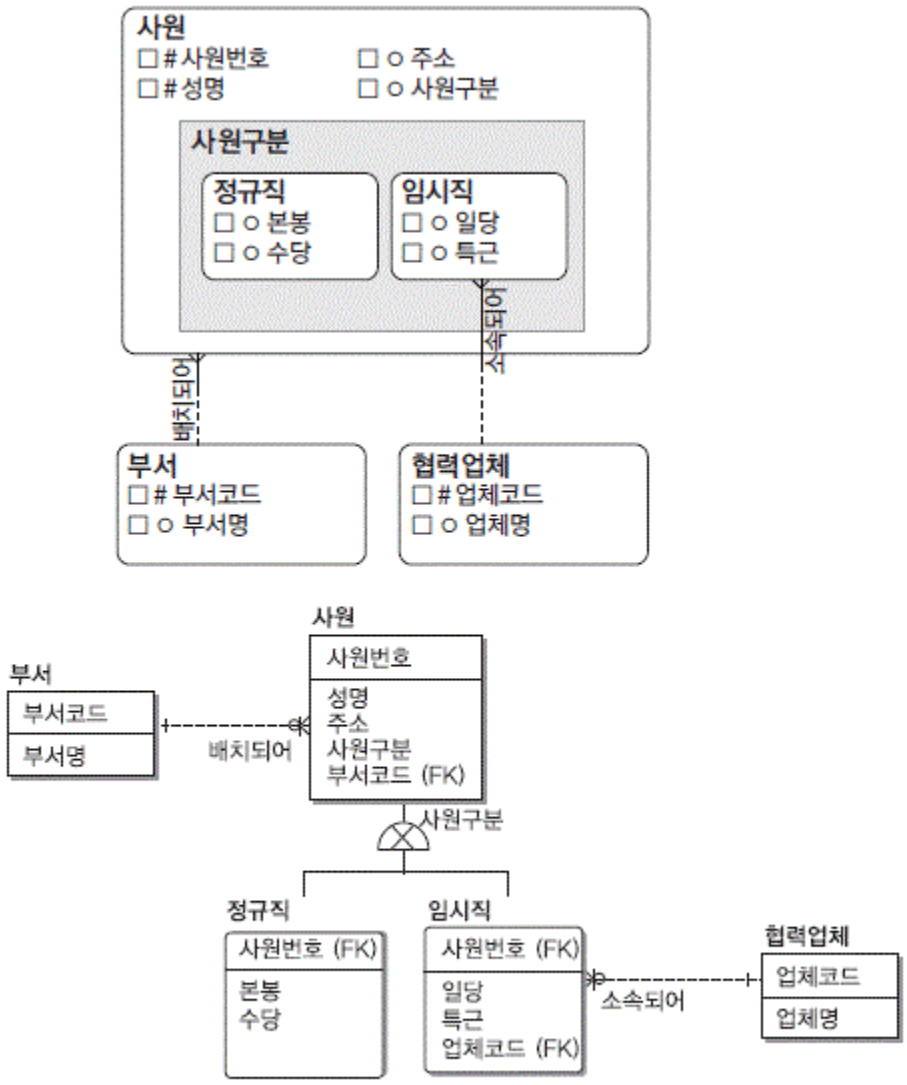
외부 데이터 집합과의 관계를 구현한 구조이다.

서브타입 변환(Transformation)

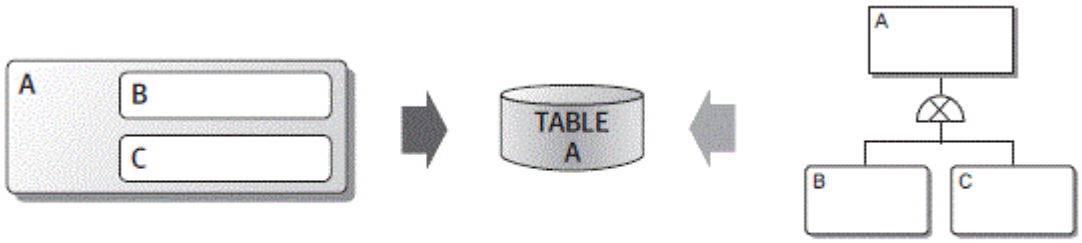
논리 데이터 모델에서는 비즈니스 또는 업무를 데이터 모델로 표현하기 위해서는 최대한 상세한 표현이 필수적이다. 그러한 목적을 달성하기 위해서 가능하면 집합(엔터티)의 구성을 서브타입을 사용하여 구체적으로 표현하는 것이 통상적이다. 또한 각각의 서브타입들이 독립적인 속성(Attribute), 관계(Relationship)를 가지고 있는 경우에는 이러한 서브타입(Sub Type)을 사용한 집합의 표현은 필수적이다. 이렇게 논리 모델링에서 표현된 서브타입은 물리 데이터 모델에서는 테이블의 형태로 설계되어야 한다. 하지만 이러한 서브타입 모델은 단순 엔터티-테이블 변환과는 다른 몇 가지 방법을 통해서 테이블로 변환 작업을 하게 된다.

- 슈퍼타입 기준 테이블 변환
- 서브타입 기준 테이블 변환
- 개별타입 기준 테이블 변환

[그림 4-4-3]과 같은 구체적인 논리 데이터 모델을 통해서 위 방법들을 살펴본다.



[그림 4-4-3] 바커 표기법(위)과 IE 표기법(아래)에 따른 서브타입 예



[그림 4-4-4] 바커 표기법(좌)과 IE 표기법(우)에 따른 슈퍼타입 기준 테이블 변환 예

슈퍼타입 기준 테이블 변환

- 서브타입을 슈퍼타입에 통합하여 하나의 테이블로 만든다
- 이 통합된 테이블에는 모든 서브타입의 데이터를 포함해야 한다
- 주로 서브타입에 적은 양의 속성이나 관계를 가진 경우에 적용된다

가) 절차

- 슈퍼타입으로 테이블 명칭 부여
- 서브타입을 구분할 수 있도록 칼럼 추가
- 슈퍼타입의 속성을 칼럼명으로
- 서브타입의 속성을 칼럼명으로
- 슈퍼타입의 관계를 FK 로
- 서브타입의 관계를 FK 로

나) 테이블 사례

[그림 4-4-3]의 논리 모델에서의 서브타입을 하나의 테이블로 통합하는 경우 테이블의 모습은 [그림 4-4-5]의 사례 데이터 표와 같다.

테이블명 : EMPLOYEE

칼럼명	EMPNO	ENAME	TYPE	SAL	DEPT	HRATE	OTRATE	UNION
키형태	PK				FK1			FK2
Nulls/ Unique	NN.U	NN	NN		NN			
견본 데이터	7540	홍길동	1	890000	40			
	5579	성삼문	1	950000	35			
	6714	김이박	1	827000	20			
	9451	오재구	2	20000	25	60000	1000	201
	3040	유성원	2	35000	40	80000	1500	180

[그림 4-4-5] 테이블 사례 예

- TYPE
  서브타입을 구분할 수 있는 칼럼이다. 즉 사원 구분에 해당하는 칼럼이다.
- DEPT
  위의 구분이 정규직일 경우에 부서로부터의 관계로 인해서 생성된 칼럼이다.
- UNION
  위의 구분이 임시직일 경우에 협력 업체로부터의 관계로 인해서 생성된 칼럼이다.

다) 하나의 테이블로의 통합이 유리한 경우

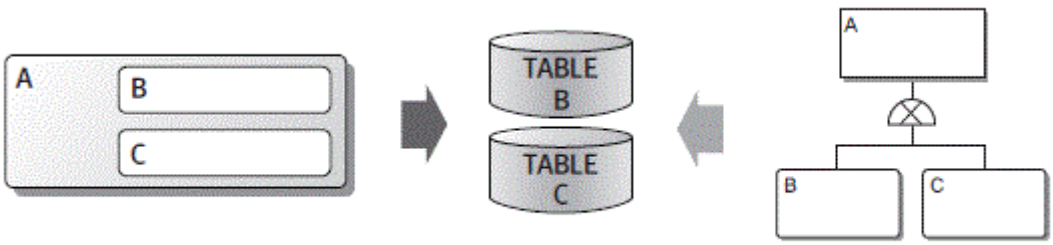
- 데이터 액세스가 좀더 간편
- 뷰를 활용하여 각 서브타입만을 액세스하거나 수정 가능
- 수행 속도가 좋아지는 경우가 많다
- 서브타입 구분없는 임의 집합의 가공 용이
- 다수의 서브타입을 통합한 경우 조인(JOIN) 감소 효과가 크다
- 복잡한 처리를 하나의 SQL 로 통합하기가 용이

라) 하나의 테이블로의 통합이 불리한 경우

- 특정 서브타입의 NOT NULL 제한 불가
- 테이블의 칼럼 수가 증가
- 테이블의 블럭 수가 증가
- 처리시마다 서브타입의 구분(TYPE)이 필요해지는 경우가 많다
- 인덱스(INDEX) 크기가 증가

서브타입 기준 테이블 변환

- 슈퍼 타입 속성들을 각 서브타입에 추가하여 각각의 서브타입마다 하나의 테이블로 만든다
- 분할된 테이블에는 해당 서브타입의 데이터만 포함돼야 한다
- 주로 서브타입에 많은 양의 속성이나 관계를 가진 경우에 적용된다



[그림 4-4-6] 바커 표기법(좌)과 IE 표기법(우)에 따른 서브타입 기준 테이블 변환 예

가) 절차

- 서브타입마다 테이블 명칭 부여
- 서브타입의 속성을 칼럼명으로
- 테이블마다 슈퍼타입의 속성을 칼럼으로
- 서브타입마다 해당되는 관계들을 FK 로
- 테이블마다 슈퍼타입의 관계를 FK 로

나) 테이블 사례

위의 [그림 4-4-3] 논리 모델에서의 서브타입을 여러 개의 테이블로 분할하는 경우 테이블의 모습은 [그림 4-4-7]과 [그림 4-4-8]의 데이터 사례 표와 같다. [그림 4-4-7]은 정규직 사원에 대한 표본 테이블의 모습과 인스턴스를 나타낸다. [그림 4-4-8]은 임시직 사원에 대한 표본 테이블의 모습과 인스턴스를 나타낸다.

테이블명 : REG_EMP

칼럼명	EMPNO	ENAME	SAL	DEPT
키형태	PK			FK
Nulls/ Unique	NN.U	NN	NN	NN
견본 데이터	9840	김문기	920000	40
	2543	박종민	850000	35
	6587	정종수	927000	20
	9476	심주현	720000	35
	5476	장국선	935000	25

[그림 4-4-7] 테이블 예 : 정규직 사원

테이블명 : TEM_EMP

칼럼명	EMPNO	ENAME	HRATE	OTRATE	UNION	DEPT
키형태	PK				FK1	FK2
Nulls/ Unique	NN.U	NN	NN	NN	NN	NN
견본 데이터	7540	홍길동	20000	6500	201	40
	5579	성삼문	50000	8900	180	35
	6714	김이박	27000	6500	201	20
	9451	오재구	20000	6000	201	35
	3040	유성원	35000	8000	180	25

[그림 4-4-8] 테이블 예 : 임시직 사원

다) 여러 개의 테이블로 분할한 경우가 유리한 경우

- 각 서브타입 속성들의 선택 사양 명확한 경우에 유리하다.
- 처리시마다 서브타입 유형 구분이 불필요하다.
- 전체 테이블 스캔시 유리하다.
- 단위 테이블의 크기가 감소한다.

라) 여러 개의 테이블로 분할한 경우가 불리한 경우

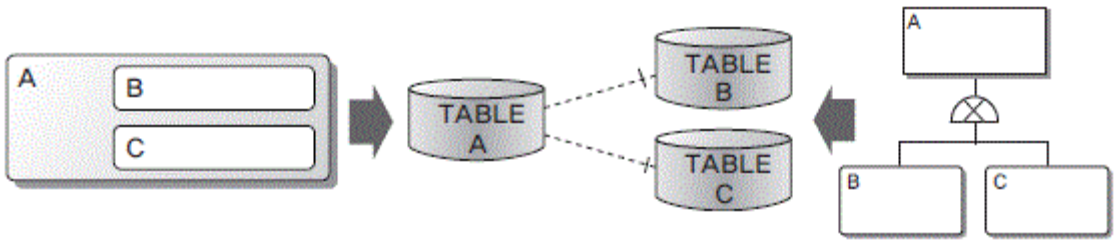
- 서브타입 구분 없이 데이터 처리하는 경우 UNION 이 발생한다.
- 처리 속도가 감소하는 경우가 많다
- 트랜젝션 처리시 여러 테이블을 처리하는 경우가 증가한다.
- 복잡한 처리의 SQL 통합이 어려워진다.
- 부분 범위 처리가 불가능해 질 수 있다.
- 여러 테이블을 합친 뷰는 조회만 가능하다.
- UID 유지관리가 어렵다.

개별타입 기준 테이블 변환

- 슈퍼타입과 서브타입을 각각 테이블로 변환한 경우이다.
- 슈퍼타입과 서브타입 테이블 간에는 1:1 관계가 생성된다.(한 쪽을 모두 합치면 전체와 같게 된다는 면에서 개념적으로 아크 관계와 유사)

다음의 여러 가지 경우를 만족할 때 사용

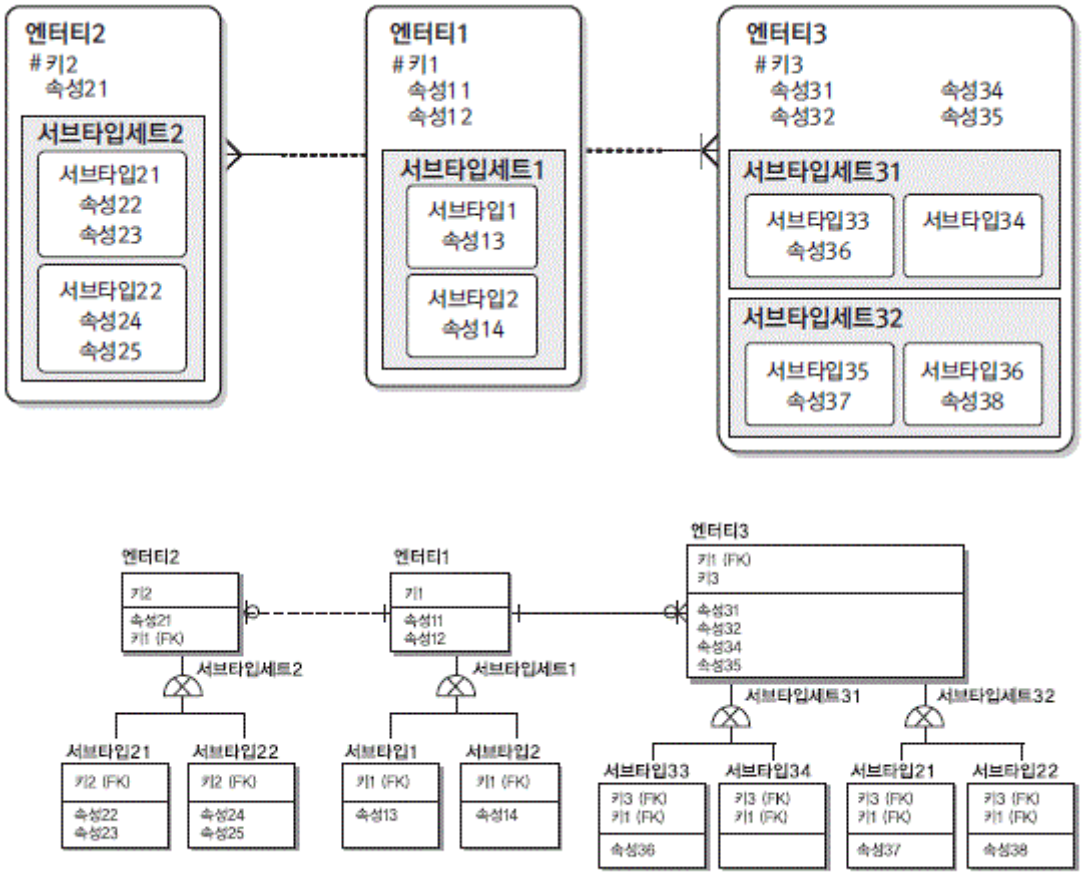
- – 전체 데이터 처리가 빈번하게 발생할 경우에 적용한다.
- – 서브타입의 처리는 주로 독립적으로 발생할 경우에 적용한다.
- – 테이블을 통합했을 때 칼럼의 수가 너무 많아지는 경우에 적용한다.
- – 서브타입의 칼럼 수가 많은 경우에 적용한다.
- – 트랜잭션이 주로 공통 부분(슈퍼타입)에서 발생한다.
- – 슈퍼타입의 처리 범위가 넓고 빈번하여 단일 테이블 클러스터링을 해야 할 때 적용한다.



[그림 4-4-9] 바커 표기법(좌)과 IE 표기법(우)에 따른 개별타입 기준 테이블 변환 예

서브타입 변환 예

[그림 4-4-10]과 같은 논리적 데이터 모델이 있다고 하자.



[그림 4-4-10] 바커 표기법(위)과 IE 표기법(아래)에 따른 서브타입 변환 대상 예

[그림 4-4-10]의 데이터 모델을 물리적 모델로 전환하는 방법은 다양하다. 하나의 데이터 모델을 이용하여 하나 이상의 노드(분산 데이터베이스를 구현하였을 때의 각각의 단위 데이터베이스)에 자 신만의 데이터베이스를 설계할 수 있을 것이다. 데이터 모델의 전부를 물리적 모델로 전환할 수도 있 지만 필요에 따라 원하는 것만 전환하고자 할 수도 있을 것이다.

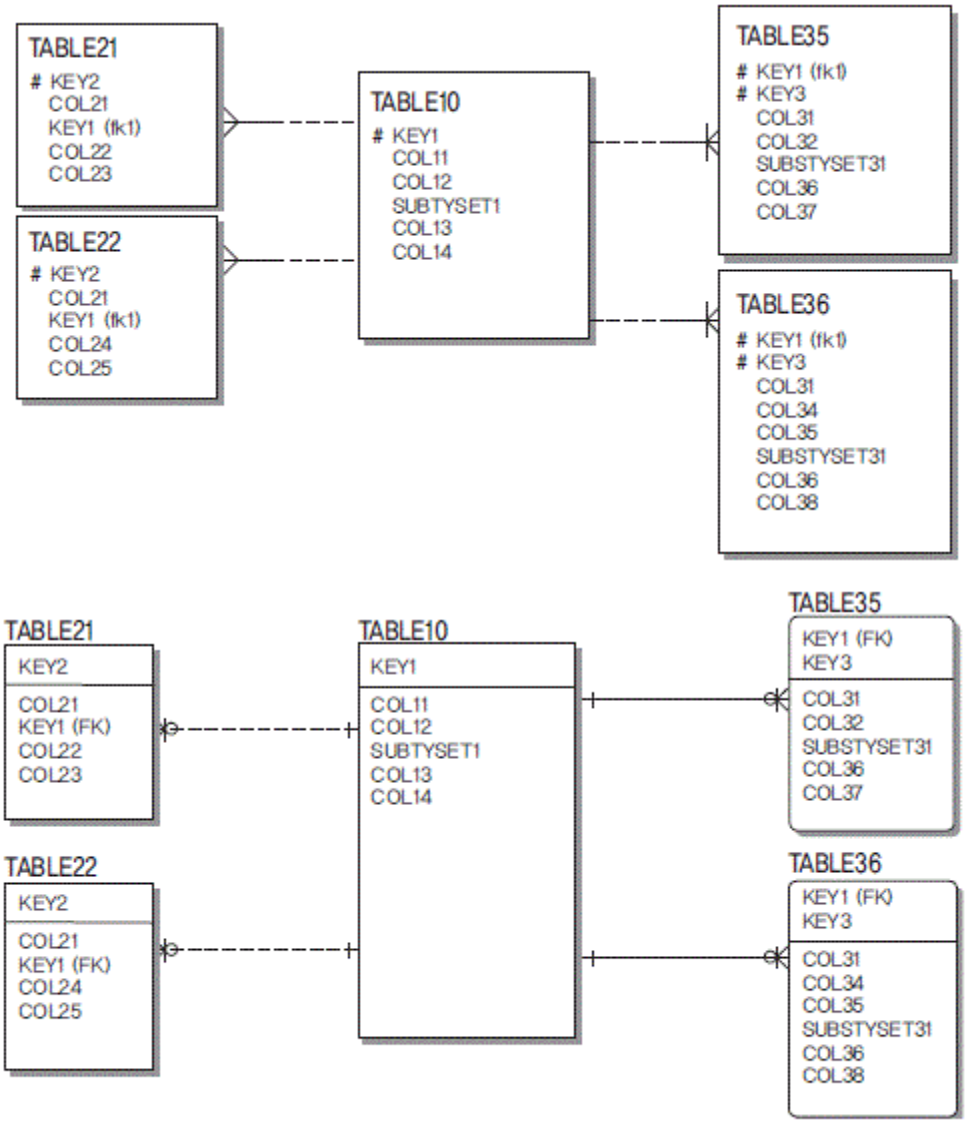
뿐만 아니라 엔터티를 하나의 테이블로 할 수도 있겠지만 각각의 서브타입 혹은 몇 개를 묶어서 하 나의 테이블로 결정하는 경우도 있을 수 있다. 만약 [그림 4-4-10]의'엔터티 3'처럼 서브타입 세트 를 이용하여 여러 차원에서 분류한 서브타입을 가지고 있다면 자신이 원하는 서브타입 세트의 일부서브타입만 테이블로 전환하는 것도 가능하다. 하나의 엔터티를 여러 개의 테이블로 설계하고 싶다 면 최소한 논리적 데이터 모델에는 서브타입으로 반드시 정의되어 있어야 가능하다. 즉, 물리적 모델 을 결정하는 단계에서

기존에 정의해 둔 서브타입이 아닌 다른 방법으로 테이블을 분할하고 싶다면 논리적 데이터 모델에 새로운 서브타입 세트를 추가로 정의해야 한다는 것을 의미한다.

속성을 물리적 모델로 전환하는 경우에도 전부나 일부만 선택하는 것은 얼마든지 가능하다. 이러 한 개념을 활용하면 하나의 논리적 데이터 모델을 이용하여 여러 개로 수직 분할된 물리 모델을 생성 할 수 있다.

1) Case 1 : 서브타입을 테이블로 분리

속성에 붙어 있는 숫자가 같은 칼럼이 서로 변환된 것으로 간주한다.(예: 속성 31 --> COL31)



[그림 4-4-11] 바커 표기법(위)과 IE 표기법(아래)에 따른 물리 모델 예제 1

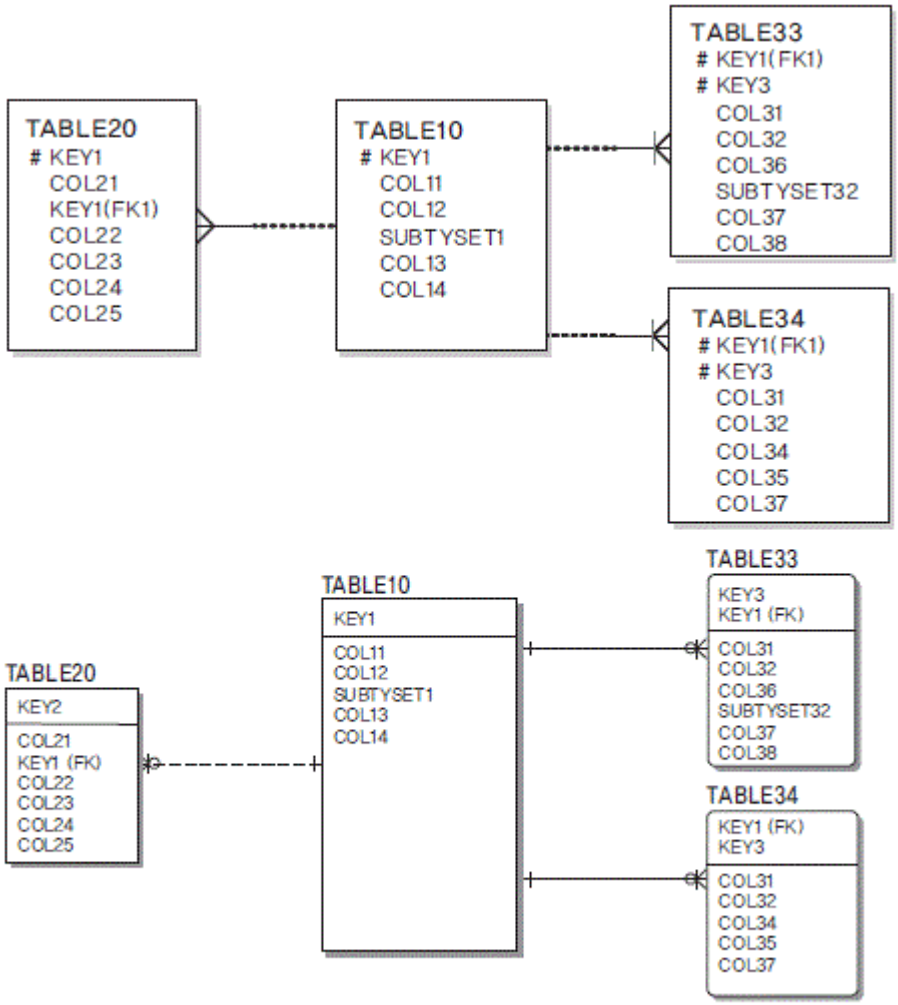
- 엔터티 1 은 그대로 TABLE10 으로 전환되었다. 서브타입 세트란 일종의 구분코드라는 속성이라고 할 수 있으며, 서브타입세트 1 은 칼럼(SUBTYSET1)으로 전환되었다. 엔터티 1 에 있던 서브타입 1,2 는 서브타입세트 1 의 속성에 들어가는 값의 내용이므로 칼럼으로 전환할 필요가 없다.
- 엔터티 2 는 서브타입별로 테이블을 분리한 모습이다. 서브타입 21 은 TABLE21 로, 서브타입 22 는 TABLE22 로 전환되었다. 공통 속성인 키 2, 속성 21 은 양쪽 모두에 전환되었고, 엔터티

- 엔터티 3 은 두 종류의 서브타입 세트를 가지고 있다. 위의 물리 모델은 이 중에서 서브타입세트 32 에 있는 서브타입별로 분리한 모습이다. 자세히 살펴보면 슈퍼타입에 있던 공통 속성 중에서 자신이 필요로 하는 일부분만 전환되었음을 발견할 수 있다. 바커표기법으로 표현할 모델에서 관계선에 세로선(UID BAR)이 붙어 있다면 식별자의 일부가 되겠다는 뜻이므로 물리적 모델이 될 때는 당연히 기본키(PK, Primary Key)이면서 외래키(FK, Foreign Key)가 되어야 한다.

2) Case 2 : 서브타입을 통합 테이블로 생성

물리적 모델에서는 [그림 4-4-12] 같이 동일한 논리적 모델을 기반으로 했지만 앞서 예제와는 다른 모습의 모델을 만들 수도 있다.

[그림 4-4-12]의 물리 모델은 앞서 소개한 물리적 모델과는 테이블의 개수도 다르며, 경우에 따라 테이블 명칭은 동일하지만 내용은 다르게 정의할 수도 있다.



[그림 4-4-12] 바커 표기법(위)과 IE 표기법(아래)에 따른 물리 모델 예제 2

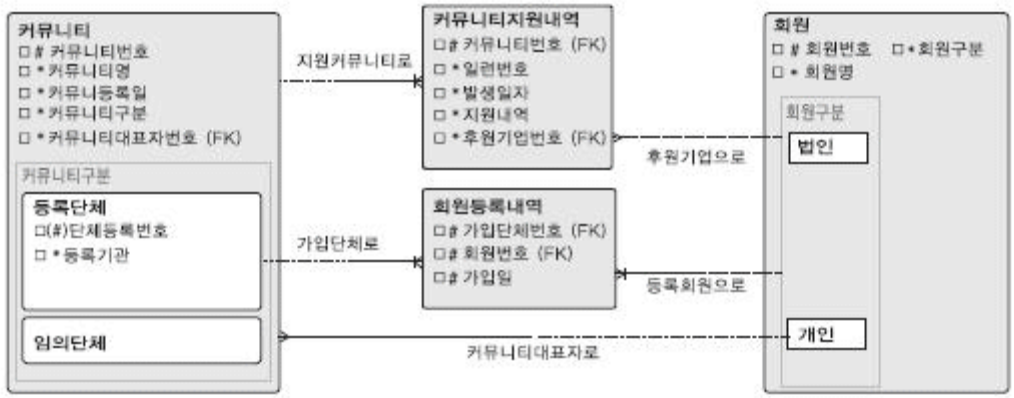
실습 예제

[문] 다음 지문의 내용을 보고 제시된 커뮤니티관리 논리 데이터 모델을 적합한 물리 데이터 모델로 변환하시오.

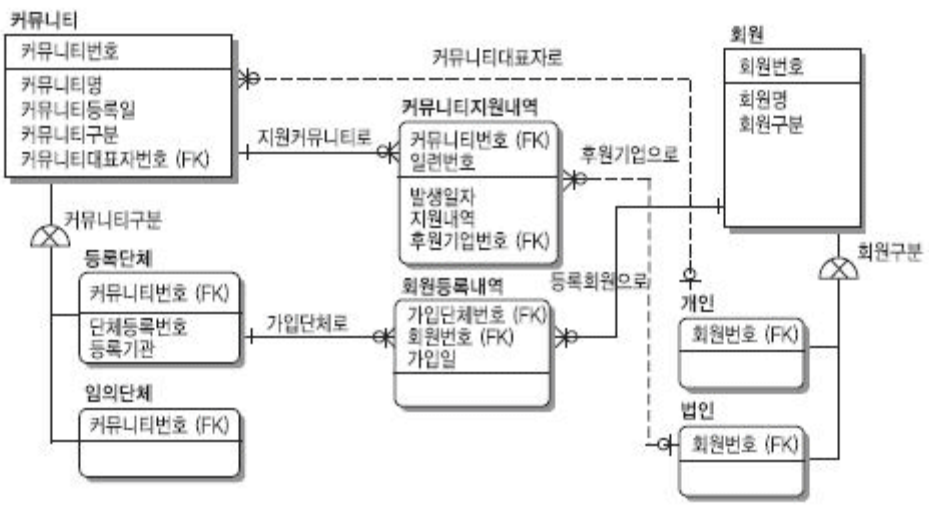
- 우리가 관리하는 커뮤티니에 대해 전체를 대상으로 하는 이벤트나 검색 경우가 빈번하다.
- 커뮤니티 대표자가 누구인지는 등록단체에 대해서만 관리할 것이다.
- 사실상 등록단체에 가입한 회원은 개인회원뿐이고, 앞으로도 그럴 것이다.
- 법인 회원과 개인 회원은 관리할 항목들이 많이 다르고, 같이 검색할 일은 별로 없다.

칼럼명 약어 사용 규칙 : 단어와 단어는 '_'로 연결

커뮤니티 → cmnt,	번호 → no,	명(이름) → nm,	등록일 → endt,
구분 → gb,	대표자 → ldr,	단체 → org,	등록번호 → regno,
등록기관 → regag,	일련 → ser,	발생일자 → actdt,	지원내역 → supdtl,
회원 → mbr,	등록내역 → regdtl,	가입단체 → jnorg,	가입일 → jndt,
후원기업 → spnco,	법인 → busi,	개인 → prsn,	.....



〈바커 표기법에 따른 커뮤니티관리 논리 데이터 모델〉



〈IE 표기법에 따른 커뮤니티관리 논리 데이터 모델〉

테이블 목록 정의서

갑을 은행 계정계 시스템	갑을 은행 계정계 시스템									
	모델명	인버터 V3					단 계			
	문서번호		버 전				작성일자	2005/10/23		

테이블 목록

테이블 ID	테이블 명	Type	분류	Table Space	Partition 여부	발생주기	월간발생량	보존기한(월)	총건수	수정일
EMP	Employee	STANDARD								2005/01/19
inv_issuelist	미결사항리스트	STANDARD								2005/01/19
inv_issue	미결사항	STANDARD								2005/01/19
inv_attribute	속성	STANDARD								2005/01/19
usr_menugrp	메뉴구성정보	STANDARD								2005/01/19
usr_menu	단위메뉴	STANDARD	Key							2005/01/19
da_model	주제영역(DA_MODEL)	STANDARD								2005/01/19
usr_alloc	사용자할당	STANDARD	Main							2005/01/19
usr_user	사용자	STANDARD	Key							2005/01/19
USR_ROLE	역할	STANDARD	Key							2005/01/19
inv_index	INDEX	STANDARD								2005/01/19
inv_project	Project	STANDARD	Key							2005/01/19

[그림 4-4-13] 테이블 목록 정의서 예

엔터티를 테이블로 변환하는 과정이 완료되면 [그림 4-4-13]과 같은 테이블 목록 정의서를 작성한다. 테이블 목록 정의서는 줄여서 테이블 목록이라 부르기도 하며, 전체 테이블이 목록으로 요약 관리되어야 한다.

속성-칼럼 변환

속성이나 관계를 물리 데이터 모델 객체로 변환하는데 있어서 사례 데이터 표를 작성해 보는 것은 실제 데이터가 어떤 형태로 저장되는지, 어떠한 예외사항이 존재할 수 있는지 등을 용이하게 파악할 수 있도록 도움을 주기 때문에 여기서는 사례 데이터 표를 활용하여 변환 과정을 설명한다.

일반 속성 변환

- 엔터티에 있는 각 속성들에 대한 칼럼명을 사례 데이터 표의 칼럼명 란에 기록한다
- 칼럼의 명칭은 속성의 명칭과 반드시 일치할 필요는 없으나 프로그래머와 사용자의 혼동을 피하기위해 가능한 표준화된 약어를 사용한다
- 표준화된 약어의 사용은 SQL 해독 시간을 감소시킨다.
- SSQL 의 예약어(reserved word)의 사용을 피한다.
- 가급적 칼럼 명칭은 짧은 것이 좋으며, 짧은 명칭은 개발자의 생산성에 긍정적인 영향을 준다.
- 필수 입력 속성은 Nulls/Unique 란에 NN 을 표시한다.

- 실제 테이블에 대한 설계를 검증하기 위한 목적으로 가능하다면 표본 데이터를 입력시킨다.

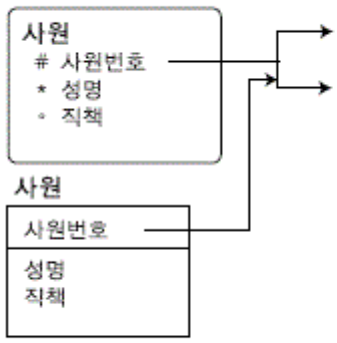
Primary UID 기본키(Primary Key) 변환

논리 데이터 모델에서의 Primary UID 는 물리 데이터 모델에서는 기본키로 생성된다. 실제 DDL 에서는 기본키 제약 조건의 형태로 오브젝트가 생성된다.

1) 변환 절차

- 사례 데이터 표의 키 형태 란에 엔터티의 Primary UID 에 속하는 모든 속성에 PK 를 표시
- PK 로 표시된 모든 칼럼은 Nulls/Unique 란에 반드시 NN,U 로 표시되어야 함
- 여러 개의 칼럼으로 UID 가 구성되어 있는 경우는 각각의 칼럼에 NN,U1 을 표시
- 또 다른 Unique Key(Secondary UID)가 있다면 U2 로 표시

2) 변환 예



칼럼명	EMPNO	ENAME	JOB
키 형태	PK		
Nulls/ Unique	NN,U	NN	
견본 데이터	1354	홍길동	부장
	4562	박경수	사원
	2763	김정구	
	2874	신동파	차장
	6538	정명훈	

[그림 4-4-14] 기본키 변환

Primary UID(관계의 UID Bar) 기본키(Primary Key) 변환

논리 데이터 모델에서 Primary UID 에 속하는 속성 중에는 해당 엔터티 자체에서 생성된 것도 존 재하지만 다른 집합(엔터티)으로부터의 관계에 의해서 생성되는 UID 속성(관계 속성)도 존재 한다. 이러한 관계 속성 UID 의 변환은 기본적인 속성 UID 변환과는 약간 다르다.

1) 변환 절차

- 테이블에 외래키 칼럼을 포함시킨다
- PK 의 일부분으로 표시
  -Nulls/Unique 란에 각각 NN,U1 을 표시

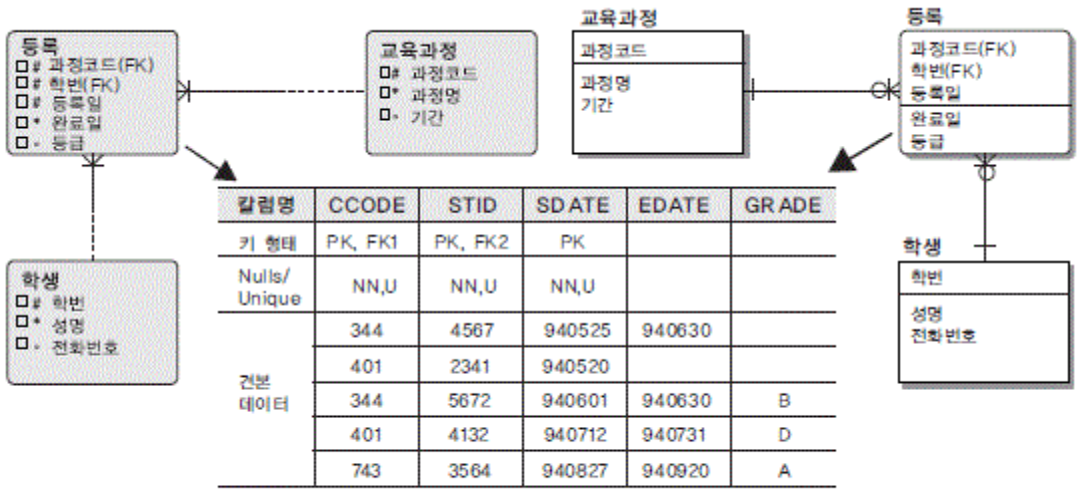
-키 형태란에 PK, FK 를 표시

-여러 UID BAR 가 있는 경우는 (PK, FK1), (PK, FK2), …

-여러 칼럼으로 구성된 경우 PK,FK1 을 각각 표시

- 추가된 FK 칼럼에 표본 데이터를 추가

2) 변환 예



칼럼명	CCODE	STID	SDATE	EDATE	GRADE
키 형태	PK, FK1	PK, FK2	PK		
Nulls/ Unique	NN,U	NN,U	NN,U		
기본 데이터	344	4567	940525	940630	
	401	2341	940520		
	344	5672	940601	940630	B
	401	4132	940712	940731	D
	743	3564	940827	940920	A

[그림 4-4-15] 바커 표기법(좌)과 IE 표기법(우)에 따른 기본키 변환 예 : 외래키에 의한

Secondary (Alternate) UID Unique Key 변환

논리 데이터 모델에서 정의한 Secondary UID 또는 Alternate Key 들은 해당 집합과 상태 집합과의 선택적인 관계를 가질 수 있도록 하는 데 중요한 역할을 수행한다. 이러한 Secondary UID 들은 물리 데이터 모델에서는 Unique Key 로 생성된다. 변환 절차는 기본적으로 Primary UID 변환 절차와 동일하다.

테이블 정의서

	갑을 은행 계정계 시스템				
	모 델 명	인버터 V3		단 계	
	문서번호		버 전	작성일자	2005/10/23

테이블 정의서

테이블 ID	EMP	테이블명	Employee	Table Space		시스템	인버터V3	총건수		월간발생량	
실체	System										
테이블정의											
파티션 정의											
파티션키											
데이터처리형태											
특이사항											
Note											

No	칼럼명	한글명	Type	Len	Null	UK	PK	FK	참조 테이블	참조칼럼	유형	Validation Rule	비고
1	Id	시스템 ID	VARCHAR2	3			Y						
2	Name	시스템 Name	VARCHAR2	20									
3	Address	Description	VARCHAR2	200	Y								
4	Birth_date	일자	DATE		Y						System		
5	Dot_id	부서	VARCHAR2	2	Y						System		

[그림 4-4-16] 테이블 정의서 예

기본적인 테이블 변환, 칼럼 변환 작업이 완성되면 [그림 4-4-16]과 같은 테이블 정의서를 생성할 수 있다. 대부분의 시스템 구축 프로세스상에서 개발자들이 프로그램 개발을 수행하는 단계에서 가장 많이 참조하는 산출물 중 하나이다 .

관계 변환

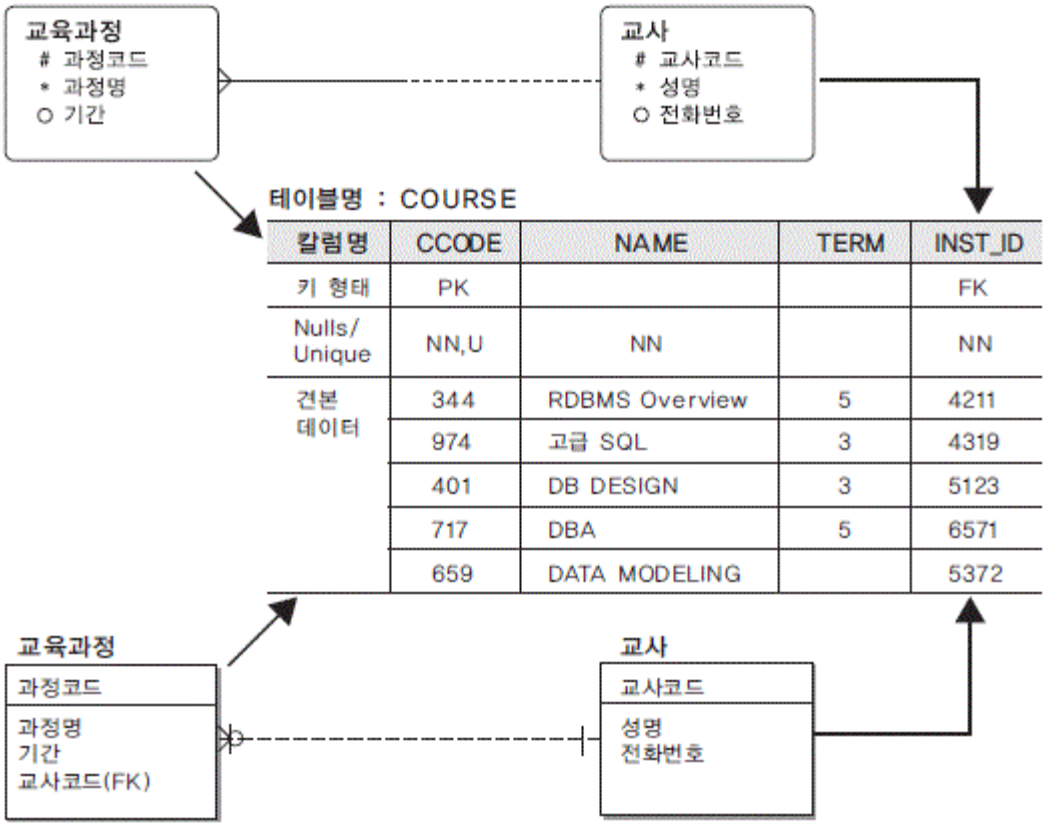
1:M 관계 변환

논리 데이터 모델에서 존재하는 관계 중에?? 따라 서 관계 칼럼의 선택사양이 결정되게 된다.

1) 변환 절차

- 1(One) 에 있는 PK 를 M(Many)의 FK 로 변환
  - FK 의 명칭 결정
  - 키 형태 란에 FK 표시
  - Nulls/Unique 란에 NN 표시(Must Be 관계시)
  - 필수 관계가 아닌 경우에는 NN 를 체크하지 않는다.
- 표본 데이터 추가
- UID BAR 가 있는 경우는 전단계에서 실시

2) 변환 예



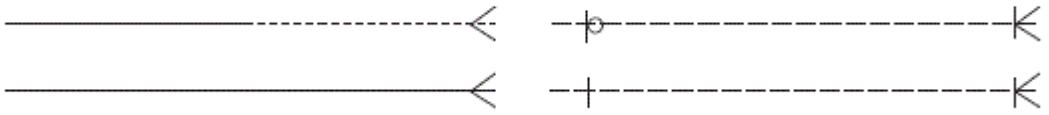
테이블명 : COURSE

칼럼명	CCODE	NAME	TERM	INST_ID
키 형태	PK			FK
Nulls/ Unique	NN,U	NN		NN
견본 데이터	344	RDBMS Overview	5	4211
	974	고급 SQL	3	4319
	401	DB DESIGN	3	5123
	717	DBA	5	6571
	659	DATA MODELING		5372

[그림 4-4-17] 바커 표기법(위)과 IE 표기법(아래)에 따른 1:M 관계 변환 예

3) 1:M 관계에서 1 쪽이 Mandatory 관계일 때의 변환시 주의사항

- 자식 쪽의 레코드(Row)가 반드시 하나 이상은 되어야만 부모 쪽의 레코드(Row)를 생성할 수 있다.
- 자식 쪽의 레코드(Row)를 삭제할 경우에는 전체를 다 삭제할 수는 없고 반드시 하나 이상의 자식 레코드(Row)를 남겨두어야 한다. 또는 자식, 부모 레코드(Row)를 동시에 삭제해야 한다.



[그림 4-4-18] 바커 표기법(좌)과 IE 표기법(우)에 따른 1:M Mandatory 관계

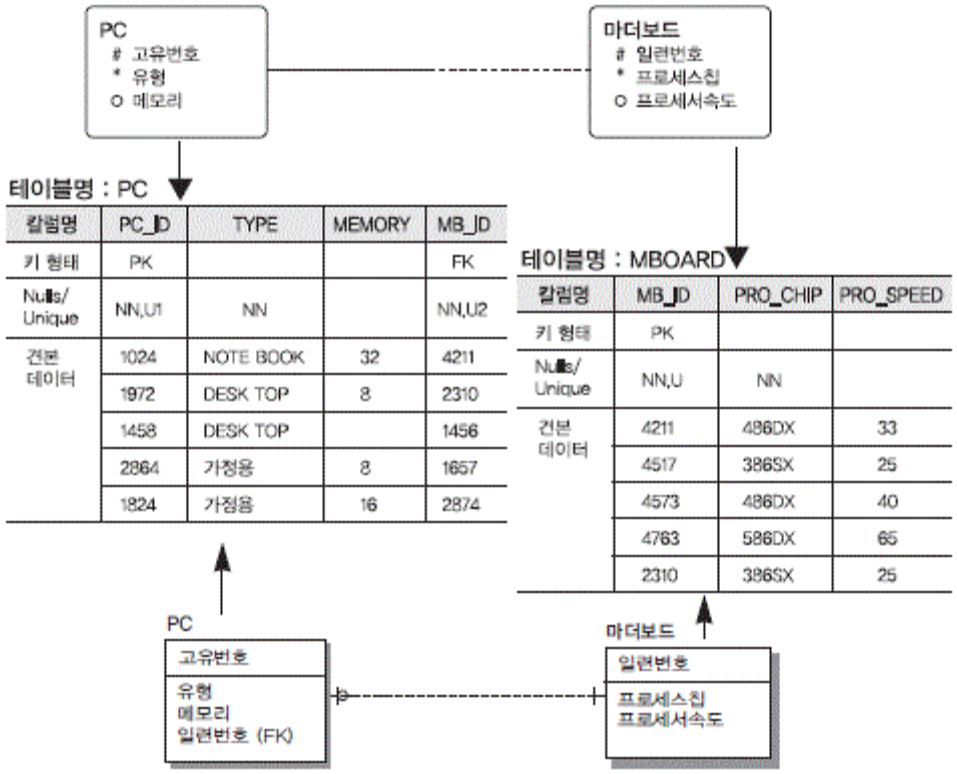
1:1 관계 변환

1:1 관계는 논리 모델에서는 자주 발생하지는 않는 관계이다. 이러한 1:1 관계를 물리 모델로 변환하는 과정은 관계의 Optionality(기수성)에 따라서 다른 방법으로 적용된다. 양쪽 다 Optional 인 경우에는 보다 빈번하게 사용되는 테이블이 외래키를 가지는 것이 유리하다.

1) 변환절차

- Mandatory 반대쪽에 있는 테이블의 기본키를 Mandatory 쪽 테이블의 외래키로 변환한다.
- NN 표시를 한다..

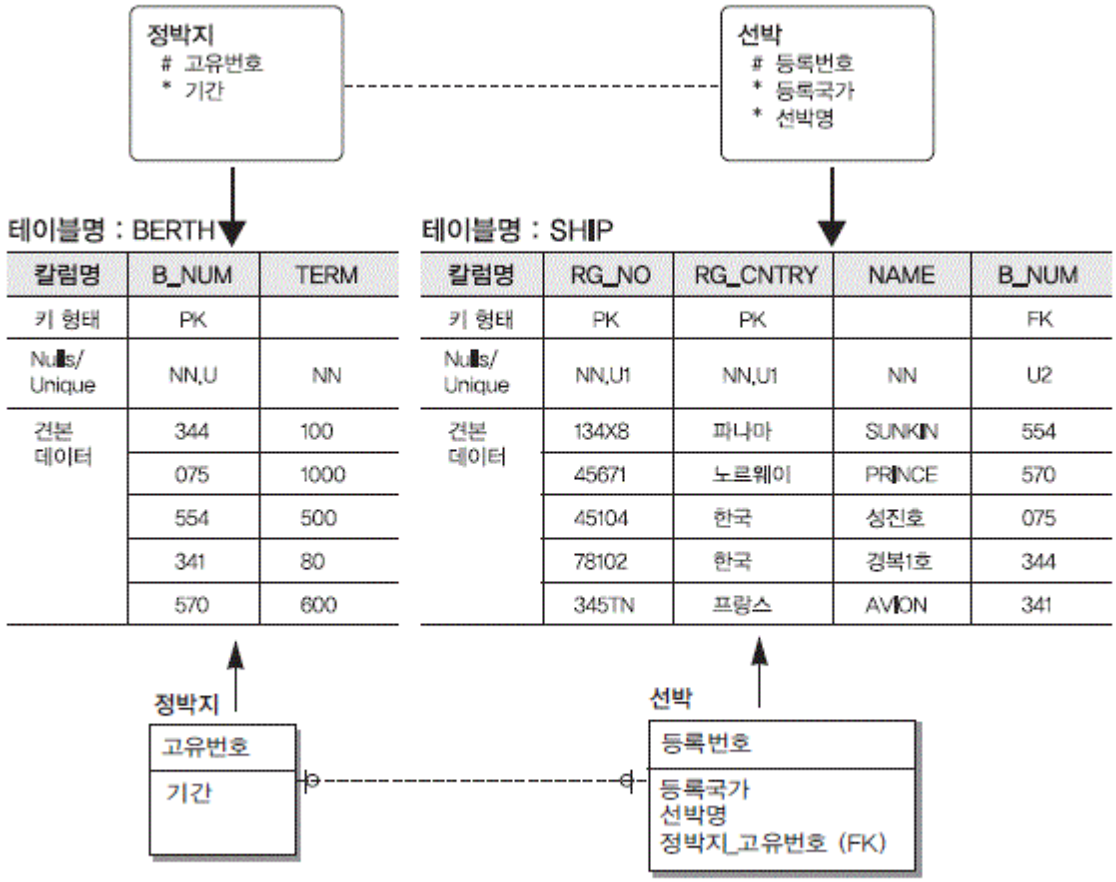
2) 변환 예



[그림 4-4-19] 바커 표기법(위)과 IE 표기법(아래)에 따른 1:1 Mandatory 관계 변환 예

3) 변환시 주의사항

- 1:1 관계에 의해서 생긴 모든 외래키 부분은 Unique Key 가 필수적이다.
- 한쪽이 Optional 이고 다른 한쪽이 Mandatory 라면 Mandatory 쪽의 테이블에 외래키가 생성된다.
- 양쪽다 Mandatory 라면 변환시에 어떤 테이블에 외래키를 생성할 것인지를 선택해야 한다.



칼럼명	B_NUM	TERM
키 형태	PK	
Nulls/ Unique	NN,U	NN
견본 데이터	344	100
	075	1000
	554	500
	341	80
	570	600

칼럼명	RG_NO	RG_CNTRY	NAME	B_NUM
키 형태	PK	PK		FK
Nulls/ Unique	NN,U1	NN,U1	NN	U2
견본 데이터	134X8	파나마	SUNKIN	554
	45671	노르웨이	PRINCE	570
	45104	한국	성진호	075
	78102	한국	경복1호	344
	345TN	프랑스	AVION	341

[그림 4-4-20] 바커 표기법(위)과 IE 표기법(아래)에 따른 1:1 Optional 관계 변환 예

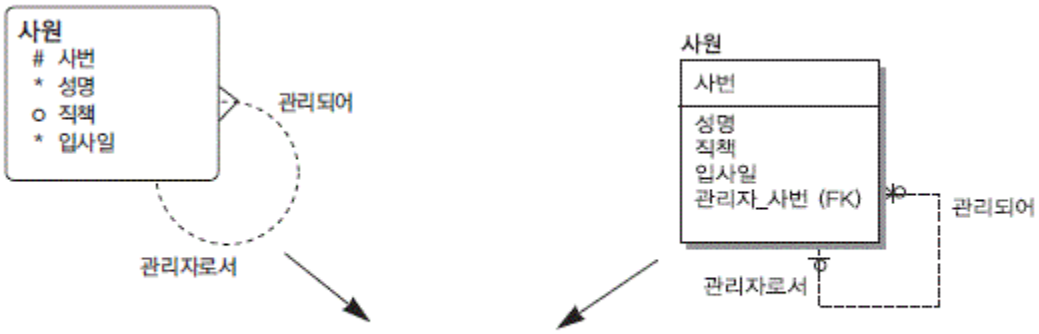
1:M 순환 관계 변환

대부분의 경우는 데이터의 계층 구조를 표현하기 위해서 이러한 관계를 사용한다. 특성상 최상위 관계 속성은 항상 Optional 인 형태의 관계이어야 한다. 하지만 경우에 따라서는 최상위의 관계 속성에 특정 값을 지정하는 경우도 존재한다.

1) 변환 절차

- 해당 테이블 내에 외래키 칼럼을 추가한다. 외래키는 같은 테이블 내의 다른 로우의 기본키 칼럼을 참조하게 된다.
- 외래키 칼럼 명칭은 가능한 한 관계 명칭을 반영한다. 외래키는 결코 NN(Not Null) 이 될 수 없다.

2) 변환 예



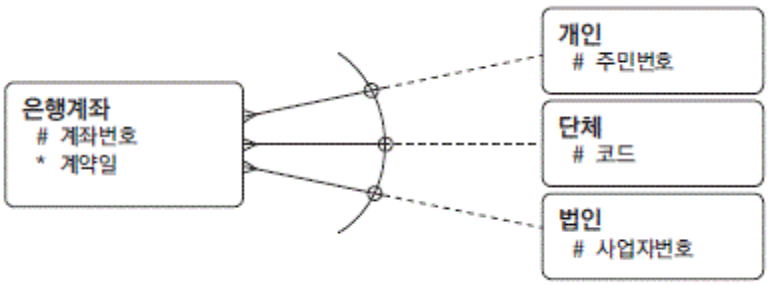
테이블명 : EMPLOYEE

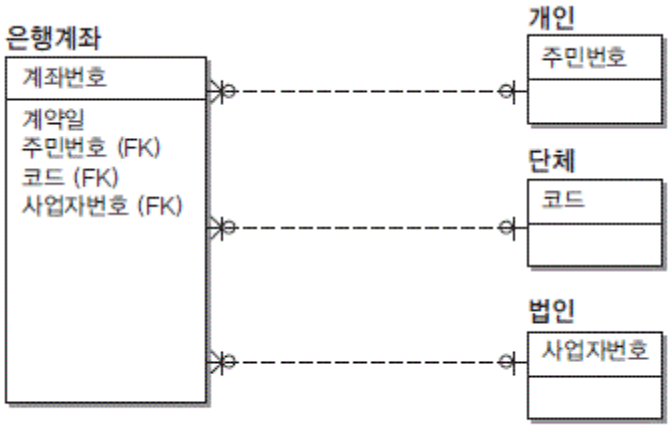
칼럼명	EMPNO	ENAME	JOB	HIREDATE	MRG_ID
키 형태	PK				FK
Nulls/ Unique	NN,U	NN		NN	NN,U
견본 데이터	7540	홍길동	대표이사	720312	
	5579	성삼문	전무이사	740801	7540
	6714	김이박	부장	820101	7863
	9451	오재구	사원	950301	3040
	3040	유성원	대리	910512	6573

[그림 4-4-21] 바커 표기법(좌)과 IE 표기법(우)에 따른 1:M 순환 관계 예

배타적 관계 변환

[그림 4-4-22]와 같은 논리 모델에서의 배타적 관계의 모델은 실제 데이터 환경에서는 자주 등장하게 된다. 하지만 이러한 관계를 물리 데이터 모델로 생성하는 방법은 일반적인 관계를 물리 데이터 모델로 변환하는 것과는 다르다. 여기에는 두 가지 정도의 대표적인 방법을 가지고 설명한다.





[그림 4-4-22] 바커 표기법(위)과 IE 표기법(아래)에 따른 배타적 관계 : 논리 모델 예

1) 외래키 (Foreign Key) 분리 방법

각각의 관계를 관계 칼럼으로 생성하는 방법이다. 이 방법을 사용하면 실제 외래키 제약조건 (Foreign Key Constraints)을 생성할 수 있다는 장점이 있다. 하지만 각각의 키 칼럼들이 Optional 이어야 한다. 또한 다음과 같은 체크 제약조건(Check Constraints)을 추가적으로 생성하 여야 한다.

추가적인 체크 제약조건

```
CHECK ( JUMIN_NO IS NOT NULL
AND PART_NO IS NULL
AND BUSINESS_ID IS NULL )
OR ( JUMIN_NO IS NULL
AND PART_NO IS NOT NULL
AND BUSINESS_ID IS NULL )
OR ( JUMIN_NO IS NULL
AND PART_NO IS NULL
AND BUSINESS_ID IS NOT NULL )
```

2) 외래키 결합 방법

각각의 관계를 하나의 관계 칼럼으로 생성하는 방법이다. 이 방법을 사용하면 실제 외래키 제약조 건을 생성할 수 없다는 단점이 있다. 또한 각각의 관계를 선택적으로 구분할 수 있는 추가적인 칼럼이 필요하게 된다.

칼럼명	ACCTNO	CDATE	JUMIN_NO	TYPE
키형태	PK		FK	
Nulls/ Unique	NN,U	NN	NN	NN
견본 데이터	7540	940614	581101-101211	J
	5579	931201	298-02-11	P
	6714	940516	711024-281071	J
	9451	930718	10-234-1955	B
	3040	921009	211-05-29	P

[그림 4-4-24] 배타적 관계 변환 : 외래키 결합 예

구분 칼럼 추가

[그림 4-4-24]에서와 같이 구분 칼럼 TYPE 이 추가되어 배타적 관계의 각 테이블들이 구분된다.

- -J : 개인
- -P : 단체
- -B : 법인

관리상 필요한 칼럼 추가

개념

논리 데이터 모델에는 존재하지 않지만 관리상의 이유로 혹은 데이터베이스를 이용하는 프로그래 밍이 좀더 빠르게 수행되도록 하기 위해서 테이블이나 칼럼을 추가할 수 있다. 예를 들면 해당 데이 터를 등록한 일자나 시스템 번호 등과 같은 관리상의 이유로 필요한 것들이다.

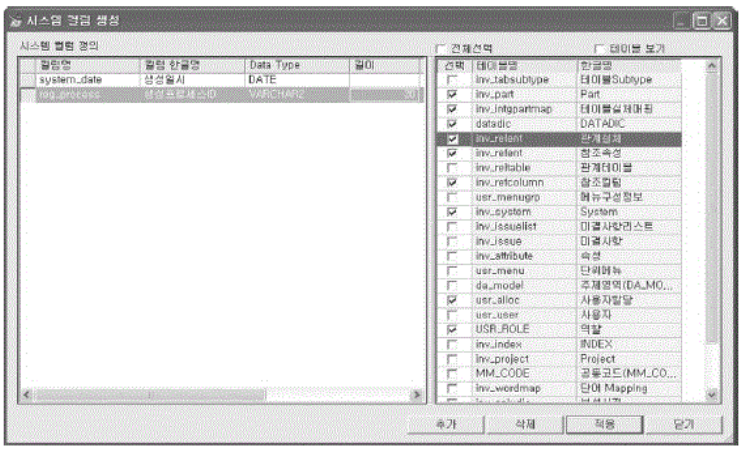
시스템 칼럼 추가 예

[그림 4-4-25]는 생성일시, 생성 프로세스 ID 라는 논리 데이터 모델에서는 존재하지 않는 속성인 데도 불구하고 물리 데이터 모델에서 추가하여 생성하고 있는 예를 보여주고 있다.

데이터 타입 선택

개념

물리 데이터 모델링 단계에서 일어나는 많은 문제 중의 하나가 칼럼의 데이터 형식을 잘못 설정하 는 데에서 발생한다. 특정 칼럼의 데이터 형식을 선택하는 것은 논리 데이터 모델에서 정의된 논리적인 데이터 타입(정보 타입 : Information Type)을 물리적인 DBMS 의 특성과 성능 등을 고려하여 최적의 데이터 타입을 선택하는 작업이다. 여기에서의 DBMS 별 특성은 이 책을 통해 설명하는 것은 불가능하다. 따라서 여기에서는 대표적인 데이터 타입의 선택 기준에 대해서만 언급한다. 여기에서 는 데이터 타입의 선택에 대한 예제를 들어 설명한다. 단, 모든 DBMS 를 예로 들 수는 없기 때문에 MS SQL Server 를 예를 들어 설명한다.



[그림 4-4-25] 시스템 칼럼 추가 예

문자 타입 (Character Data Types)

논리 데이터 모델에서의 형식이 문자였다면 세부적으로 많은 문자 형식들 중에서 칼럼의 값이 어 떤 범주를 만족하는지를 판단해야 한다. 여기에는 현재 칼럼이 가지는 값의 특성도 고려되어야 하고, 미래에 가질 값의 특성들도 고려되어야만 한다.

1) 세부 문자 타입 선택을 위한 기준

가) 영문만 사용되는가?

유니코드 형식은 모든 문자를 2 바이트(Byte) 체계로 설계한 국제 표준 문자 형식이다. 한글을 저장하기 위해서 반드시 유니코드 형식을 사용해야 하는 것은 아니다. 하지만 일반 문자 형식들에 값을 지정한다면 영문은 1 바이트, 한글은 2 바이트라는 부조화를 일으키게 된다. 이러한 이유로 유 니코드를 저장하는 데이터 타입에는 일반적으로 NCHAR, NVARCHAR 등과 같이 앞에 N 이 들어 가는 데이터 타입을 사용한다.

나) 4000 자 혹은 8000 자 이상의 문자열이 포함되는가?

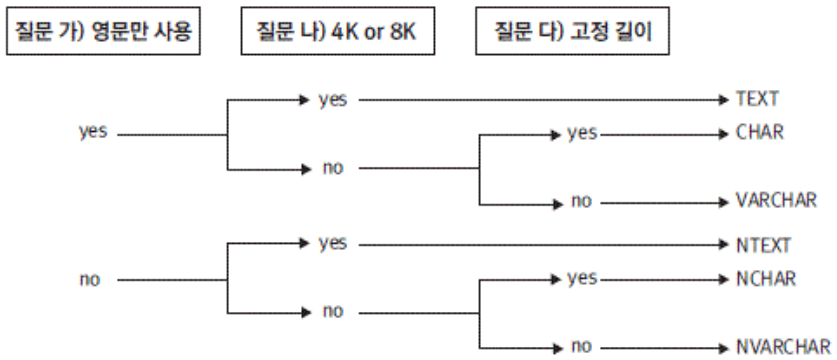
일반적인 문자들을 저장하는 데이터 타입은 4K 혹은 8K 를 상한선으로 하고 있다. 물론 이 기준은 DBMS 마다 조금씩 다르다. 이렇게 큰 데이터를 저장하기 위해서는 일반적인 문자열을 저장하 는 데이터 타입이 아닌 다른 데이터 타입을 사용해야 한다. 예를 들면 , LONG, TEXT 등과 같은 데이터 타입을 사용할 수 있다.

다) 입력되는 값의 길이가 일정한가?

값의 길이가 일정하다는 것은 입력되는 칼럼 값의 길이가 일정하다는 것을 의미한다. 반대로 가 변적이라는 것은 입력되는 값의 길이가 일정하지 않다는 것을 의미한다.

2) 문자 형식 데이터 타입 설정 예

[[그림 4-4-26]은 MS SQL Server 에서의 데이터 타입을 기준으로 예를 든 것이다.



[그림 4-4-26] 문자 타입 지정 예

숫자 타입 (Numeric Types)

숫자 타입의 데이터 타입도 DBMS 마다 많은 형식이 존재한다. 많은 숫자 타입들 중에서 주어진 상황에 맞는 가장 적절한 데이터 타입을 설정해야 한다.

1) 세부 숫자 타입 선택을 위한 기준

가) 정말 숫자 데이터인지 판단한다.

많은 경우 숫자처럼 보이는 숫자가 아닌 값들을 관리하는 경우가 존재한다. 예를 들면 6810301633318 과 같은 주민등록번호는 숫자처럼 보이지만 숫자가 아니다. 즉, 우리가 이 주민 번호를 가지고 연산을 하거나 할 가능성이 있는지를 보면 이것은 숫자 타입이 아닌 문자 타입의 데 이터라는 것을 알 수 있다.

나) 세부 숫자 타입 결정

불린(Boolean)

참(TRUE) 혹은 거짓(False)을 저장하는 경우에 선택한다.

정수(Integer)

소수점 이하를 처리하지 않는 경우에 선택한다.

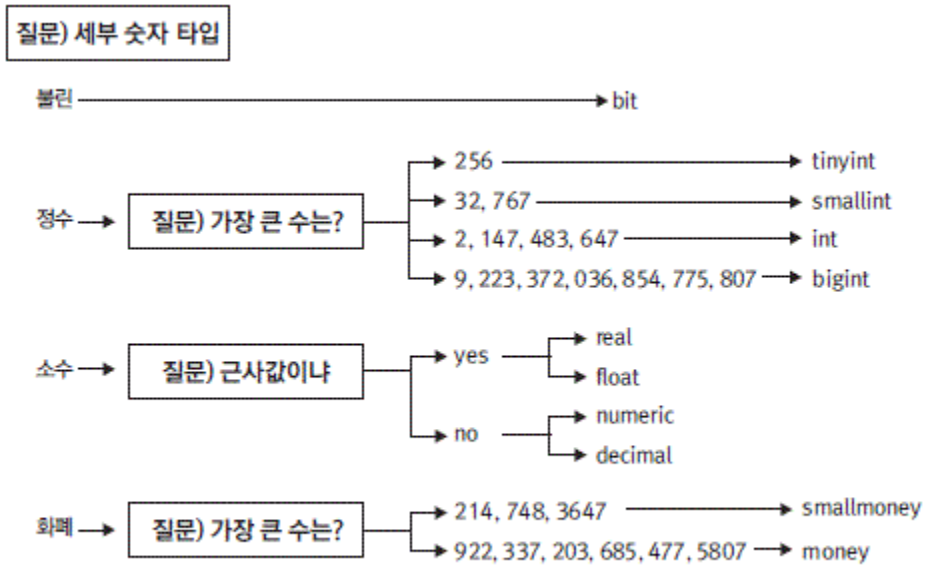
소수(Decimal)

소수점 이하를 처리하는 경우에 선택한다.

화폐(Money)

금액을 저장하기 위한 경우에 선택한다.

2) 숫자 형식 데이터 타입 지정 예



[그림 4-4-27] 숫자 타입 지정 예

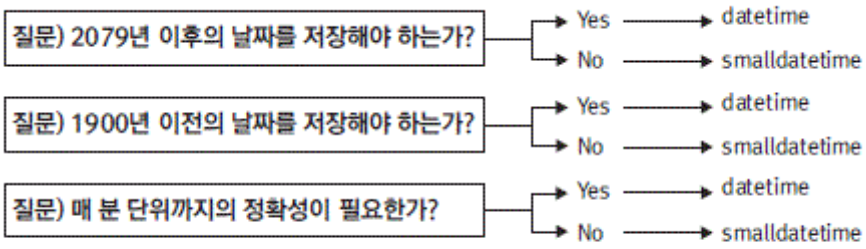
날짜 타입 (Datetime Types)

특정 데이터 항목에 대해서 날짜 타입으로 할 것인지 아니면 문자 타입으로 할 것인지는 이미 논리 데이터 모델에서 결정된다. 그렇기 때문에 물리 데이터 모델링에서는 논리 데이터 모델링에서 날짜 타입으로 결정된 부분을 DBMS 특성에 맞게 여러 개의 날짜 타입들 중에서 어떤 날짜 타입을 선택 할 것인지를 결정하는 것이다.

1) 세부 날짜 타입 선택을 위한 기준

대부분의 DBMS 에서는 날짜 타입에 일자뿐만 아니라 시분초의 정보도 같이 저장한다. 심지어는 0.001 초 차이까지도 저장하기도 한다. 그래서 그냥 일반적인 시간까지를 저장할 것이냐 아니면 이러 한 정밀한 시간을 저장할 것이냐에 따라 날짜 타입을 결정한다.

2) 세부 날짜 타입 지정 예



[그림 4-4-28] 날짜 타입 선택 예

데이터 표준 적용

개념

논리 데이터 모델링 과정에서 정의된 엔터티, 속성, 관계들을 여러가지 기준으로 물리 데이터 모델 로 변환한다. 이 과정에서 필수적으로 엔터티명에 해당하는 테이블명을 생성하고, 속성 또는 관계에 해당하는 칼럼명을 생성하게 된다. 이러한 이름을 변환하는 과정에서 전사적으로 미리 생성된 데이 터 표준을 따르게 된다. 이러한 데이터 표준에는 대표적으로 표준 용어, 표준 도메인, 표준 명명 규 칙 등이 존재한다.

데이터 표준 적용 대상

물리 데이터 모델에서의 데이터 표준화는 다음과 같은 객체를 대상으로 수행하게 된다.

1) 데이터베이스(Database)

테이블의 집합으로 통합 모델링 단계의 주제 영역이나 애플리케이션 모델링 단계의 업무 영역에 대응되는 오브젝트이다.

2) 스토리지 그룹(Storage Group)

DASD(Direct Access Storage Device) 즉, 물리적인 디스크(Disk)를 묶어서 하나의 그룹으로 정의해 놓은 것이며 테이블스페이스, 인덱스 스페이스 생성시 스토리지 그룹명을 지정하여 물리적인 영역을 할당하도록 한다.

3) 테이블스페이스(Tablespace)

테이블이 생성되는 물리적인 영역이며, 하나의 테이블 스페이스에 하나 또는 그 이상의 테이블을 저장할 수 있다.

4) 테이블(Table)

논리 설계 단계의 엔터티에 대응하는 객체이다.

5)칼럼(Column)

논리 설계 단계의 속성에 대응하는 객체이다.

6) 인덱스(Index)

테이블에서 특정 조건의 데이터를 효율적으로 검색하기 위한 색인 데이터로 대표적인 인덱스 대상 으로는 기본키(Primary Key), 외래키(Foreign Key)등이 있다.

7) 뷰(View)

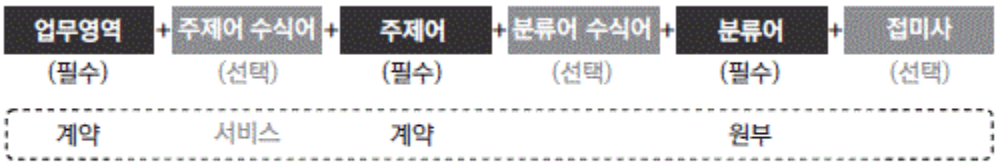
테이블에 대한 재정의로서 물리적으로 테이블의 특정 칼럼, 특정 로우를 뷰로 정의하여 특정 사용 자만 접근이 가능하도록 할 수 있다.

데이터 표준 적용 방법

1) 명명 규칙에 의한 표준화 적용

테이블에 대한 명명 규칙과 적용 기준들의 예를 들면 다음과 같다.

- 논리 데이터 모델을 물리 데이터 모델로 전환시 테이블명은 엔터티 한글명과 동일하게 용어를 사 용하면서 해당 용어를 영문명으로 전환한다.
- 영문명은 영문 약어를 사용하며, 표준 용어 사전에 등록된 표준 영문 약어를 참조한다.
- 테이블의 명명 순서는 업무 영역 + 주제어 수식어 + 주제어 + 분류어 수식어 + 분류어 + 접미사 순으로 한다.



[그림 4-4-29] 테이블 명명 규칙 예

- 테이블 영문명에서는 각 어소별 구분자로 Under Bar(_)를 사용한다.
  예) 업무관계자_사원_정보 IVPT_EMP_INFO

2) 표준 용어집에 의한 표준화 적용

사전에 사용될 모든 객체명과 해당 객체에 대한 데이터 타입, 길이 등의 표준을 정의해 놓고 이 표준 들을 적용하는 방식이다. 현재 대부분의 회사들은 위 두 가지의 방법을 병행하여 사용하고 있다고 볼 수 있다.

Part 2. 응용 SQL 작성하기

Chapter 5. JDBC의 이해

1 JDBC (Java Database Connectivity)

자바에서는 JDBC라는 강력한 도구를 제공해 자바 프로그램에서 데이터베이스를 처리할 수 있게 해준다. JDBC는 일관성 있는 방식으로 데이터베이스에 접근할 수 있는 API를 제공하는 클래스들이라고 볼 수 있다.

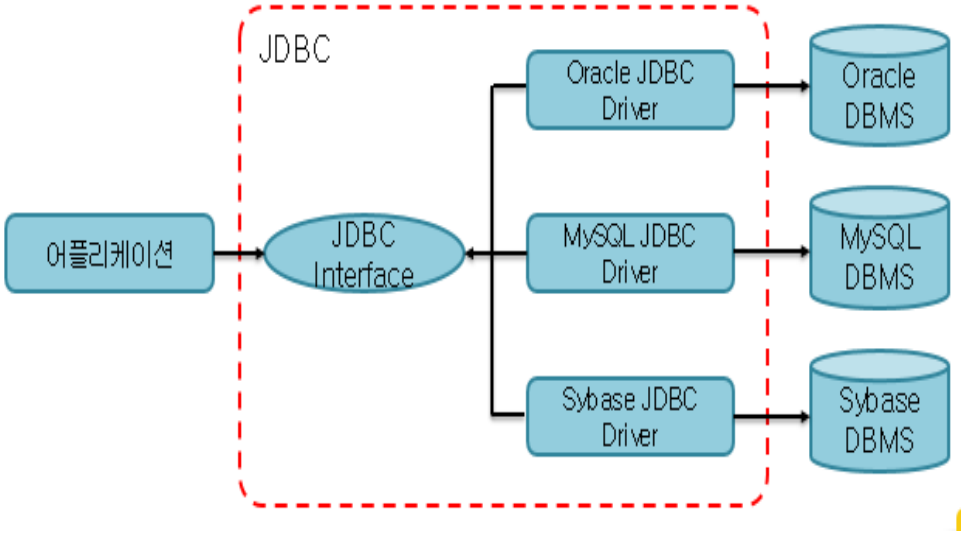
데이터를의 집합이라고 볼 수 있는 데이터베이스는 데이터베이스 관리 시스템(DBMS, Database Management System)에 의해 관리된다. DMBS에는 오라클, MSSQL Server, MySql 등이 있지만 자바 웹 프로젝트에서는 오라클 DMBS를 주로 사용하게 된다.

자바와 데이터베이스는 별도의 시스템으로 동작하기 때문에 자바에서 제공되는 순수 API에는 데이터베이스에 접근할 수 있는 기능을 가진 클래스가 제공되지 않는다. 자바 프로그램에서 데이터베이스 작업이 필요할 때 사용할 수 있도록 제공되는 것이 JDBC이다.

> 자바 언어에서 Database 에 접근할 수 있게 해 주는 Programming API 이다.

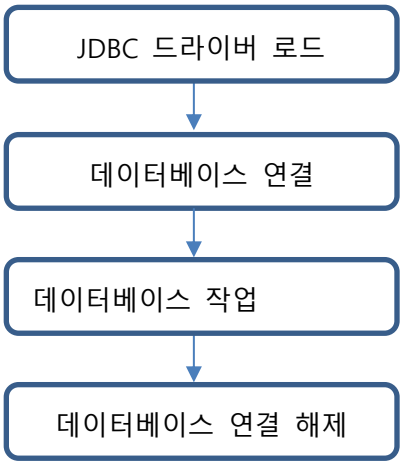
Java에서 DBMS연동에 필요한 메소드를 java.sql 패키지에서 제공하고 있다.

각 데이터베이스와 자바를 연결하기 위한 Jdbc driver 는 각 dbms 밴더별로 구현하여 jar 파일 형태로 제공하고 있다.



2 JDBC 동작

JDBC를 이용한 데이터베이스와 연결에는 4단계를 거쳐 동작하게 된다.



Chapter 6. JDBC 구현 순서

1 JDBC 드라이버 로드

자바에서 데이터베이스 연결을 위한 JDBC 드라이버는 데이터베이스 제작 업체에서 제공한다. 오라클이 설치된 폴더에서 JDBC 드라이버를 얻어와 자바 개발 환경에 설치해야만 드라이버를 사용할 수 있다.

JDBC 드라이버의 이름은 ojdbcX.jar 형식이며 버전별로 X가 달라질 수 있다.

- 오라클 11g 익스프레스 에디션을 설치할 경우 경로 예시

```
C:₩oraclexe₩app₩oracle₩product₩11.2.0₩server₩jdbc₩lib
```

JDBC 드라이버 파일(.jar)을 복사하여 Dynamic Web Project의 WEB-INF 폴더 내의 lib 폴더에 붙여넣기하면 드라이버가 설치된다.

우선 JDBC의 기능을 사용할 수 있도록 JDBC 클래스를 import 해 주어야 한다. JDBC 클래스는 java.sql 패키지에 존재한다.

- java.sql 패키지 import

```
import java.sql.*;
```

이후 JDBC 클래스를 로드한다.

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

java.lang.Class 클래스의 forName 메소드는 문자열 형태로 전달된 클래스를 JVM으로 로드해주는 기능을 가지고 있다. Oracle JDBC 드라이버를 지정해 로드하여 DriverManager에 등록이 되도록 하는 코드이다.

2 데이터베이스 연결

JDBC 드라이버를 로딩한 후 데이터 베이스에 연결하기 위해 Connection 객체를 사용할 수 있다. Connection 객체는 데이터베이스에 연결해 작업을 수행할 수 있도록 도와주는 객체이다. DriverManager 클래스의 getConnection( ) 메소드를 호출하여 사용할 수 있다.

- Connection 객체 얻기

```
Connection conn = DriverManager.getConnection(url, uid, pwd);
```

url : JDBC 형식 URL

uid : 사용자 ID

pwd : 패스워드

Connection 객체를 얻기 위해 URL, UID, PWD를 지정해야 하는데 URL은 JDBC 형식의 URL이고 UID와 PWD는 사용하는 DBMS계정인 사용자명과 패스워드를 뜻한다.

- JDBC 형식으로 오라클 DMBS를 지정하는 형식 예

```
jdbc:oracle:thin:[hostname][:port]:dbname
```

익스프레스 에디션을 설치하였다면 dbname은 XE 이다.

- 오라클 익스프레스 에디션의 scott 계정에 Connection 객체 얻기

```
Connection conn = DriverManager.getConnection(
  "jdbc:oracle:thin:@localhost:1521:XE",
  "scott",
  "tiger");
```

이후 데이터베이스 작업을 진행할 수 있다.

데이터베이스 작업은 conn 객체를 통해 데이터베이스 작업을 할 수 있으며 모든 데이터 베이스 작업이 끝나면 연결을 종료시켜주면 된다.

- 데이터 베이스 연결 끊기

```
conn.close( );
```

3 데이터베이스 작업

데이터베이스 연결을 한 후 실제 SQL문을 수행하기 위해서 Statement 객체를 생성한다.

데이터베이스 연결에 사용한 conn 객체를 통해 createStatement( ) 메소드를 호출해 Statement 객체를 얻어올 수 있다

- Statement 객체 얻기

```
Statement stmt = conn.createStatement( );
```

stmt : 표준 SQL문을 수행하기 위한 Statement 객체

데이터베이스 작업이 완료되면 Statement 객체를 반환할 필요가 있어 close( )메소드를 호출해야 한다.

- Statement 객체 닫기

```
stmt.close( );
```

Statement 객체를 사용해 SQL 표준 쿼리문을 자바 프로그램에서 사용할 수 있다. 쿼리문을 실행시키는 메소드는 두가지가 있다. executeQuery메소드의 결과는 ResultSet으로 받게 된다.

메소드	설명
executeQuery	select 문과 같이 결과값이 여러 개의 레코드로 구해지는 경우 사용
executeUpdate	insert, update, delete 문과 같이 테이블 내에 데이터 변경 후 반환하는 결과값이 행의 개수인 경우 사용

- Select 구문 예시

```
String sql = "Select * from emp";
ResultSet rs = stmt.executeQuery(sql);
```

executeQuery 메소드의 리턴인 ResultSet 객체는 여러 개의 행으로 결과값을 받을 수 있으며 각 행에 접근할 수 있도록 Iterator와 비슷한 메소드를 제공한다.

- ResultSet의 대표 메소드

메소드	설명
next( )	현재 행에서 한 행 앞으로 이동
previouse( )	현재 행에서 한 행 뒤로 이동
first( )	첫 번째 행으로 이동
last( )	마지막 행으로 이동

Chapter 7. JDBC CRUD

1 Database 연동하기

- 예제 테이블 스크립트

```
DROP TABLE TEST_MEMBER CASCADE CONSTRAINTS;

CREATE TABLE TEST_MEMBER(
    ID      VARCHAR2(15 BYTE),
    PASSWD      VARCHAR2(15 BYTE) NOT NULL,
    NAME      VARCHAR2(20 BYTE) NOT NULL,
    EMAIL      VARCHAR2(30 BYTE),
    CONSTRAINT PK_MEMBER PRIMARY KEY (ID)
);

INSERT ALL
INTO TEST_MEMBER VALUES('admin','admin', '관리자','admin@test.or.kr')
INTO TEST_MEMBER VALUES('user11','pass11','사용자1','user11@test.or.kr')
INTO TEST_MEMBER VALUES('user22','pass22','사용자2','user22@test.or.kr')
SELECT * FROM DUAL;

COMMIT;
```

- 예제 CSS

myWeb/web/css/style.css

```
01  @charset "UTF-8";
02  body {font-size : 14pt; color   : black; line-height   : 160%; }
03  table, tr, td{
04    border-width:1px; border-style: solid; border-color: #113a08; background-color:#a8ffc0;
05  }
06  td { font-size : 14pt; color : black; line-height : 160%; }
```

- Web 실습 예제

first.common.JDBCTemplate.java

```
01  package first.common;
02
03  import java.io.BufferedReader;
04  import java.io.FileReader;
05  import java.sql.Connection;
06  import java.sql.DriverManager;
07  import java.sql.ResultSet;
08  import java.sql.SQLException;
09  import java.sql.Statement;
10  import java.util.Properties;
11
12  public class JDBCTemplate {
13    public JDBCTemplate() {}
14
15  public static Connection getConnection() {
16      Connection con = null;
17      Properties prop = new Properties();
18      try {
19        String currentPath = JDBCTemplate.class.getResource("./").getPath();
20        prop.load(new BufferedReader(new FileReader(currentPath+"driver.properties")));
21        Class.forName(prop.getProperty("driver"));
22        con = DriverManager.getConnection(prop.getProperty("url"), prop.getProperty("user"),
23            prop.getProperty("pwd"));
24      } catch (Exception e) {
25        e.printStackTrace();
26      }
27      return con;
28    }
29
30    public static void close(Connection con) {
31      try {
32        if (con != null && !con.isClosed()) con.close();
```

```
33      } catch (SQLException e) {
34          e.printStackTrace();
35      }
36    }
37    public static void close(Statement stmt) {
38      try {
39        if (stmt != null && !stmt.isClosed())   stmt.close();
40      } catch (SQLException e) {
41        e.printStackTrace();
42      }
43    }
44    public static void close(ResultSet rset) {
45      try {
46        if (rset != null && !rset.isClosed())   rset.close();
47      } catch (SQLException e) {
48        e.printStackTrace();
49      }
50    }
51    public static void commit(Connection con) {
52      try {    // DB 에서 일어나는 트랜잭션 커밋 처리
53        if (con != null && !con.isClosed())   con.commit();
54      } catch (SQLException e) {
55        e.printStackTrace();
56      }
57    }
58
59    public static void rollback(Connection con) {
60      try {    // DB 에서 일어나는 트랜잭션 롤백 처리
61        if (con != null && !con.isClosed())   con.rollback();
62      } catch (SQLException e) {
63        e.printStackTrace();
64      }
65    }
```

```
66  }
```

myWeb/src/first/common/driver.properties

```
01  # Driver.properties
02  driver=oracle.jdbc.driver.OracleDriver
03  url=jdbc:oracle:thin:@localhost:1521:xe
04  user=student
05  pwd=student
```

first.member.model.vo.Member.java

```
01  package first.member.model.vo;
02
03  public class Member implements java.io.Serializable {
04    private static final long serialVersionUID = 1L;
05    private String id;
06    private String passwd;
07    private String name;
08    private String email;
09
10    public Member() {}
11    public Member(String id, String passwd, String name, String email) {
12      this.id = id;
13      this.passwd = passwd;
14      this.name = name;
15      this.email = email;
16    }
17
18    @Override
19    public String toString() {
20      return "Member [id=" + id + ", passwd=" + passwd + ", name=" + name + ", email=" +
21  email + "]";
22    }
23  /* ... Getter & setter 구현 부 */
24  }
```

first.member.model.dao.MemberDao.java

```
01  package first.member.model.dao;
02
03  import java.sql.*;
04  import java.util.ArrayList;
05  import first.member.model.vo.Member;
06  import static first.common.JDBCTemplate.*;
07
08  public class MemberDao {
09
10    public MemberDao() {}
11
12    public ArrayList<Member> selectAll(Connection con) {
13      ArrayList<Member> list = new ArrayList<Member>();
14      Statement stmt = null;
15      ResultSet rset = null;
16
17      String query = "SELECT * FROM TEST_MEMBER";
18      try {
19        stmt = con.createStatement();
20        rset = stmt.executeQuery(query);
21         // 조회한 데이터를 ArrayList<Member>에 하나씩 추가
22        if(rset != null) {
23          while(rset.next()) {
24            list.add(new Member(rset.getString("ID"),rset.getString("PASSWD"),
25                rset.getString("NAME"),rset.getString("EMAIL")));
26          }
27        }
28      } catch (SQLException e) {
29        System.out.println(e.getMessage());
30      } finally {        // 사용한 ResultSet과 Statement를 반환한다.
31        close(rset);
32        close(stmt);
33      }
```

```
34      return list;
35    }
36  }
```

first.member.model.service.MemberService.java

```
01  package first.member.model.service;
02
03  import static first.common.JDBCTemplate.*;
04  import java.sql.Connection;
05  import java.util.ArrayList;
06  import first.member.model.dao.MemberDao;
07  import first.member.model.vo.Member;
08
09  public class MemberService {
10
11    public MemberService() {}
12    public ArrayList<Member> selectAll() {
13      Connection con = getConnection();
14      ArrayList<Member> list = new MemberDao().selectAll(con);
15      close(con);
16      return list;
17    }
18  }
```

first.member.controller.MemberSelectServlet.jsp

```
01  package first.member.controller;
02
03  import java.io.IOException;
04  import java.util.ArrayList;
05  import javax.servlet.*;
06  import javax.servlet.annotation.WebServlet;
07  import javax.servlet.http.*;
08  import first.member.model.service.MemberService;
09  import first.member.model.vo.Member;
10
```

```
11  @WebServlet("/mselect.do")
12  public class MemberSelectServlet extends HttpServlet {
13    private static final long serialVersionUID = 1L;
14
15      public MemberSelectServlet() { }
16
17    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
18  ServletException, IOException {
19      RequestDispatcher view = null;
20      ArrayList<Member> mlist = new MemberService().selectAll();
21      if(mlist != null && mlist.size() > 0) {
22        view = request.getRequestDispatcher("view/member/memberView.jsp");
23  // RequestDispatcher를 통해 서블릿의 request와 response를 대상 jsp에 연결한다.
24        request.setAttribute("mlist", mlist);
25        view.forward(request, response);
26      } else {
27        view = request.getRequestDispatcher("view/error/errorPage.jsp");
28        request.setAttribute("msg", "유저 정보 조회 실패!!");
29        view.forward(request, response);
30      }
31    }
32    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
33  ServletException, IOException {
34      doGet(request, response);
35    }
36  }
```

myWeb/web/view/member/memberView.jsp

```
01  <%@ page language="java" contentType="text/html; charset=UTF-8"
02    pageEncoding="UTF-8" import="java.util.*, first.member.model.vo.Member"%>
03  <%
04    ArrayList<Member> mlist = (ArrayList<Member>)request.getAttribute("mlist");
05  %>
06  <!DOCTYPE html>
```

```
07  <html>
08  <head>
09  <meta charset="UTF-8">
10  <title>MVC2 방식의 데이터베이스 연동</title>
11  <link href="/test/css/style.css" rel="stylesheet" type = "text/css">
12  </head>
13  <body>
14  <h2>MVC2 방식의 데이터베이스 연동</h2>
15  <h3>회원 정보 조회</h3>
16    <table>
17      <tr>
18        <td><strong>ID</strong></td>
19        <td><strong> PASSWD </strong></td>
20        <td><strong>NAME</strong></td>
21        <td><strong>E_MAIL</strong></td>
22      </tr>
23      <% for(Member m : mlist){ %>
24        <tr>
25          <td><%=m.getId() %></td>
26          <td><%=m.getPasswd() %></td>
27          <td><%=m.getName() %></td>
28          <td><%=m.getEmail() %></td>
29        </tr>
30      <% } %>
31    </table>
32    <br>
33    total records : <%= mlist.size() %>
34  </body>
35  </html>
```

Chapter 8. JDBC Statement와 PreparedStatement

1. DriverManager

데이터 원본에 JDBC 드라이버를 통하여 커넥션은 만드는 역할을 한다. Class.forName() 메소드를 통해 생성되며, 반드시 예외처리를 해야 한다.

직접 객체 생성이 불가능하고, getConnection()메소드를 사용하여 객체를 생성할 수 있다

2. Connection

Connection 객체는 특정 데이터 원본과 연결된 커넥션을 나타낸다.

Statement객체를 생성할 때도 Connection객체를 사용하여 createStatement()메소드를 호출하여 생성한다.

SQL문장을 실행시키기 전에 우선 Connection객체가 있어야 한다.

3. Statement

Connection 객체에 의해 프로그램에 리턴되는 객체에 의해 구현되는 일종의 메소드 집합을 정의한다. Connection 클래스의 createStatement()메소드를 호출하여 얻어지며, 생성된 Statement 객체로 질의문장을 String 객체에 담아 인자로 전달하여 excuteQuery()메소드를 호출하여 SQL 질의를 수행한다

```
예)  try{
            String query = "SELECT ID, LAST_NAME FROM EMP"
            stmt = conn.createStatement();
            rset = stmt.excuteQuery(query);

     }catch(SQLException e){
            e.printStackTrace();
     }
```

4. PreparedStatement

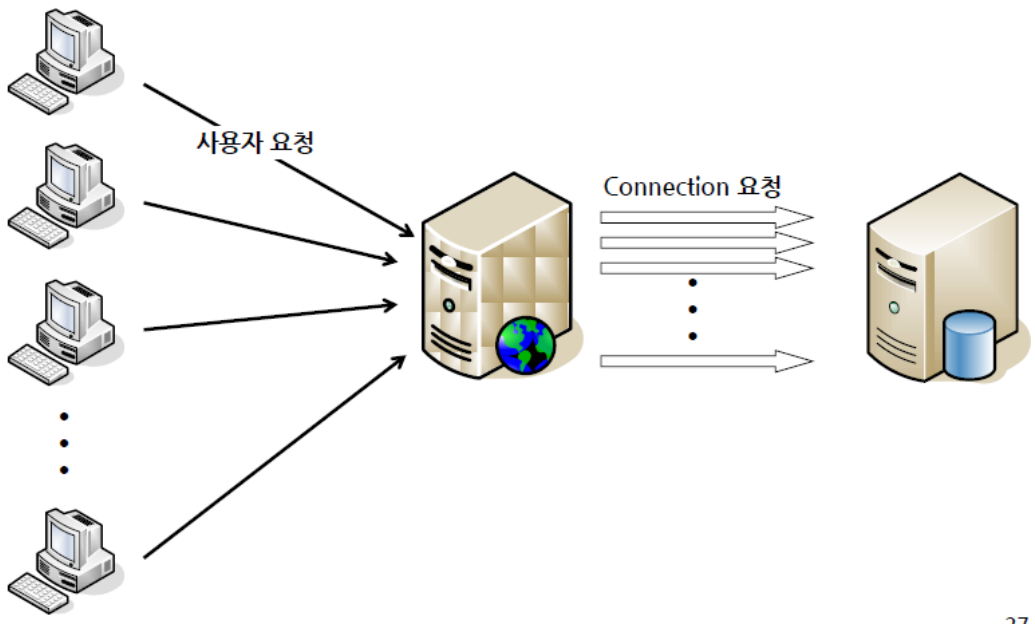
Connection 객체의 preparedStatement()메소드를 사용하여 객체를 생성한다. SQL문장이 미리 컴파일되고, 실행시간동안 인수 값을 위한 공간을 확보할 수 있다는 점에서 Statement와 다르다.

각각의 인수에 대해 위치홀더(?)를 사용하여 SQL문장을 정의할 수 있게 해준다.

예)
```
try{
        String query = "insert into member value (?, ?);
        pstmt = conn.preparedStatement(query);
        pstmt.setString(1, id);
        pstmt.setString(2, password);
    }catch(SQLException e){
        e.printStackTrace();
    }
```

Chapter 9. JDBC Connection Pool

만약 동시에 100 명이 동일한 서비스를 요청해서 웹서버에서 데이터베이스에 100 번의 접속 요청을 동시에 해야 한다면?



1. DB Connection Pooling

Pooling 기법이란,

미리 데이터베이스 Connection을 여러 개 만들어서 특정 공간에 저장해 놓고, 여러 사용자가 필요할 때마다 하나씩 꺼내서 사용하고 다시 집어 넣는 방식을 말한다.

1. Connection을 미리 생성해서 보관
2. Connection에 대한 요청이 들어오면, 보관 중인 Connection 중 하나를 넘겨줌
3. 사용이 끝난 Connection을 다시 보관

Connection Pooling의 장점

1. 속도 향상
2. 자원의 효율적인 활용
3. Connection 객체 수 제어 가능

Chapter 10. JDBC 공통모듈 구현하기

common.JDBCTemplete.java

```
01  package common;
02
03  import java.sql.*;
04
05  public class JDBCTemplate {
06          public static Connection getConnection(){
07                  Connection conn = null;
08
09                  try {
10                          Class.forName("oracle.jdbc.driver.OracleDriver");
11                          conn = DriverManager.getConnection(
12                                          "jdbc:oracle:thin:@127.0.0.1:1521:xe",
13                                          "student", "student");
14                          conn.setAutoCommit(false);
15                  } catch (Exception e) {
16                          e.printStackTrace();
17                  }
18
19                  return conn;
20          }
21
22          public static void close(Connection conn){
23                  try {
24                          if(conn != null && !conn.isClosed())
25                                  conn.close();
26                  } catch (Exception e) {
27                          e.printStackTrace();
28                  }
29          }
```

```
30
31          public static void close(Statement stmt){
32                  try {
33                          if(stmt != null && !stmt.isClosed())
34                                  stmt.close();
35                  } catch (Exception e) {
36                          e.printStackTrace();
37                  }
38          }
39
40          public static void close(ResultSet rset){
41                  try {
42                          if(rset != null && !rset.isClosed())
43                                  rset.close();
44                  } catch (Exception e) {
45                          e.printStackTrace();
46                  }
47          }
48
49          public static void commit(Connection conn){
50                  try {
51                          if(conn != null && !conn.isClosed())
52                                  conn.commit();
53                  } catch (Exception e) {
54                          e.printStackTrace();
55                  }
56          }
57
58          public static void rollback(Connection conn){
59                  try {
60                          if(conn != null && !conn.isClosed())
61                                  conn.rollback();
62                  } catch (Exception e) {
```

```
63                                  e.printStackTrace();
64                          }
65                  }
66  }
```

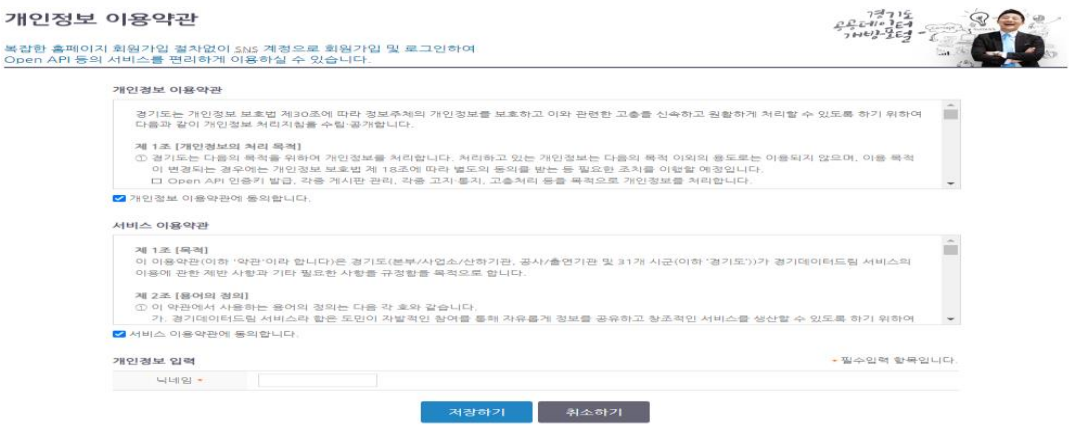
Chapter 11. 공공데이터 PROCEDURE 적용하기

경기데이터드림에서 제공하는 경기도지역 약국데이터를 기반으로 DB 구성 및 구성된 내용을 통해 프로시저 등록을 해보자.

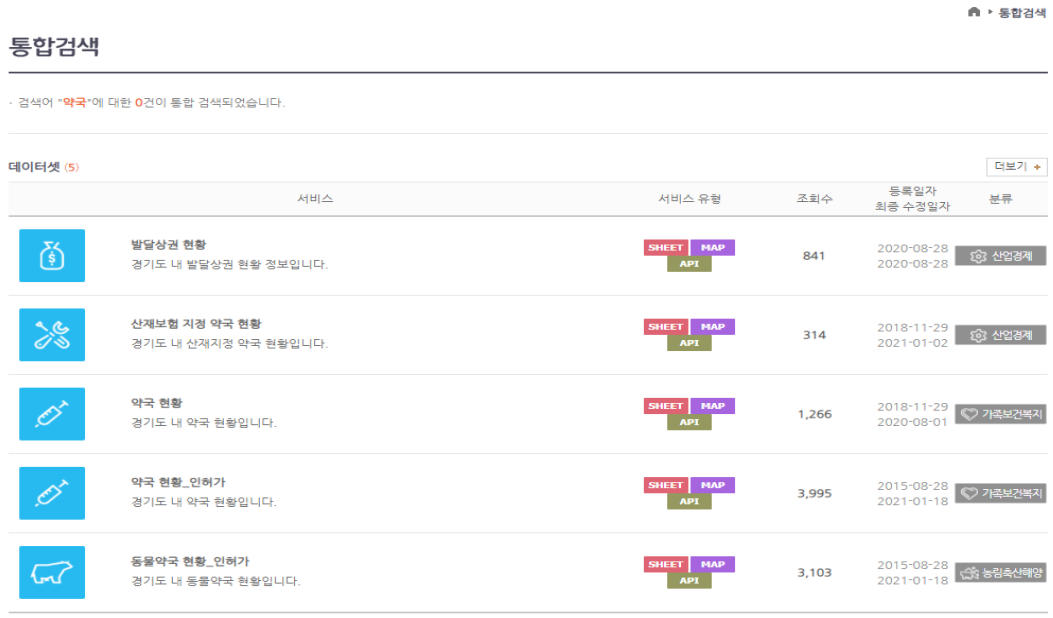
1. 경기데이터드림 공공데이터 가져오기

경기데이터그림에서 제공하는 데이터를 받아오기 위해서는 SNS 인증 및 로그인을 해야하는데 홈페이지(https://data.gg.go.kr/portal/mainPage.do)로 접속하여 소셜을 통한 로그인을 진행하면 된다. 개인정보 이용약관 동의, 닉네임을 설정한다.

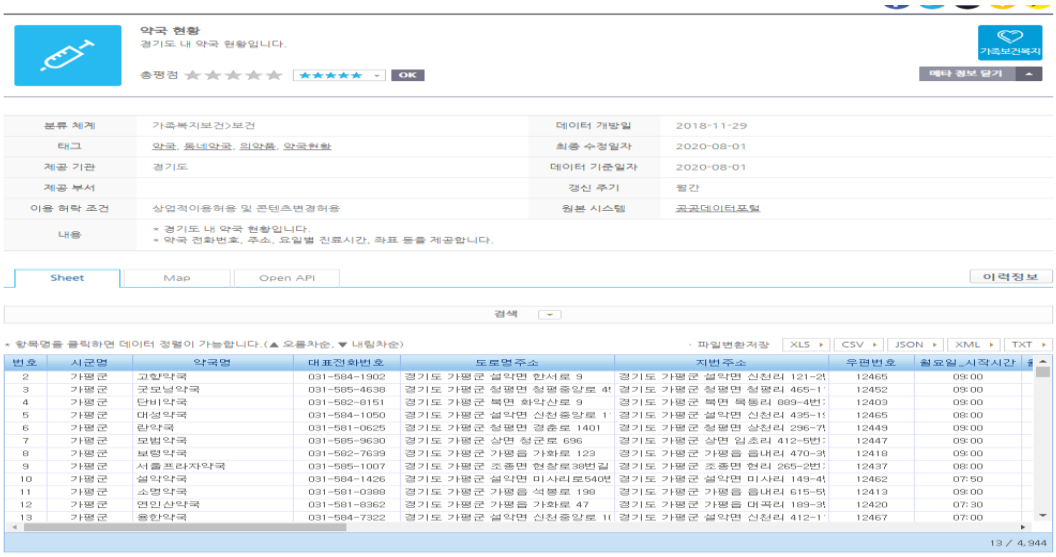




회원가입후 메인화면 조회창에서 약국을 검색 후 약국현황 데이터 셋을 클릭한다.



데이터를 받아와 처리를 해야하기 때문에 sheet 탭에서 csv 형식으로 데이터를 다운로드 한다. 다운로드시 활용목적에 대해 작성하는데 업무활용으로 선택하자.



2. 데이터 구조 분석 및 설계하기

이제 데이터를 받았으니 데이터를 DB 에 저장하기 위해 데이터 내용을 보고 구조를 분석한다. 필요한 컬럼과 필요하지 않은 컬럼을 지정하고, 데이터활용에 따라 테이블을 설계하자. 공공데이터를 기준으로 프로시저를 생성하여 활용을 DB 설계의 초점으로 맞췄다. 상황에 맞게 공공데이터를 분석하고 설계하면 된다.

TB_API_PARMACY : CSV파일 저장 테이블

TB_PARMACY : 서비스 테이블, TB_API_PARMACY + 기본키 + 등록일자 + 수정일자

TB_PRAMACY_META : HISTORY 테이블 (기본키, 등록일, 총 개수, 수정 개수, 추가 개수)





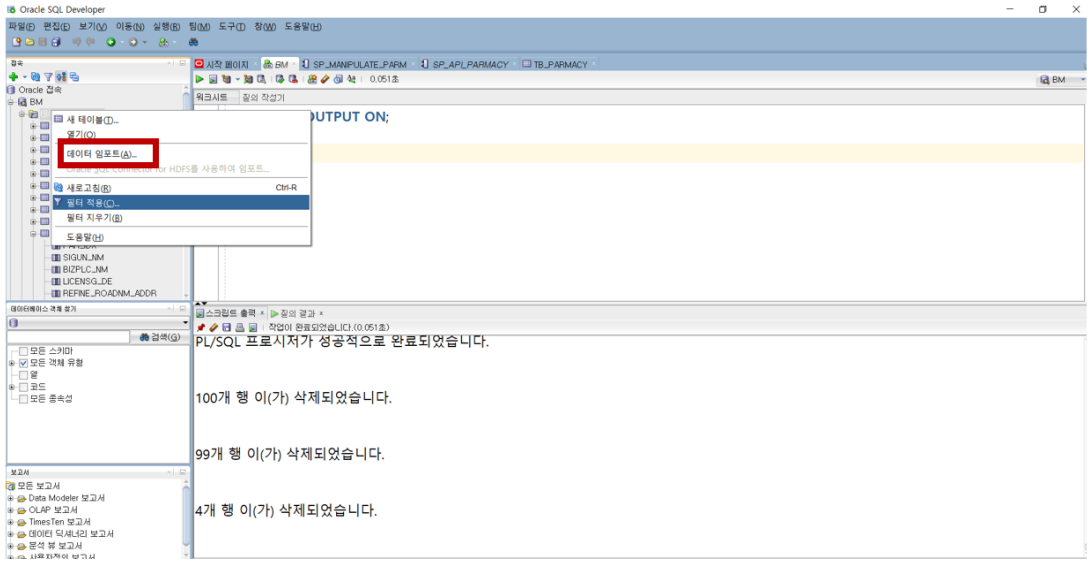
데이터 전처리하기

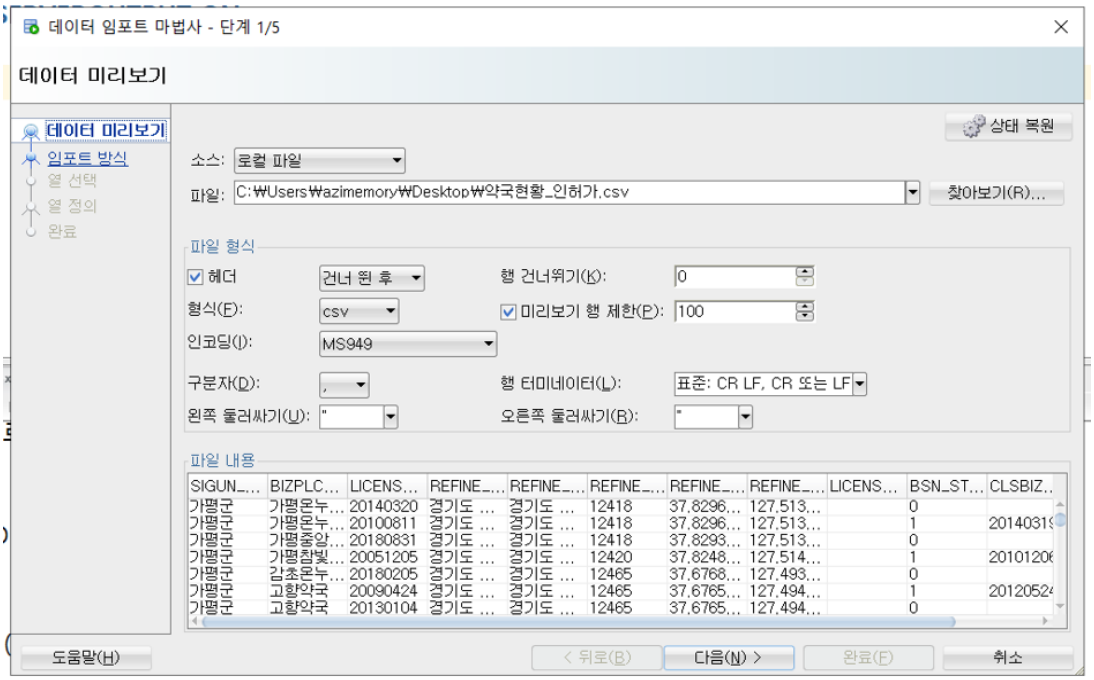
1.필요하지 않은 데이터 삭제하기 - 위치를 나타내는 좌표값, 양국영업 면적, 지정일자

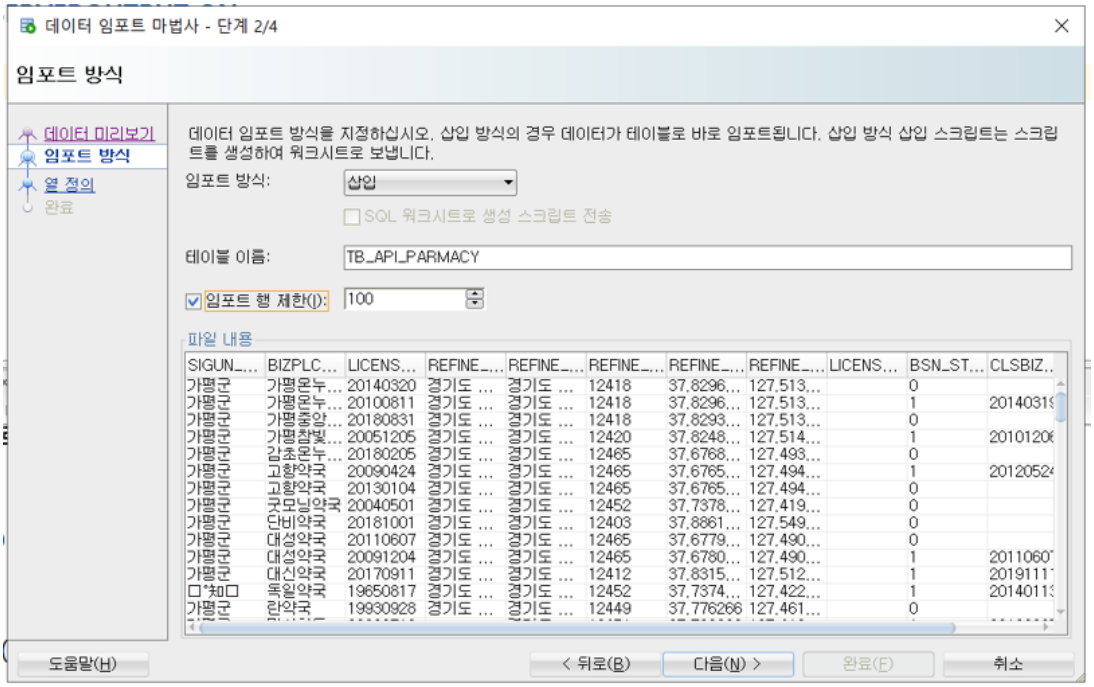
2. 테이블 컬럼명 변경하기

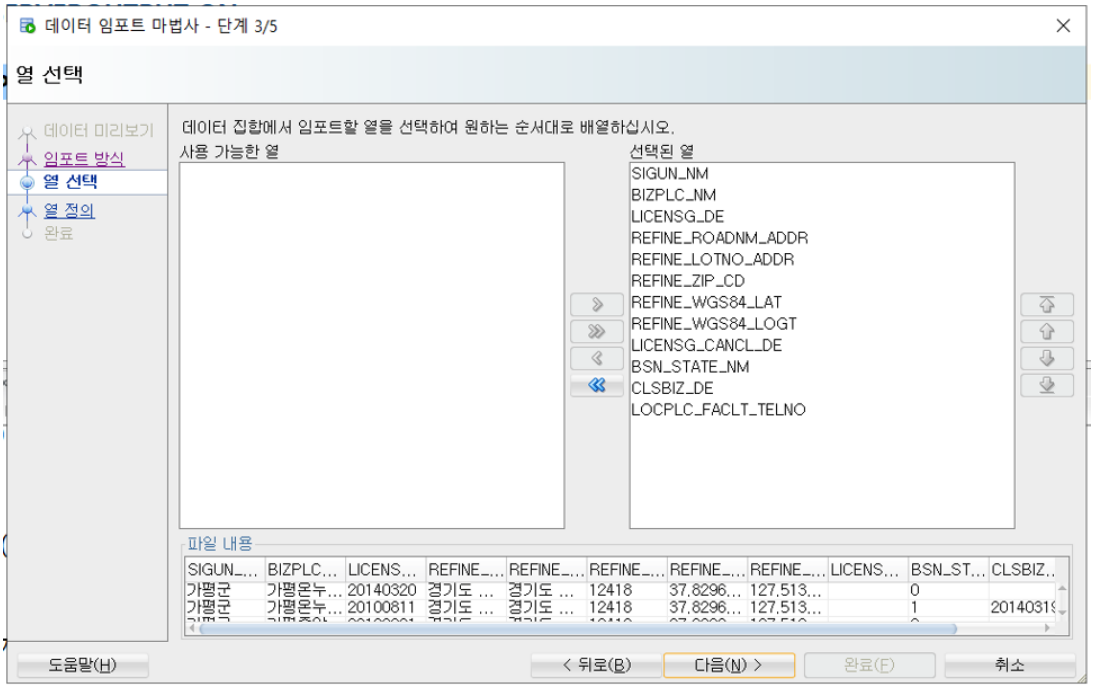
3. 데이터수정 영업상태 : 영업중 -> 0 / 영업종료 -> 1

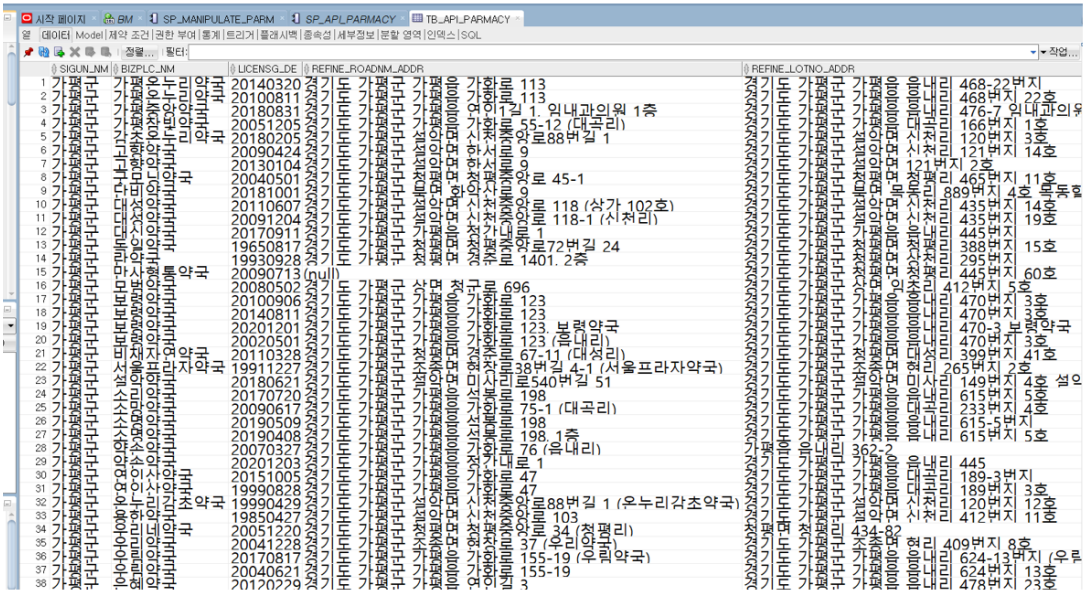
4. csv 파일 DB 에 넣기






데이터 임포트 마법사 - 단계 2/4
임포트 방식
데이터 미리보기
임포트 방식
열 정의
완료
데이터 임포트 방식을 지정하십시오. 삽입 방식의 경우 데이터가 테이블로 바로 임포트됩니다. 삽입 방식 삽입 스크립트는 스크립트를 생성하여 워크시트로 보냅니다.
임포트 방식:
삽입
SQL 워크시트로 생성 스크립트 전송
테이블 이름:
TB_API_PARMACY
임포트 행 제한(I):
100
파일 내용
SIGUN_... BIZPLC... LICENS... REFINE_... REFINE_... REFINE_... REFINE_... REFINE_... LICENS... BSN_ST... CLSBIZ...
가평군 가평온누... 20140320 경기도 ... 경기도 ... 12418 37.8296... 127.513... 0
가평군 가평온누... 20100811 경기도 ... 경기도 ... 12418 37.8296... 127.513... 1 20140319
가평군 가평중앙... 20180831 경기도 ... 경기도 ... 12418 37.8293... 127.513... 0
가평군 가평참빛... 20051205 경기도 ... 경기도 ... 12420 37.8248... 127.514... 1 20101206
가평군 감초온누... 20180205 경기도 ... 경기도 ... 12465 37.6768... 127.493... 0
가평군 고향약국 20090424 경기도 ... 경기도 ... 12465 37.6765... 127.494... 1 20120524
가평군 고향약국 20130104 경기도 ... 경기도 ... 12465 37.6765... 127.494... 0
가평군 굿모닝약국 20040501 경기도 ... 경기도 ... 12452 37.7378... 127.419... 0
가평군 단비약국 20181001 경기도 ... 경기도 ... 12403 37.8861... 127.549... 0
가평군 대성약국 20110607 경기도 ... 경기도 ... 12465 37.6779... 127.490... 0
가평군 대성약국 20091204 경기도 ... 경기도 ... 12465 37.6780... 127.490... 1 20110607
가평군 대신약국 20170911 경기도 ... 경기도 ... 12412 37.8315... 127.512... 1 20191117
□*知□ 독일약국 19650817 경기도 ... 경기도 ... 12452 37.7374... 127.422... 1 20140113
가평군 한약국 19930928 경기도 ... 경기도 ... 12449 37.776266 127.461... 0
도움말(H)
< 뒤로(B)
다음(N) >
완료(F)
취소


데이터 임포트 마법사 - 단계 3/5
열 선택
데이터 미리보기
임포트 방식
열 선택
열 정의
완료
데이터 집합에서 임포트할 열을 선택하여 원하는 순서대로 배열하십시오.
사용 가능한 열
선택된 열
SIGUN_NM
BIZPLC_NM
LICENSG_DE
REFINE_ROADNM_ADDR
REFINE_LOTNO_ADDR
REFINE_ZIP_CD
REFINE_WGS84_LAT
REFINE_WGS84_LOGT
LICENSG_CANCL_DE
BSN_STATE_NM
CLSBIZ_DE
LOCPLC_FACLT_TELNO
파일 내용
SIGUN_... BIZPLC... LICENS... REFINE_... REFINE_... REFINE_... REFINE_... REFINE_... LICENS... BSN_ST... CLSBIZ...
가평군 가평온누... 20140320 경기도 ... 경기도 ... 12418 37.8296... 127.513... 0
가평군 가평온누... 20100811 경기도 ... 경기도 ... 12418 37.8296... 127.513... 1 20140319
도움말(H)
< 뒤로(B)
다음(N) >
완료(F)
취소



	SIGUN_NM	BIZPLC_NM	LICENSG_DE	REFINE_ROADNM_ADDR	REFINE_LOTNO_ADDR
1	가평군	가평온누리약국	20140320	경기도 가평군 가평읍 가화로 113	경기도 가평군 가평읍 읍내리 468-22번지
2	가평군	가평중앙약국	20100811	경기도 가평군 가평읍 가화로 113	경기도 가평군 가평읍 읍내리 468번지 22호
3	가평군	가평중앙온누리약국	20180831	경기도 가평군 가평읍 가화로 113	경기도 가평군 가평읍 읍내리 476-7 읶내과의원 1층
4	가평군	가평참좋은약국	20051205	경기도 가평군 가평읍 가화로 55-12 (대곡리)	경기도 가평군 가평읍 대곡리 166번지 1호
5	가평군	강촌온누리약국	20180205	경기도 가평군 설악면 신천주양로88번길 1	경기도 가평군 설악면 신천리 120번지 3호
6	가평군	고려한방약국	20090424	경기도 가평군 설악면 하서로 9	경기도 가평군 설악면 신천리 121번지 14호
7	가평군	고려한방약국	20130104	경기도 가평군 설악면 하서로 9	경기도 가평군 설악면 121번지 2호
8	가평군	금강미담약국	20040501	경기도 가평군 가평읍 청평중앙로 45-1	경기도 가평군 가평읍 청평리 465번지 11호
9	가평군	금강미담약국	20181001	경기도 가평군 북면 하와산로 9	경기도 가평군 북면 목동리 889번지 4호 목동홈
10	가평군	대성약국	20110607	경기도 가평군 설악면 신천주양로 118 (상가 102호)	경기도 가평군 설악면 신천리 435번지 14호
11	가평군	대성신신약국	20091204	경기도 가평군 설악면 신천주양로 118-1 (신천리)	경기도 가평군 설악면 신천리 435번지 19호
12	가평군	대일약국	20170911	경기도 가평군 가평읍 청간내로 1	경기도 가평군 가평읍 읍내리 445번지
13	가평군	대일약국	19650817	경기도 가평군 가평읍 청평중앙로72번길 24	경기도 가평군 가평읍 청평리 388번지 15호
14	가평군	대한약국	19930928	경기도 가평군 가평읍 청평중앙로 1401. 2층	경기도 가평군 가평읍 상천리 295번지
15	가평군	만사형통약국	20090713	(null)	경기도 가평군 가평읍 청평리 445번지 60호
16	가평군	모보보보비자약국	20080502	경기도 가평군 가평읍 청군로 696	경기도 가평군 가평읍 읶초리 412번지 5호
17	가평군	보보보보비자약국	20100906	경기도 가평군 가평읍 가화로 123	경기도 가평군 가평읍 읍내리 470번지 3호
18	가평군	보보보보비자약국	20140811	경기도 가평군 가평읍 가화로 123	경기도 가평군 가평읍 읍내리 470번지 3호
19	가평군	보보보보비자약국	20201201	경기도 가평군 가평읍 가화로 123. 보령약국	경기도 가평군 가평읍 읍내리 470-3 보령약국
20	가평군	보보보보비자약국	20020501	경기도 가평군 가평읍 가화로 123 (읍내리)	경기도 가평군 가평읍 읍내리 470번지 3호
21	가평군	보보보보비자약국	20110328	경기도 가평군 청평면 경춘로 67-11 (대성리)	경기도 가평군 청평면 대성리 399번지 41호
22	가평군	서울프라자약국	19911227	경기도 가평군 조종면 현천길38번길 4-1 (서울프라자약국)	경기도 가평군 조종면 현리 265번지 2호
23	가평군	설악온누리약국	20180621	경기도 가평군 설악면 미사리로540번길 51	경기도 가평군 설악면 미사리 149번지 4호 설악
24	가평군	소나무약국	20170720	경기도 가평군 가평읍 가화로 198	경기도 가평군 가평읍 읍내리 615번지 5호
25	가평군	소나무약국	20090617	경기도 가평군 가평읍 가화로 75-1 (대곡리)	경기도 가평군 가평읍 대곡리 233번지 4호
26	가평군	소나무약국	20190509	경기도 가평군 가평읍 가화로 198	경기도 가평군 가평읍 읍내리 615-5번지
27	가평군	소나무약국	20190408	경기도 가평군 가평읍 가화로 198. 1층	경기도 가평군 가평읍 읍내리 615번지 5호
28	가평군	한솔소아과약국	20070327	경기도 가평군 가평읍 가화로 76 (읍내리)	경기도 가평군 가평읍 읍내리 362-2
29	가평군	한양약국	20201203	경기도 가평군 가평읍 장터내로 1	경기도 가평군 가평읍 읍내리 445
30	가평군	여여이산부인과약국	20151005	경기도 가평군 가평읍 가화로 47	경기도 가평군 가평읍 읍내리 189-3번지
31	가평군	여여이산부인과약국	19990828	경기도 가평군 가평읍 가화로 47	경기도 가평군 가평읍 대곡리 189번지 3호
32	가평군	온누리감초약국	19990429	경기도 가평군 설악면 신천주양로88번길 1 (온누리감초약국)	경기도 가평군 설악면 신천리 120번지 12호
33	가평군	온누리약국	19850427	경기도 가평군 설악면 신천주양로 103	경기도 가평군 설악면 신천리 412번지 11호
34	가평군	온누리네약국	20051220	경기도 가평군 청평면 청평중앙로 34 (청평리)	경기도 가평군 청평면 청평리 434-82
35	가평군	우리약국	20041228	경기도 가평군 조종면 현천길로 37 (우리약국)	경기도 가평군 조종면 현리 409번지 8호
36	가평군	우리약국	20170817	경기도 가평군 가평읍 가화로 155-19 (우리약국)	경기도 가평군 가평읍 읍내리 624-13번지 (우리
37	가평군	우리약국	20040621	경기도 가평군 가평읍 가화로 155-19	경기도 가평군 가평읍 읍내리 624번지 13호
38	가평군	우현	20120229	경기도 가평군 가평읍 여히킬 3	경기도 가평군 가평읍 읍내리 478번지 23호

3. PROCEDURE 작성

데이블 내용을 조작하는 두개의 PROCEDURE를 작성하자

1. SP_API_PARMACY

1) 기본키 생성 : 테이블의 ROW 데이터를 해싱하여 기본키 생성

2) SP_MANIPULATE_PARM 프로시저 호출

3) 등록일자, 수정 개수, 추가 개수, 총 개수 정보를 TB_PARMACY_META 테이블에 추가

SP_API_PARMACY

```
create or replace PROCEDURE SP_API_PARMACY

IS
    --기본키
    S_PAR_IDX VARCHAR2(30);
```

```
    --수정개수
    MODIFY_CNT NUMBER := 0;
    --추가개수
    ADD_CNT NUMBER := 0;
    --총 개수
    TOTAL_CNT NUMBER := 0;
BEGIN
    --IMPORT한 데이터가 들어있는 TB_API_PARMACY 의 모든 ROW를 탐색
    FOR R IN (SELECT * FROM TB_API_PARMACY)
    LOOP

        --사업체명, 사업체 등록일, 사업체 상태  데이터를 해싱하여 기본키 생성
        SELECT
        'P_IDX'                                                              ||ora_hash(R.BIZPLC_NM||R.LICENSG_DE||R.BSN_STATE_NM,99999)
        INTO S_PAR_IDX FROM DUAL;

        --수정, 추가를 판단할 SP_MANIPULATE_PARM 호출
        SP_MANIPULATE_PARM(S_PAR_IDX,MODIFY_CNT,ADD_CNT,R);

    END LOOP;

    --전체 개수
    SELECT COUNT(*) INTO TOTAL_CNT FROM TB_API_PARMACY;

    --전체 개수, 수정 개수, 추가 개수, 등록일 정보를 TB_PARMACY_META에 저장
    INSERT INTO TB_PARMACY_META VALUES
```

```
(sc_parm_meta_idx.NEXTVAL,SYSDATE,TOTAL_CNT,MODIFY_CNT,ADD_CNT)
;

    --COMMIT
   COMMIT;
END;
```

2. SP_MANIPULATE_PARM

1) 생성한 기본키의 데이터가 TB_PARMACY 테이블에 존재하는 지 확인 ROW 데이터를 해싱하여 기본키를 생성하기 때문에 데이터가 같다면 같은 기본키가 만들어 진다. 생성된 기본키를 TB_PARMACY 테이블에서 조회

2) 생성한 기본키를 가진 ROW가 TB_PARMACY에 있다면 해당 ROW를 수정 없다면 새로운 ROW를 추가

SP_MANIPULATE_PARM

```
create or replace PROCEDURE SP_MANIPULATE_PARM
(
--기본키
S_PAR_IDX IN VARCHAR2,
--수정 개수
MODIFY_CNT IN OUT NUMBER,
--추가 개수
ADD_CNT IN OUT NUMBER,
-- 탐색할 ROW
R IN TB_API_PARMACY%ROWTYPE
)
IS
```

```
--수정/입력 판단 플래그
MANIPULATE_FLAG VARCHAR2(30);
BEGIN
  -- 플래그 값 초기화
   SELECT PAR_IDX INTO MANIPULATE_FLAG
        FROM TB_PARMACY P WHERE P.PAR_IDX = S_PAR_IDX;

        -- NULL이 아니라면
        IF MANIPULATE_FLAG IS NOT NULL
        THEN
          --수정
           UPDATE TB_PARMACY P SET
                SIGUN_NM = R.SIGUN_NM
                ,BIZPLC_NM = R.BIZPLC_NM
                ,LICENSG_DE = R.LICENSG_DE
                ,REFINE_ROADNM_ADDR = R.REFINE_ROADNM_ADDR
                ,REFINE_LOTNO_ADDR = R.REFINE_LOTNO_ADDR
                ,REFINE_ZIP_CD = R.REFINE_ZIP_CD
                ,REFINE_WGS84_LAT = R.REFINE_WGS84_LAT
                ,REFINE_WGS84_LOGT = R.REFINE_WGS84_LOGT
                ,LICENSG_CANCL_DE = R.LICENSG_CANCL_DE
                ,BSN_STATE_NM = R.BSN_STATE_NM
                ,CLSBIZ_DE = R.CLSBIZ_DE
                ,LOCPLC_FACLT_TELNO = R.LOCPLC_FACLT_TELNO
                ,MODIFY_DATE = SYSDATE
                WHERE PAR_IDX = S_PAR_IDX;

           --수정 개수 1증가
           MODIFY_CNT := MODIFY_CNT + 1;
           END IF;
```

```
EXCEPTION
        -- 만약 기본키로 데이터를 찾지 못하면 NO_DATA_FOUND 예외 발
생
        WHEN OTHERS THEN
         -- 데이터 추가
         INSERT INTO
                TB_PARMACY VALUES
                (S_PAR_IDX
                , R.SIGUN_NM
                ,R.BIZPLC_NM
                ,R.LICENSG_DE
                ,R.REFINE_ROADNM_ADDR
                ,R.REFINE_LOTNO_ADDR
                ,R.REFINE_ZIP_CD
                ,R.REFINE_WGS84_LAT
                ,R.REFINE_WGS84_LOGT
                ,R.LICENSG_CANCL_DE
                ,R.BSN_STATE_NM
                ,R.CLSBIZ_DE
                ,R.LOCPLC_FACLT_TELNO
                ,SYSDATE
                ,SYSDATE
                );
            --추가 개수 1 증가
            ADD_CNT := ADD_CNT + 1;
END;
```

Appendix. 이클립스 디버깅 사용법

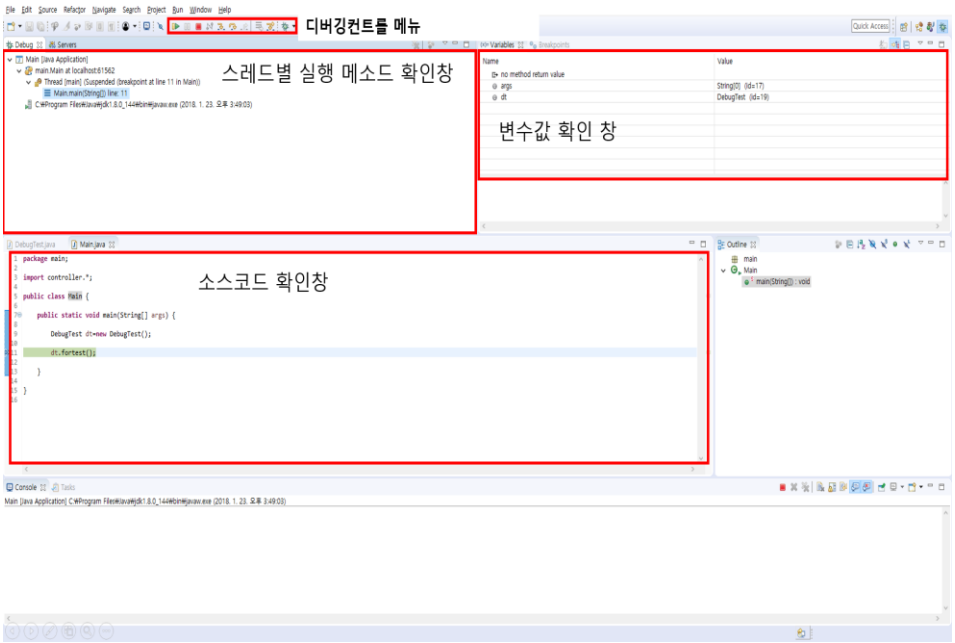
1 디버깅 이란

프로그램의 개발에서 프로그램의 오류를 발견하고 그 원인을 밝히는 과정을 말한다. 쉽게 말해 구성된프로그램의 흐름을 따라가면서 로직에 맞게 변수(객체, 배열, 기본자료형 등)에 제대로된 값이 들어가는지 확인하고 처리로직(메소드)의 매개변수, 반환값도 제대로 된 값이 나오는지 확인하는 것이다. 그리고 디버깅을 도와주는 소프트웨어가 있는데 이를 디버거라고 한다. 우리는 Eclipse에서 디버깅 하는 방법을 절차에 따라 공부해보자.

2 이클립스 디버깅 순서

문제가 될 부분에 BreakPoint설정 -> 디버그 실행(run-debug 또는 )-> 디버그모드 화면전환 -> variables에서 각변수별 저장된 값 확인

3 이클립스 디버그 모드 화면



3.1 디버깅 컨트롤 메뉴

①  **(F8) : 중단된 코드 재실행 명령**

②  **: 실행중인 코드를 중단하는 명령**

③  **: 디버깅 모드 중단 명령**

④  **(F5) : 브레이크 포인트 다음에 호출되는 소스코드 실행**
(내부에서 실행되는 메소드까지 들어가서 확인)

⑤  **(F6) : 브레이크 포인트 다음에 있는 소스코드 실행**
(내부에서 실행되는 메소드는 제외)

⑥  **(F7) : 다음 브레이크 포인트 실행**

⑦  **(F5) : 디버그모드 실행**

3.2 스레드별 메소드 확인창

각 스레드별로 BreakPoint를 설정한 메소드를 확인할 수 있는창으로 해당하는 메소드를 클릭하면 변수값을 확인하는 창이 그 메소드를 포함하고 클래스(this) 정보와 메소드에서 사용하는 변수를 보여준다.

3.3 변수값 확인창

Variables탭과 BreakPoint탭으로 나눠진다.

Variables : BreakPoint로 지정된 소스코드가 있는 범위의 모든 변수의 값을 표시해준다. Name에는 변수명이 values에는 값들이 들어간다. 일반 변수는 그냥 value값이 출력되지만 객체나 배열은 클릭해야 그안의 멤버변수 및 변수를 확인할 수 있다.

☞ 객체에 선언된 메소드는 확인할 수 없다.

BreakPoint : BreakPoint가 설정되어있는 코드를 확인할 수 있으며 앞에 있는 체크박스를 해제하면 BreakPoint가 해제된다.

4 BreakPoint 설정방법

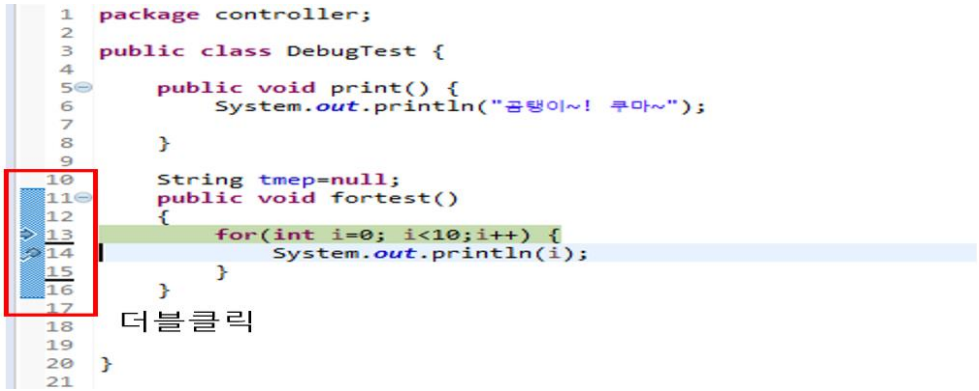
4.1 BreakPoint란

소스코드의 흐름대로 실행중인 프로그램이 진행이 되다가 중단되는 위치를 의미한다.

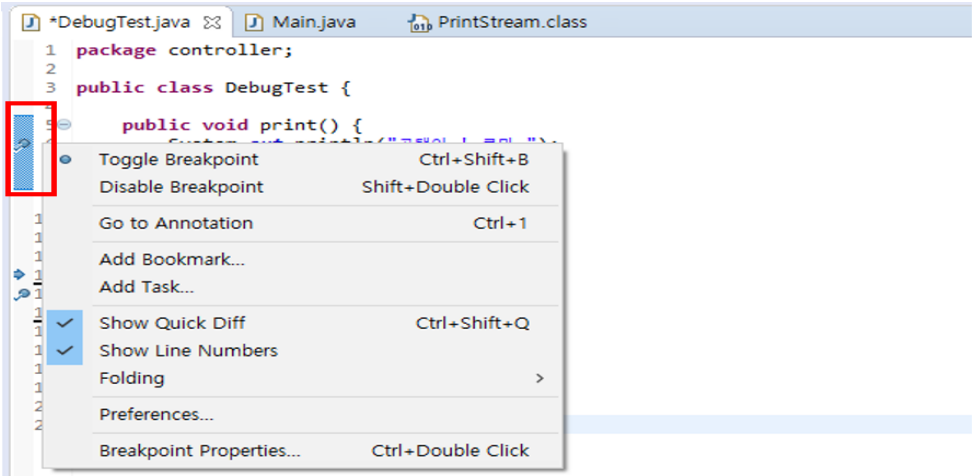
4.2 설정방법

설정방법에는 두가지가 있는데 소스코드 창에서 확인을 원하는 소스코드의 앞쪽 부분에서 더블클릭하는 방법과 우클릭하여 컨택스트메뉴를 이용하는 방법이 있다. 이렇게 설정하게 되면 해당부분의 line부분에 점같이 생긴 것이 나타나면서 BreakPoint가 생성이 되고 한번 더 더블클릭을 누르면 해제가 된다.

더블클릭 이용



우클릭 후 컨택스트 메뉴이용 (disable BreakPoint : 해제)



우클릭 후 Togle BreakPoint 선택

5 디버그 모드 실행

디버그실행 아이콘을 클릭하거나 Run-debug선택, 단축키 F11로 실행

6 for문의 증감값 변경 확인하기

main메소드에서 for문이 실행되는 fortest()메소드를 호출하여 값을 출력하는 코드

원하는 소스코드 System.out.println(i) 부분에 BreakPoint설정

소스코드

```
1  package main;
2
3  import controller.*;
4
5  public class Main {
6
7      public static void main(String[] args) {
8
9          DebugTest dt=new DebugTest();
10
11         dt.fortest();
12
13     }
14
15 }
16
```

```java
package controller;

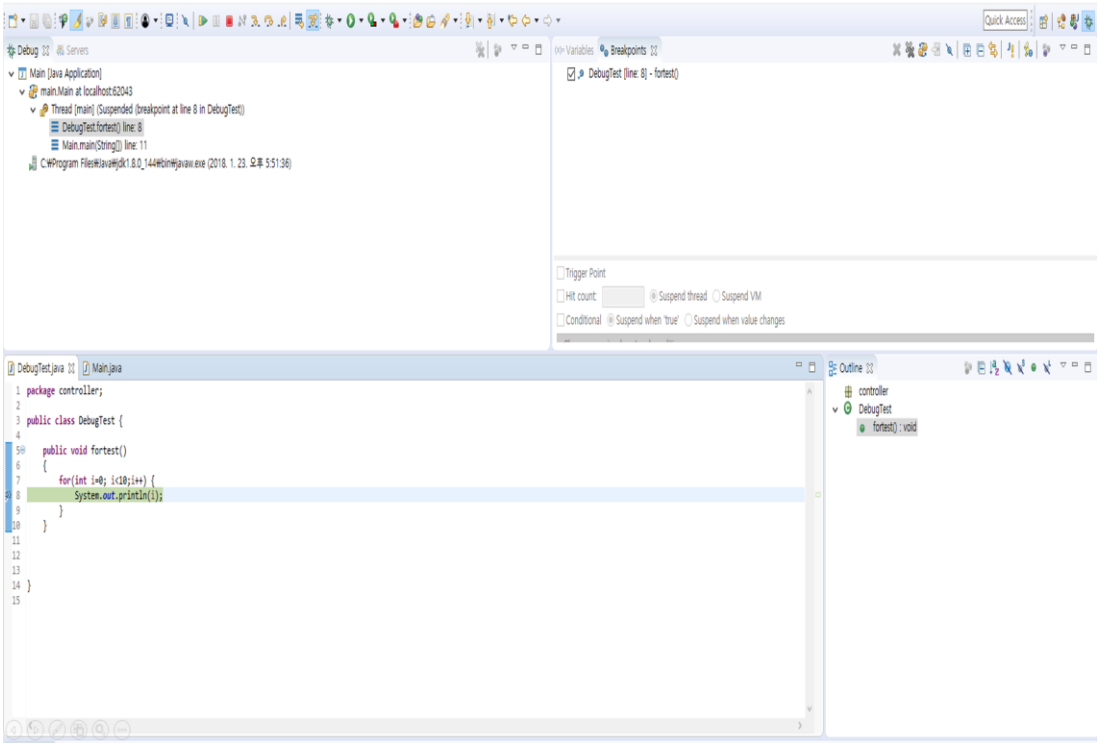
public class DebugTest {

    public void fortest()
    {
        for(int i=0; i<10;i++) {
            System.out.println(i);
        }
    }


}

```

디버그모드 실행



debug모드가 실행되면 debug모드로 화면을 변경하는지에 대한 여부를 선택하는 메시지 창이 뜨고 yes를 클릭하면그 다음 위와 같은 debug창이 나온다. 우선 debug를 실행하면 프로그램 흐름상 가장 먼저 나오는 BreakPoint지점에서 프로그램 실행이 중단되고 메소드확인창에는 실행된 메소드가 나오게 되고, variables창과

BreakPoint창에서 각각 변수과 BreakPoint리스트가 나온다. 상세화면으로 보면 아래와 같다.

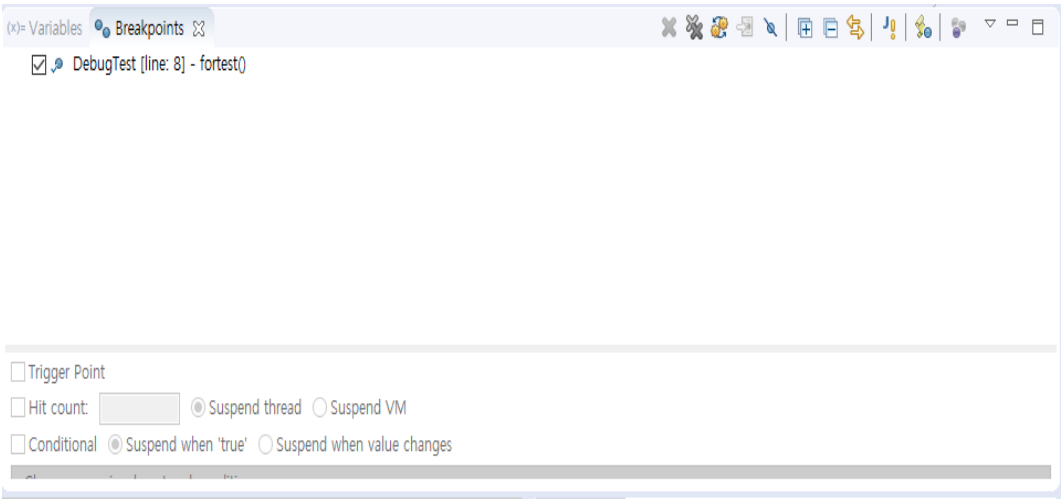
메소드창

메소드 확인 창 main 스레드에서 실행된 메소드인 main()와 fortest()를 보여준다. 프로그램의 흐름에 따라 실행되는 객체와 메소드가 클래스명.메소드방식으로 표시된다.



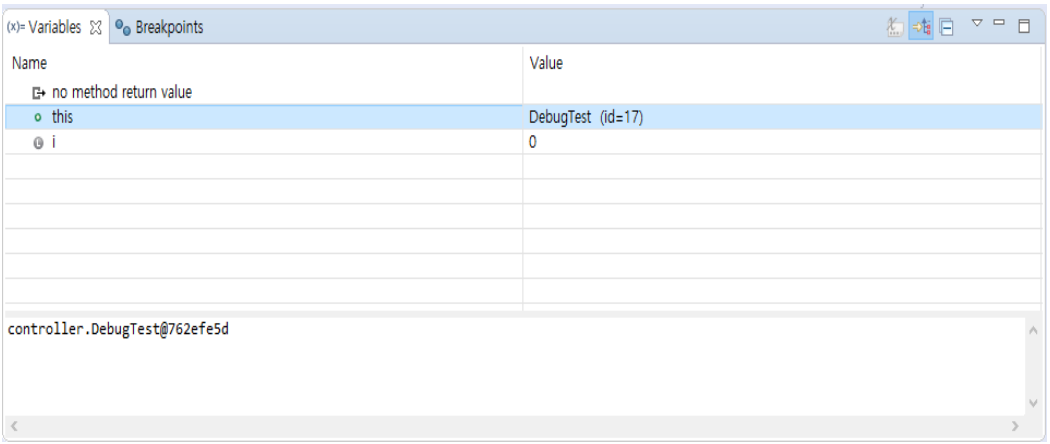
BreakPoint창

BreakPoint창에는 현재 BreakPoint가 설정되어있는 부분을 표시해준다. 현재는 BreakPoint를 DebugTest객체의 fortest메소드 내부 System.out.println에 한 개만 설정이 되어 있어서 하나만 보이는 것을 확인 할 수 있다. 그리고 옆에는 그 BreakPoint의 위치가 line으로 나와있다. 앞에 checkbox가 있는데 이부분을 체크해제하면 설정된 것이 해제 되어 더이상 이곳에서 멈추지 않고 계속 실행된다.



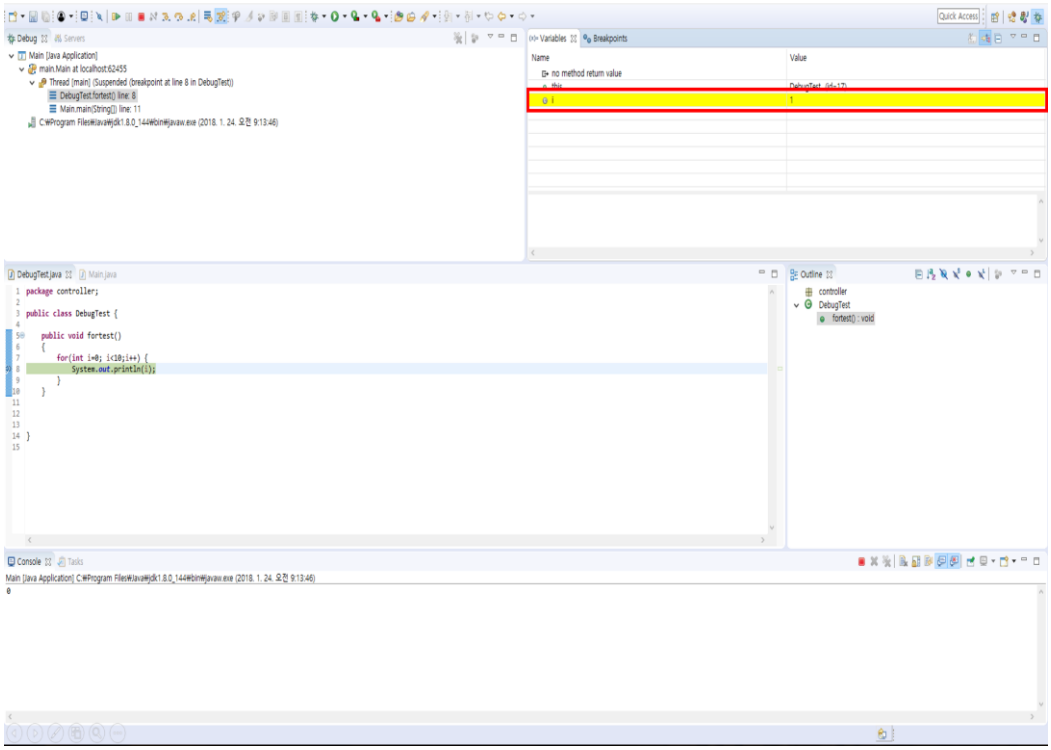
Variables창

Variables창에는 지금 선언되어 있는 객체 및 변수에 대한 정보가 나타난다. This와 i라는값이 보이는데 this는 DebugTest객체이고 value값으로 Id값이 출력되고 아래부분 출력창에는 객체의 주소값이 출력된다. i는 for문에서 선언된 지역변수이다. for이 시작되고 아직 증감연산이 되지 않았기 때문에 Value가 0으로 되어 있다.



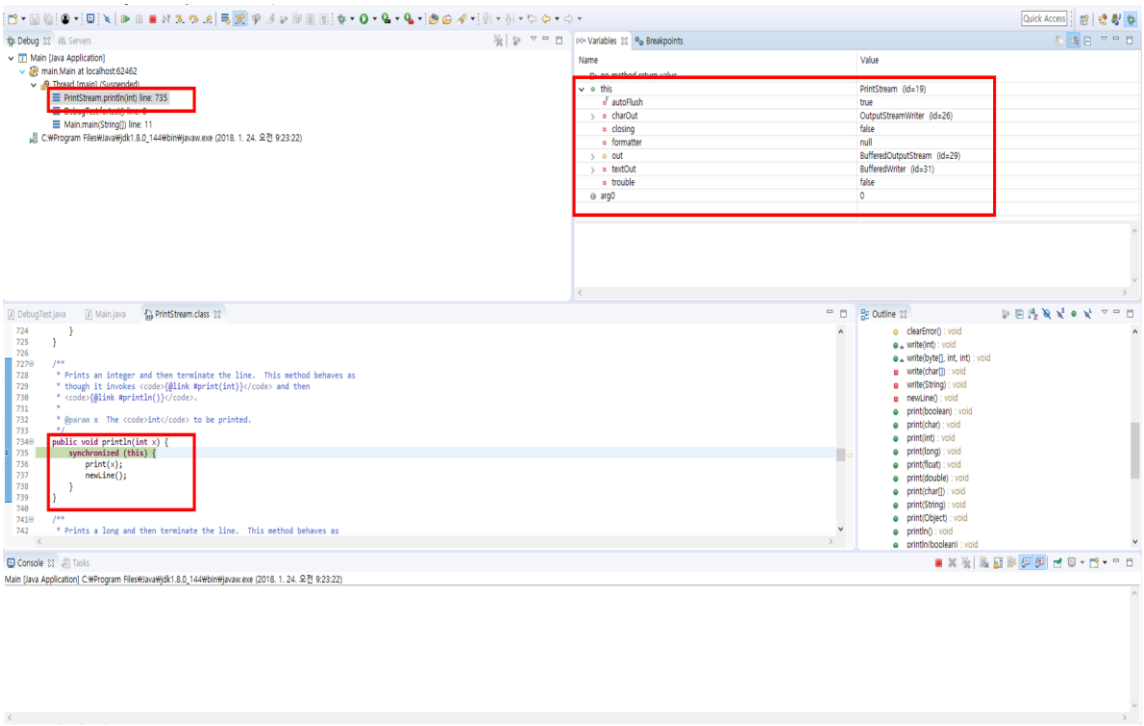
값을 확인하고 프로그램을 계속해서 실행하려면 (F8)를 클릭하고 BreakPoint다음에 실행되는 메소드로 이동하려면 (F5), (F7)을 실행하면 된다. 두개의 차이는 위 메뉴에서 설명했다.

(F8)실행



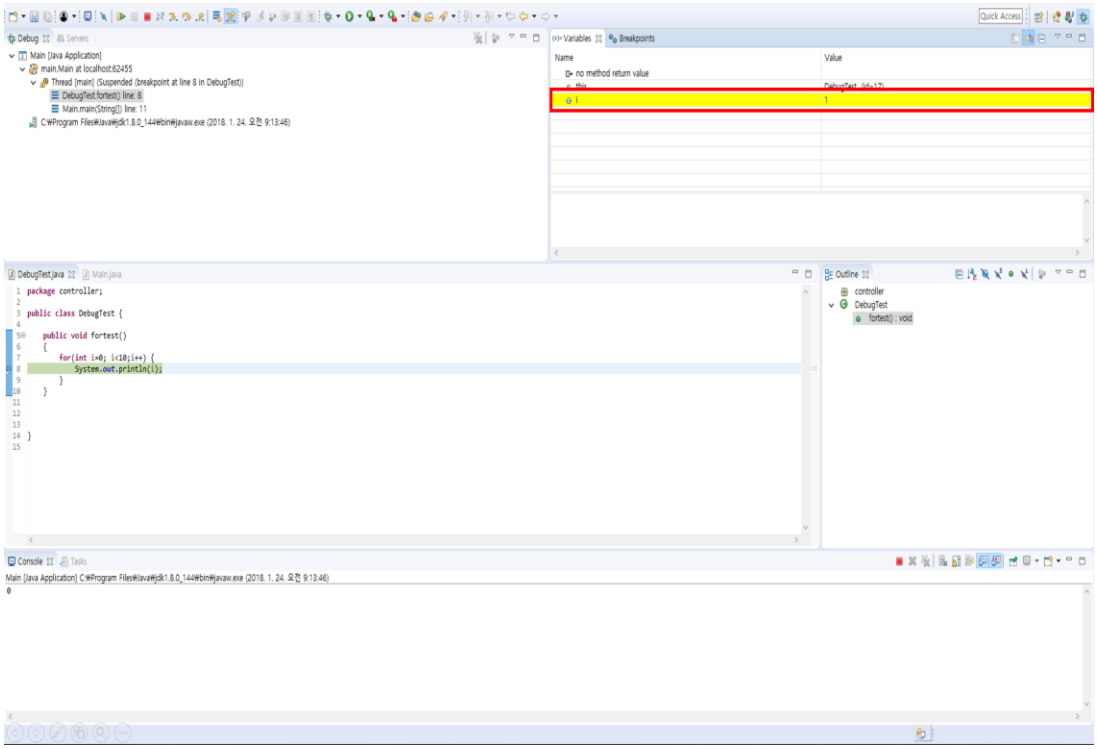
위 결과와 같이 Resume(F8)을 실행을 하면 프로그램이 이어서 실행되면서 다음 BreakPoint에서 실행을 멈추고 그 시점의 저장된 값을 출력하고 프로그램에 대한 결과를 Console에 출력해준다. 출력값과 변수값이 차이가 있는 것은 로직상에서 변수는 이미 증감연산을 했기 때문에 값이 증가하여 1이 출력되고 console창에 출력된 값은 출력된 시점이기 때문에 0으로 표시된 것이다.

(F5) 실행



위 결과와 같이 step into(F5)을 실행을 하면 BreakPoint에서 실행하는 System.out.println()메소드에 내부를 출력해주고 관련값을 확인할 수 있다. 메소드출력창을 보면 PrintStream객체의 print()매소드가 출력된걸 확인 할 수 있고, variables창을 보면 PrintStream객체가 가지고 있는 변수들의 값을 출력하는 것을 볼 수 있다. 다시 Step Into(F5)실행하면 소스코드창에서 볼 때 print(x)가 실행되는 곳으로 넘어가서 그것을 확인할 수 있다. 이후 진행은 각자가 해보자.

(F7) 실행



위 결과와 같이 step over(F6)을 실행을 하면 Step Into와 다른점은 BreakPoint에 있는 내부 메소드까지 확인하는 반면 지금 작성된 작성코드만 프로그램 흐름에 따라 진행된다. 그래서 System.out.println()메소드에는 접근하지 않고 바로 for문이 진행되는 것을 확인할 수 있다.

7 디버깅을 이용한 상속관계 및 다형성, collection정보 확인

디버깅을 이용해서 객체에 대한 정보를 더 정확하게 확인 할 수 있다. 예제에서 간단한 Person, Student객체를 만들어 Student객체가 Person을 상속하고 Person 변수에 Student를 보관하는 코드를 작성하겠다. 그리고 추가적으로 ArrayList에 Student객체를 생성해서 3개만 넣어보겠다.

간단히 각 객체에는 멤버변수만 지정하여 사용하겠다.

소스코드

Person 객체

Person.java

```
01  package kh.oop.inherit.dao;
02
03  public class Person {
04    private String name;
05    private int age;
06    private char gender;
07
08
09    public Person() {}
10    public Person(String name, int age, char gender) {
11              this.name=name;
12              this.age=age;
13              this.gender=gender;
14    }
15    public String getName() {
16              return name;
1`7   }
18    public void setName(String name) {
19              this.name = name;
20    }
21    public int getAge() {
22              return age;
23    }
24    public void setAge(int age) {
25              this.age = age;
26    }
27    public char getGender() {
28              return gender;
30    }
```

```
31    public void setGender(char gender) {
32            this.gender = gender;
33    }
34    public String toString() {
35            return name+age+gender;
36    }
37    public void print() {
38            System.out.println("Person");
39    }
40  }
```

Student 객체

Student.java

```
01  package kh.oop.inherit.dao;
02
03  public class Student extends Person{
04
05    private String studentId;
06    private String major;
07    private int grade;
08
09
10    public Student() {System.out.println("Student");}
11    public Student(String studentId, String major, int grade) {
12            this.grade=grade;
13            this.studentId=studentId;
14            this.major=major;
15    }
16
1`7   public Student(String name, int age, char gender,String studentId, String major, int
18    grade) {
19            super(name,age,gender);
20            this.grade=grade;
```

```
21              this.studentId=studentId;
22              this.major=major;
23              super.setAge(age);
24      }
25      public String getStudentId() {
26              return studentId;
27      }
28      public void setStudentId(String studentId) {
30              this.studentId = studentId;
31      }
32      public String getMajor() {
33              return major;
34      }
35      public void setMajor(String major) {
36              this.major = major;
37      }
38      public int getGrade() {
39              return grade;
40      }
41      public void setGrade(int grade) {
42              this.grade = grade;
43      }
44      public String toString() {
45              return super.toString()+studentId+grade+major;
46      }
    }
```

Main 메소드

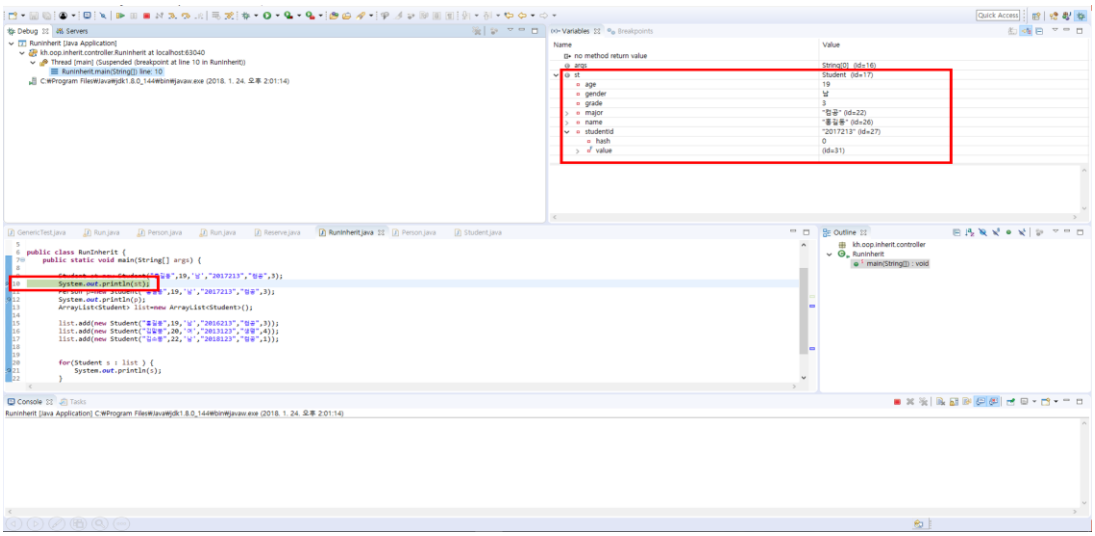
RunInherit.java

```
01  package kh.oop.inherit.controller;
02
03  import kh.oop.inherit.dao.*;
```

```
04  import java.util.*;
05
06  public class RunInherit {
07    public static void main(String[] args) {
08
09              Student st=new Student("홍길동",19,'남',"2017213","컴공",3);
10              System.out.println(st);   //Breakpoint 설정
11              Person p= new Student("홍길동",19,'남',"2017213","컴공",3);
12              System.out.println(p);   //Breakpoint 설정
13              ArrayList<Student> list=new ArrayList<Student>();
14
15              list.add(new Student("홍길동",19,'남',"2016213","컴공",3));
16              list.add(new Student("김말똥",20,'여',"2013123","생명",4));
1`7             list.add(new Student("김소똥",22,'남',"2018123","컴공",1));
18
19              System.out.println(list.get(0));
20              System.out.println(list.get(1));
21              System.out.println(list.get(2));
22
23              }
24    }
25  }
```

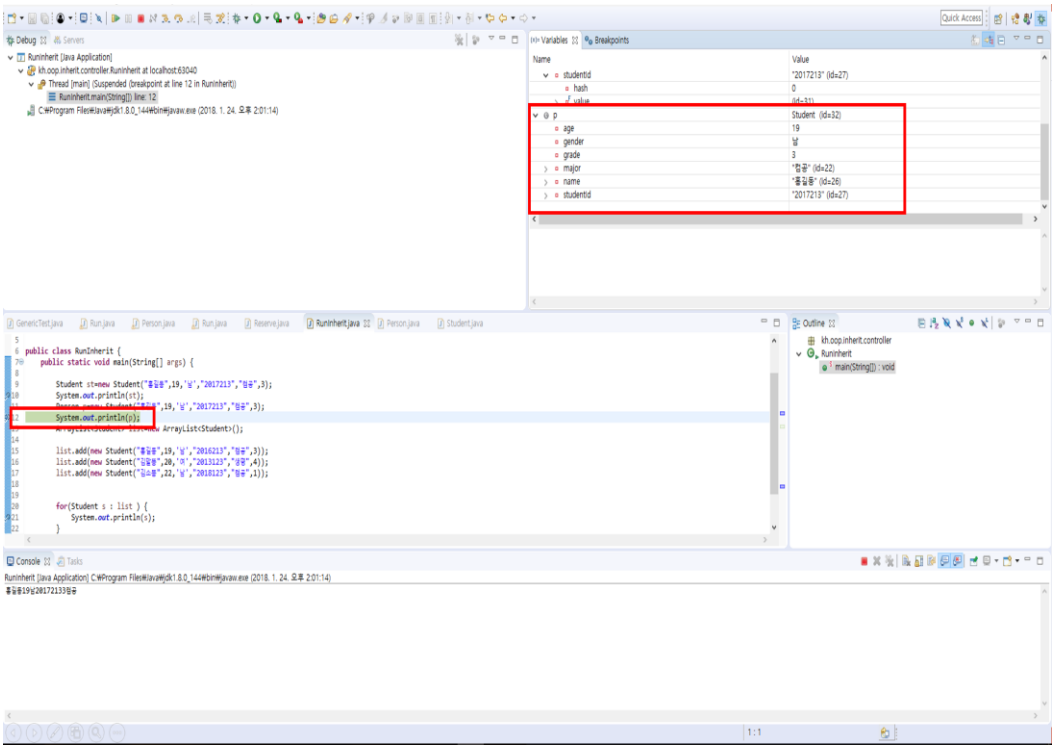
각 객체를 출력해주는 소스코드에 BreakPoint를 설정하고 Debug모드를 실행시켜보겠다.

첫번째 BreakPoint에서 student 객체확인



Student객체를 출력하는 코드에 BreakPoint를 설정 프로그램이 그 위치에서 중단된 상태이고 소스코드상에서 st변수에 Student객체가 생성되어 있어 Variables창에서 정보를 확인할 수 있다. Student객체는 Person객체를 상속받았기 때문에 Person이 가지고 있는 멤버변수도 같이 있는 것을 확인할 수 있다. 그리고 value를 보면 실제도 변수가 보관하고 있는 값을 볼 수가 있다.

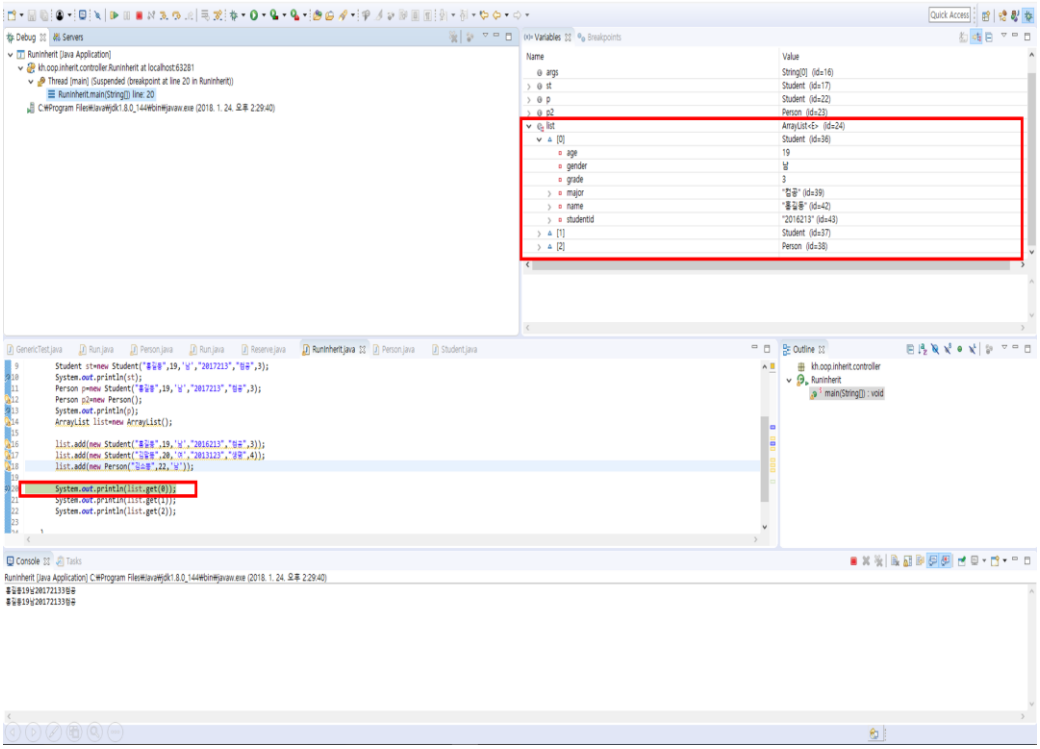
두번째 BreakPoint에서 Person p 확인



다형성을 이용하여 Person 변수 p에 Student객체가 생성하여 넣었다. 실제로 값은 Student객체가 보관하고 있는 멤버변수를 모두 가지고 있는걸 확인할 수 있다. 하지만 소스코드상에서 현재 자료형이 Person이기 때문에 값을 가지고는 있지만 Person객체가 실제로 가지고 있는 값인 name, age, gender에만 get메소드로 접근할 수 있다. 그리고 이렇게 해당하는 정보를 다 가지고 있기 때문에 형변환을 Student로 하면 저장된 정보가 있기 때문에 모든 정보를 볼수있다는 것을 눈으로 확인 할 수 있다. 그리고 p변수의 자료형은 Person이지만 value값을 보면 Student라고 나오는것도 확인 가능하다.

만약 p변수에 Person객체를 생성해서 대입하면 Person이라는 값이 나올것이다. 이건 각자가 해보자.

세번째 BreakPoint에서 ArrayList 확인



ArrayList list변수에 추가된 객체를 확인 할 수도 있다. Variables창에서 각각의 객체명, 멤버변수들의 값을 확인할 수 있다.

소스코드상에서 에러가 발생하는 경우 변수에 이상한 값이 저장되어 있어서 프로그램이 에러가 발생하는 경우가 많다. 이런것을 위와 같이 디버거를 이용하면 값을 확인하기가 쉽고 에러를 조금더 빨리 찾아낼 수 있다.